Unidad 2. Creación de aplicaciones con un framework de servidor

Programación de aplicaciones utilizando frameworks · 2.º DAW

IES Hontanilla · Tarancón

Curso 2026-2027

Índice general

Presentación de la unidad **8**

Introducción 8

Resultados de aprendizaje y criterios de evaluación 8

Temporalización 9

Entorno de trabajo 10

Convenciones de este documento 10

1 Introducción a Python **12**

Para empezar 12

1.1 Python frente a PHP y Java 12

1.1.1 El intérprete interactivo 13

Ejercicio 1.1. Primer contacto con el intérprete 14

1.2 Tipos básicos y cadenas 14

1.2.1 Tipos básicos 14

1.2.2 Cadenas 15

1.2.3 Cadenas con formato 15

Ejercicio 1.2. Cadenas y f-strings 16

1.3 Colecciones 16

Ejercicio 1.3. Colecciones 18

1.4 Control de flujo 19

1.4.1 Condicionales 19

1.4.2 Bucles 19

Ejercicio 1.4. Control de flujo 20

1.5 Funciones 21

Ejercicio 1.5. Funciones y argumentos 22

1.6 Comprensiones 23

Ejercicio 1.6. Comprensiones 24

1.7 Clases, objetos y herencia . . . 24
1.7.1 Las clases en Django . . . 26
Ejercicio 1.7. Clases y herencia . . . 26
1.8 Módulos y paquetes . . . 27
1.8.1 Ejecución como programa principal . . . 27
1.8.2 La biblioteca estándar . . . 27
1.9 Excepciones . . . 28
Ejercicio 1.8. Excepciones . . . 29
1.10 Tipado opcional . . . 29
Ejercicio 1.9. Tipado opcional . . . 30
1.11 Ejercicio integrador . . . 31
Ejercicio 1.10. Gestor de incidencias en memoria . . . 33
1.12 Resumen comparativo . . . 34

2 Estructura de un proyecto Django 35
Para empezar . . . 35
2.1 Proyecto y aplicaciones . . . 35
2.2 Creación del proyecto «Incidencias» . . . 35
Ejercicio 2.1. Creación del proyecto y de la aplicación . . . 36
2.3 La estructura generada . . . 36
2.4 El fichero de configuración `settings.py` . . . 37
2.4.1 Registrar la aplicación . . . 39
2.4.2 Idioma y zona horaria . . . 39
Ejercicio 2.2. Configuración inicial del proyecto . . . 40
2.5 La utilidad `manage.py` . . . 40
2.5.1 El servidor de desarrollo . . . 41
Ejercicio 2.3. Órdenes de administración . . . 41
2.6 El recorrido de una petición . . . 42
2.7 Control de versiones . . . 42
Ejercicio 2.4. Repositorio del proyecto . . . 43

3 Rutas, vistas y plantillas 44
Para empezar . . . 44
3.1 El sistema de rutas . . . 44
3.1.1 La función `path()` . . . 44

3.1.2 Convertidores de ruta . . . 45
3.1.3 Inclusión de rutas y espacios de nombres . . . 46
3.1.4 Inversión de rutas . . . 46
3.2 Vistas de función . . . 47
3.2.1 La petición . . . 47
3.2.2 Las respuestas . . . 47
3.2.3 Primeras vistas de la aplicación . . . 48
Ejercicio 3.1. Rutas y vistas de función . . . 49
Ejercicio 3.2. Parámetros de consulta, JSON y redirecciones . . . 49
Ejercicio 3.3. Nombres de ruta y `reverse()` . . . 50
3.3 Vistas basadas en clase . . . 51
Ejercicio 3.4. Vistas basadas en clase . . . 52
3.4 Plantillas . . . 52
3.4.1 Dónde se guardan . . . 53
3.4.2 Sintaxis del lenguaje de plantillas . . . 53
3.4.3 Etiquetas principales . . . 53
3.4.4 Filtros más usados . . . 54
Ejercicio 3.5. Primeras plantillas . . . 55
3.5 Herencia de plantillas y ficheros estáticos . . . 56
3.5.1 Ficheros estáticos . . . 57
Ejercicio 3.6. Plantilla base y estilos . . . 58
3.6 El contexto de las plantillas . . . 58
Ejercicio 3.7. Contexto, filtros y enlaces con parámetros . . . 58

4 Middleware **60**
Para empezar . . . 60
4.1 Qué es un middleware . . . 60
4.2 El middleware de serie . . . 61
4.2.1 Orden de los componentes . . . 62
Ejercicio 4.1. Identificar el middleware en las respuestas . . . 62
4.3 Middleware propio . . . 63
4.3.1 Middleware como función . . . 63
4.3.2 Middleware como clase . . . 63
4.3.3 El registro de la aplicación . . . 64
Ejercicio 4.2. Middleware que mide el tiempo de respuesta . . . 65

4.4 Middleware que corta el recorrido . . . 65
Ejercicio 4.3. Orden del middleware y respuestas cortadas . . . 66
4.5 Middleware y `request.user` . . . 66
Ejercicio 4.4. Dependencias entre capas . . . 67
4.6 Otros métodos del middleware . . . 67

5 Modelos y migraciones **69**
Para empezar . . . 69
5.1 El ORM de Django . . . 69
5.2 Definición de modelos . . . 70
5.2.1 Tipos de campo . . . 70
5.2.2 Opciones de los campos . . . 70
5.2.3 Opciones cerradas . . . 71
5.2.4 Claves ajenas . . . 71
5.2.5 El modelo de usuario . . . 72
5.2.6 La clase `Meta` . . . 72
5.2.7 Los modelos de «Incidencias» . . . 73
5.3 Migraciones . . . 74
Ejercicio 5.1. Primer modelo y primera migración . . . 75
Ejercicio 5.2. Modelo con relaciones . . . 75
5.4 Modificar modelos existentes . . . 76
5.4.1 Campos nuevos en tablas con datos . . . 76
5.4.2 Deshacer una migración . . . 76
Ejercicio 5.3. Añadir un campo y deshacer el cambio . . . 77
5.5 Buenas prácticas con las migraciones . . . 77
Ejercicio 5.4. Historial de migraciones . . . 78

6 Consultas, administración y formularios **79**
Para empezar . . . 79
6.1 La consola de Django . . . 79
6.1.1 El gestor `objects` . . . 79
Ejercicio 6.1. Primeros objetos desde la consola . . . 80
6.1.2 Órdenes de administración propias . . . 81
Ejercicio 6.2. Orden `cargar_datos` . . . 82
6.2 Consultas con QuerySets . . . 83

6.2.1 Los métodos básicos . . . 83
6.2.2 Evaluación perezosa . . . 84
6.2.3 Búsquedas por campo (*lookups*) . . . 84
6.2.4 Consultas a través de relaciones . . . 84
Ejercicio 6.3. Consultas básicas . . . 85
6.2.5 Objetos Q y F . . . 85
6.2.6 Agregaciones y anotaciones . . . 86
6.2.7 El problema de las N+1 consultas . . . 86
Ejercicio 6.4. Consultas avanzadas y número de consultas . . . 87
6.3 El panel de administración . . . 87
6.3.1 Registrar los modelos . . . 88
Ejercicio 6.5. Panel de administración . . . 89
6.4 Vistas que consultan la base de datos . . . 90
6.4.1 Vistas de función con el ORM . . . 90
6.4.2 Vistas genéricas de listado y detalle . . . 90
6.4.3 Plantillas con objetos del modelo . . . 92
Ejercicio 6.6. Listado y detalle desde la base de datos . . . 93
6.5 Formularios . . . 94
6.5.1 El proceso de validación . . . 95
6.5.2 Validación en el modelo . . . 95
6.5.3 Vista de creación . . . 95
6.5.4 La plantilla del formulario . . . 96
Ejercicio 6.7. Formulario de alta de incidencias . . . 97

7 API REST con Django REST Framework 99
Para empezar . . . 99
7.1 Arquitectura REST . . . 99
7.1.1 Operaciones sobre un recurso . . . 100
7.1.2 Códigos de estado . . . 100
7.2 Django REST Framework . . . 101
7.2.1 Instalación . . . 101
7.3 Serializadores . . . 102
7.3.1 Los serializadores no llaman a `clean()` del modelo . . . 104
Ejercicio 7.1. Serializadores desde la consola . . . 104
7.4 ViewSets y routers . . . 105

7.4.1 El router . . . 106
Ejercicio 7.2. Primera API . . . 107
7.5 Operaciones de escritura . . . 108
Ejercicio 7.3. Crear, modificar y borrar desde `curl.exe` . . . 108
7.6 Filtros, paginación y formatos . . . 110
7.6.1 Filtros por parámetros de consulta . . . 110
7.6.2 Paginación . . . 110
Ejercicio 7.4. Paginación de un listado . . . 111
7.6.3 La API navegable . . . 112
Ejercicio 7.5. Filtros y API navegable . . . 112
7.7 Pruebas de una API sin navegador . . . 112

8 Seguridad de la aplicación 114
Para empezar . . . 114
8.1 Autenticación y autorización . . . 114
8.2 Contraseñas . . . 115
8.2.1 Validadores de contraseñas . . . 115
Ejercicio 8.1. Hash y validación de contraseñas . . . 116
8.3 Inicio de sesión y vistas protegidas . . . 116
8.3.1 Las vistas de autenticación de Django . . . 116
8.3.2 Proteger vistas . . . 118
Ejercicio 8.2. Inicio y cierre de sesión . . . 119
8.4 Protección CSRF . . . 119
Ejercicio 8.3. Peticiones sin token CSRF . . . 120
8.5 Autenticación por token en la API . . . 120
Ejercicio 8.4. Tokens de acceso . . . 121
8.6 Permisos en la API . . . 122
Ejercicio 8.5. Permiso «solo el autor» . . . 124
8.7 Validación de entrada, inyección SQL y XSS . . . 124
8.7.1 Validar siempre en el servidor . . . 124
8.7.2 Inyección SQL . . . 125
8.7.3 XSS y el escapado automático . . . 125
Ejercicio 8.6. Inyección SQL y XSS . . . 125
8.8 CORS . . . 126
8.8.1 La política del mismo origen . . . 126

8.8.2 django-cors-headers 126
Ejercicio 8.7. Cabeceras CORS 127
8.9 Comprobación de despliegue y secretos 128
Ejercicio 8.8. Revisión de seguridad 128

9 Cierre de la unidad **130**
9.1 Resumen 130
9.2 Glosario 131
9.3 Preguntas de autoevaluación 132
9.4 Referencias 133

Presentación de la unidad

Introducción

En la Unidad 1 se instaló Django y se programó una primera página con una vista y una ruta. Esta unidad desarrolla por completo la parte de **servidor** (*back-end*) de una aplicación web con Django: desde la sintaxis del lenguaje hasta una API REST protegida que podrá consumir un cliente escrito con otro framework.

Todos los ejemplos construyen paso a paso la misma aplicación, **«Incidencias»**, un gestor de incidencias técnicas de un centro educativo: el profesorado registra averías (un proyector que no enciende, una wifi que no funciona) y el equipo de mantenimiento las clasifica, las atiende y las marca como resueltas. La misma aplicación crecerá en las unidades siguientes con un cliente en Vue, un despliegue con Docker, una interfaz con PrimeVue y una batería de pruebas.

La unidad sigue el orden de los contenidos de la programación del módulo y tiene ocho apartados:

1. **Introducción a Python.** Las características de la sintaxis del lenguaje de servidor (tipos, colecciones, control de flujo, funciones, clases, módulos, excepciones y tipado opcional), comparadas con PHP y Java.
2. **Estructura de un proyecto Django.** El proyecto y sus aplicaciones, el fichero de configuración `settings.py` y las órdenes de `manage.py`.
3. **Rutas, vistas y plantillas.** Definición de rutas y controladores: rutas con parámetros, vistas de función y basadas en clase, y plantillas con herencia.
4. **Middleware.** Qué es, cómo se ordena, qué hacen los que trae Django y cómo se programa uno propio.
5. **Modelos y migraciones.** El ORM de Django, los tipos de campo, las relaciones y las migraciones de la base de datos.
6. **Consultas, administración y formularios.** Consultas con *QuerySets*, el panel de administración, las vistas genéricas y los formularios basados en modelos.
7. **API REST con Django REST Framework.** Serializadores, *ViewSets*, *routers*, filtros, paginación y códigos de estado HTTP.
8. **Seguridad de la aplicación.** Autenticación, permisos, tokens, CSRF, contraseñas, inyección SQL, XSS, CORS y la comprobación de despliegue.

Resultados de aprendizaje y criterios de evaluación

La unidad desarrolla por completo el resultado de aprendizaje 1, iniciado en la Unidad 1.

RA	Descripción	Peso en la nota final
RA1	Crea aplicaciones BackEnd utilizando framework de servidor	22 %

Criterio	Contenido	Apartado
RA1.a	Estructura base de un proyecto creado con un framework de servidor (incluida la sintaxis del lenguaje)	1, 2
RA1.b	Creación de rutas y controladores (vistas de función, vistas basadas en clase, plantillas y formularios)	3, 6.4, 6.5
RA1.c	Utilización de distintos elementos *middleware*	4, 8.7
RA1.d	Migraciones de bases de datos con el framework	5
RA1.e	Consultas a bases de datos	6.1 a 6.3
RA1.f	Implementación de una API RESTful	7
RA1.g	Medidas de securización de la aplicación	8

Temporalización

Sesiones	Contenido
S05–S06	Apartados 1.1 a 1.5. Python (I): tipos, cadenas, colecciones y control de flujo
S07–S08	Apartados 1.6 a 1.11. Python (II): funciones, comprensiones, clases, módulos, excepciones y tipado
S09–S10	Apartados 2 y 3.1 a 3.4. Estructura del proyecto, rutas y vistas
S11–S12	Apartados 3.5 a 3.7 y 4. Plantillas y *middleware*
S13–S14	Apartado 5. Modelos, ORM y migraciones
S15–S16	Apartado 6. Consultas, panel de administración, vistas genéricas y formularios
S17–S18	Apartado 7. API REST con Django REST Framework
S19–S20	Apartado 8. Seguridad de la aplicación

Entorno de trabajo

Se trabaja en **Windows 10 u 11**, directamente sobre el equipo, con lo instalado en la Unidad 1 más dos paquetes nuevos que se instalan con pip dentro del entorno virtual del proyecto:

Herramienta	Versión	Uso
Python	3.13 (válidas de la 3.10 a la 3.13)	Lenguaje del servidor
Django	5.2 LTS	Framework de servidor
Django REST Framework	3.16	API REST (apartado 7)
django-cors-headers	4.x	Peticiones desde otro origen (apartado 8)
SQLite	La incluida en Python	Base de datos de desarrollo
Visual Studio Code	Actual, con la extensión Python	Editor
`curl.exe`	El incluido en Windows	Peticiones HTTP desde la terminal
Navegador	Firefox, Chrome o Edge	Pruebas de las páginas y de la API navegable

Los proyectos se guardan en `$HOME\proyectos\pauf`, una carpeta sin espacios, sin tildes y fuera de OneDrive, como en la Unidad 1. La aplicación de ejemplo se crea en `$HOME\proyectos\pauf\incidencias`.

Django 5.2 no garantiza su funcionamiento con Python 3.14 o posterior. Si el equipo tiene una versión más reciente, hay que instalar también Python 3.13 y crear el entorno virtual con ella (`py -3.13 -m venv .venv`).

Convenciones de este documento

- El texto en `fuente monoespaciada` es lo que se escribe tal cual: comandos, nombres de fichero, direcciones, código.
- Los comandos se escriben en la terminal **PowerShell** de Windows. Cuando un bloque muestra un comando precedido de `>`, el carácter `>` no se escribe: representa el indicador de la terminal. En el intérprete interactivo de Python el indicador es `>>>` y tampoco se escribe.
- Salvo que se diga lo contrario, los comandos de Django se ejecutan desde la carpeta del proyecto (`incidencias`) y con el entorno virtual activado: el indicador debe empezar por `(.venv)`.

- La primera línea de cada bloque de código es un comentario con la ruta del fichero, relativa a la carpeta del proyecto (por ejemplo, `# incidencias/models.py`). En las plantillas el comentario tiene la forma `{# ... #}`.
- Cuando un fichero ya apareció antes, solo se muestran sus partes nuevas o modificadas, y se indica así en el texto.
- Cada ejercicio termina con un apartado **Verificación**, que indica el resultado exacto que se debe obtener. No se pasa al ejercicio siguiente hasta obtenerlo.
- Los bloques **Para reflexionar** plantean una pregunta para pensar o comentar en clase; no tienen una única respuesta correcta.

Capítulo 1

Introducción a Python

Para empezar

1. ¿Qué diferencias de sintaxis recordáis entre PHP y Java para declarar una variable, escribir un bucle o definir una clase?
2. Si un lenguaje no obliga a declarar el tipo de las variables, ¿cuándo se detecta que una operación entre tipos no tiene sentido?
3. En PHP y en Java los bloques de código se delimitan con llaves. ¿Qué otra forma se os ocurre de indicar dónde empieza y dónde acaba un bloque?

1.1 Python frente a PHP y Java

Django está escrito en **Python** y las aplicaciones Django se programan en Python. Ya sabéis programar en PHP y en Java, de modo que este apartado no explica qué es una variable o un bucle: explica cómo se escriben en Python y qué lo diferencia de los lenguajes que ya conocéis.

Característica	PHP	Java	Python
Ejecución	Interpretado	Compilado a *bytecode* (JVM)	Interpretado (compila a *bytecode* de forma transparente)
Tipado	Dinámico	Estático: el tipo se declara	Dinámico y **fuerte**: no se declara, pero no se mezclan tipos sin convertir
Variables	`$nombre = "Ana";`	`String nombre = "Ana";`	`nombre = "Ana"`
Fin de instrucción	`;`	`;`	Salto de línea
Bloques	Llaves `{ }`	Llaves `{ }`	Dos puntos `:` y **sangría**

Característica	PHP	Java	Python
Comentarios	`//` y `/* */`	`//` y `/* */`	`#` (y cadenas de documentación `"""..."""`)
Nulo	`null`	`null`	`None`
Booleanos	`true`, `false`	`true`, `false`	`True`, `False` (con mayúscula)
Operadores lógicos	`&&`, `||`, `!`	`&&`, `||`, `!`	`and`, `or`, `not`
Gestor de paquetes	Composer	Maven, Gradle	pip

Tres ideas resumen el estilo del lenguaje:

- **La sangría es sintaxis.** Un bloque empieza tras los dos puntos (`:`) y abarca las líneas siguientes que tienen más sangría. Por convenio se usan **cuatro espacios** (nunca tabuladores mezclados con espacios). Una sangría incorrecta produce un error `IndentationError`.
- **Tipado dinámico pero fuerte.** Una variable puede contener primero un número y después una cadena, pero `"3" + 4` es un error, a diferencia de PHP, que convertiría el texto en número.
- **Legibilidad.** La guía de estilo oficial, **PEP 8**, fija convenios que se siguen en todo el código Python, incluido Django: nombres de variables y funciones en minúsculas con guiones bajos (`hora_inicio`, *snake_case*), clases con mayúscula inicial en cada palabra (`IncidenciaHardware`, *CamelCase*) y constantes en mayúsculas (`PRIORIDADES`).

1.1.1 El intérprete interactivo

Python incluye un **intérprete interactivo** (también llamado REPL, *Read-Eval-Print Loop*): se escribe una expresión y muestra su resultado al instante. Es la forma más rápida de probar algo del lenguaje. Se abre con `python` y se cierra con `exit()`:

```
> python
Python 3.13.x (...) on win32
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> 2 ** 10
1024
>>> exit()
```

Los programas se guardan en ficheros con extensión `.py` y se ejecutan con `python fichero.py`.

Ejercicio 1.1. Primer contacto con el intérprete

Objetivo: comprobar en el intérprete interactivo el tipado dinámico y fuerte de Python.

1. Cread la carpeta `$HOME\proyectos\pauf\python_basico` y abrid en ella una terminal. Todos los ejercicios de este apartado se guardan en esa carpeta. No hace falta entorno virtual: solo se usa la biblioteca estándar.
2. Abrid el intérprete con `python` y escribid, una a una, estas líneas:

```
>>> x = 3
>>> type(x)
>>> x = "tres"
>>> type(x)
>>> "3" + 4
>>> int("3") + 4
>>> "3" + str(4)
>>> 0.1 + 0.2 == 0.3
```

3. Salid con `exit()`.

Verificación. `type(x)` muestra primero `<class 'int'>` y después `<class 'str'>`. La expresión `"3" + 4` produce `TypeError: can only concatenate str (not "int") to str`. `int("3") + 4` da `7` y `"3" + str(4)` da `'34'`. La última comparación da `False`, como en Java y en PHP, porque los números decimales en coma flotante no son exactos.

1.2 Tipos básicos y cadenas

1.2.1 Tipos básicos

Tipo	Ejemplos	Equivalente en Java
`int`	`42`, `-7`, `1_000_000`	`int`, `long` (en Python no tiene límite de tamaño)
`float`	`3.14`, `2.5e3`	`double`
`bool`	`True`, `False`	`boolean`
`str`	`"hola"`, `'hola'`	`String`
`NoneType`	`None`	`null`

Las conversiones se hacen llamando al tipo como si fuera una función: `int("42")`, `float("2.5")`, `str(42)`, `bool(0)`. La función `type()` devuelve el tipo de un valor e `isinstance(valor, tipo)` comprueba si un valor es de un tipo.

Algunos operadores no existen en PHP o en Java, o se escriben de otra forma:

Operación	Python	Resultado
División real	`7 / 2`	`3.5` (siempre `float`)
División entera	`7 // 2`	`3`
Potencia	`2 ** 3`	`8`
Comparaciones encadenadas	`0 < nota <= 10`	`True` si se cumplen las dos
Igualdad de valor	`a == b`	Compara el contenido (como `equals()` en Java)
Identidad	`a is None`	Comprueba si es el mismo objeto; se usa sobre todo con `None`
Pertenencia	`"wifi" in texto`	`True` si contiene el elemento

1.2.2 Cadenas

Las cadenas (`str`) se escriben con comillas simples o dobles indistintamente; las comillas triples (`"""..."""`) permiten varias líneas. Son **inmutables**: los métodos no modifican la cadena, devuelven otra nueva.

Operación	Ejemplo	Resultado
Longitud	`len("aula")`	`4`
Índice	`"aula"[0]`, `"aula"[-1]`	`'a'`, `'a'` (los índices negativos cuentan desde el final)
Porción (*slice*)	`"Proyector"[0:4]`, `"Proyector"[4:]`	`'Proy'`, `'ector'`
Mayúsculas y minúsculas	`"Alta".upper()`, `"Alta".lower()`	`'ALTA'`, `'alta'`
Quitar espacios extremos	`"  hola ".strip()`	`'hola'`
Dividir y unir	`"a,b".split(",")`, `"-".join(["a", "b"])`	`['a', 'b']`, `'a-b'`
Reemplazar	`"aula 1".replace("1", "2")`	`'aula 2'`
Empieza o termina por	`"foto.png".endswith(".png")`	`True`

La notación de porciones `[inicio:fin]` toma desde `inicio` (incluido) hasta `fin` (excluido); si se omite uno de los dos, se toma desde el principio o hasta el final. Funciona igual en listas y tuplas.

1.2.3 Cadenas con formato

Una **f-string** es una cadena precedida de la letra `f` en la que se pueden insertar expresiones entre llaves. Es la forma habitual de componer texto en Python, equivalente a la interpolación `"Hola $nombre"` de PHP o a `String.format()` en Java:

```
# python_basico/cadenas.py
titulo = "  Proyector del aula 12 no enciende  "
limpio = titulo.strip()

print(limpio.upper())
print(len(limpio))
print(limpio[:9])
print(limpio[-8:])
print(limpio.split()[2])
print(limpio.replace("aula", "laboratorio"))
print("aula" in limpio)

prioridad = "alta"
horas = 2.5
print(f"Prioridad: {prioridad.capitalize()}, estimación: {horas:.2f} h")
print(f"[{'Estado':<10}][{'abierta':>10}]")
```

Después de los dos puntos de una llave se puede indicar un **formato**: `:.2f` muestra dos decimales; `:<10` y `:>10` alinean el texto a la izquierda o a la derecha en un ancho de 10 caracteres.

Ejercicio 1.2. Cadenas y f-strings

Objetivo: practicar los métodos de cadena, las porciones y las f-strings.

1. Guardad el código anterior en `cadenas.py` y ejecutadlo con `python cadenas.py`.
2. Añadid al final dos líneas que muestren, con una f-string, el texto `Caracteres: 33, palabras: 6`, calculando los dos números a partir de `limpio` (no escribiéndolos a mano).

Verificación. La salida completa es:

```
PROYECTOR DEL AULA 12 NO ENCIENDE
33
Proyector
enciende
aula
Proyector del laboratorio 12 no enciende
True
Prioridad: Alta, estimación: 2.50 h
[Estado    ][   abierta]
Caracteres: 33, palabras: 6
```

La última línea se obtiene con `print(f"Caracteres: {len(limpio)}, palabras: {len(limpio.split())}")`, o con dos variables intermedias.

1.3 Colecciones

Python tiene cuatro colecciones integradas en el lenguaje, con sintaxis propia. Son las estructuras de datos que aparecen constantemente en Django: las rutas son una **lista**, el contexto de una plantilla es un **diccionario**, las opciones de un campo son **tuplas**.

Colección	Sintaxis	Ordenada	Modificable	Duplicados	Equivalente aproximado
Lista (`list`)	`[1, 2, 3]`	Sí	Sí	Sí	`ArrayList` en Java; array indexado en PHP
Tupla (`tuple`)	`(1, 2, 3)`	Sí	No	Sí	Registro inmutable
Diccionario (`dict`)	`{"id": 1, "titulo": "..."}`	Sí (orden de inserción)	Sí	Claves únicas	`HashMap` en Java; array asociativo en PHP
Conjunto (`set`)	`{1, 2, 3}`	No	Sí	No	`HashSet` en Java

Operaciones habituales:

Colección	Operaciones
Lista	`lista.append(x)` añade al final; `lista.insert(0, x)` inserta en una posición; `lista.remove(x)` quita la primera aparición; `lista.pop()` quita y devuelve el último; `lista.sort()` ordena; `sorted(lista)` devuelve una copia ordenada; `len(lista)`
Tupla	Índices y porciones como en las listas; **desempaquetado**: `a, b = (1, 2)`
Diccionario	`d["clave"]` lee (error `KeyError` si no existe); `d.get("clave", defecto)` lee sin error; `d["clave"] = valor` crea o modifica; `del d["clave"]`; `d.keys()`, `d.values()`, `d.items()`
Conjunto	`c.add(x)`; `x in c`; unión `a | b`; intersección `a & b`; diferencia `a - b`

Un diccionario vacío se escribe `{}`; un conjunto vacío, `set()`. Una tupla de un solo elemento necesita una coma: `("baja",)`.

```
# python_basico/colecciones.py
estados = ["abierta", "en_curso", "resuelta"]
estados.append("cerrada")
print(estados)
print(estados[0], estados[-1], len(estados))
estados.remove("cerrada")
print(estados[1:])

punto = (40.0, -3.0)
latitud, longitud = punto
print(latitud)
```

```
incidencia = {"id": 1, "titulo": "Ratón roto", "prioridad": "baja"}
incidencia["estado"] = "abierta"
print(incidencia["titulo"])
print(incidencia.get("autor", "sin autor"))
print(list(incidencia.keys()))
for clave, valor in incidencia.items():
    print(clave, "=", valor)

etiquetas = {"red", "wifi", "red", "biblioteca"}
print(len(etiquetas))
print("wifi" in etiquetas)
print(sorted(etiquetas & {"wifi", "proyector"}))
```

Hay que distinguir entre **modificar** una colección y **reasignar** una variable. Las listas, diccionarios y conjuntos son *mutables*: si dos variables apuntan a la misma lista, un `append()` hecho a través de una se ve desde la otra. Igual que en Java con los objetos, la asignación `b = a` no copia la lista; para copiarla se usa `a.copy()` o `list(a)`.

Ejercicio 1.3. Colecciones

Objetivo: usar listas, tuplas, diccionarios y conjuntos, y distinguir sus comportamientos.

1. Guardad el código anterior en `colecciones.py` y ejecutadlo.
2. Al final del fichero, cread una lista `copia = estados` y otra `independiente = estados.copy()`. Añadid `"cerrada"` a `copia` e imprimid `len(estados)` y `len(independiente)`.

Verificación. La salida del código original es:

```
['abierta', 'en_curso', 'resuelta', 'cerrada']
abierta cerrada 4
['en_curso', 'resuelta']
40.0
Ratón roto
sin autor
['id', 'titulo', 'prioridad', 'estado']
id = 1
titulo = Ratón roto
prioridad = baja
estado = abierta
3
True
['wifi']
```

El conjunto tiene 3 elementos porque `"red"` aparece dos veces. Las dos líneas añadidas en el paso 2 imprimen `4` y `3`: `copia` y `estados` son la misma lista, mientras que `independiente` es otra.

1.4 Control de flujo

1.4.1 Condicionales

```
if prioridad == "alta":
    print("Atender hoy")
elif prioridad == "media":
    print("Atender esta semana")
else:
    print("Sin prisa")
```

No hay paréntesis obligatorios en la condición, se escribe `elif` (no `else if`) y no existe `switch` clásico (desde Python 3.10 existe `match`, que no se usará en este módulo). La expresión condicional de una línea se escribe `valor_si_cierto if condicion else valor_si_falso`, equivalente al operador `? :` de PHP y Java:

```
texto = "urgente" if prioridad == "alta" else "normal"
```

Valores verdaderos y falsos. En una condición, se consideran falsos `False`, `None`, el número `0`, la cadena vacía `""` y las colecciones vacías (`[]`, `{}`, `()`, `set()`). Todo lo demás es verdadero. Por eso es habitual escribir `if incidencias:` para preguntar si una lista tiene elementos.

1.4.2 Bucles

El bucle `for` de Python recorre directamente los elementos de una colección, como el `foreach` de PHP o el `for (String s : lista)` de Java. Para recorrer números se usa `range()`:

Expresión	Valores
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(10, 0, -3)`	10, 7, 4, 1

Dos funciones ayudan a recorrer colecciones:

- `enumerate(coleccion, start=1)` devuelve parejas (posición, elemento).
- `zip(a, b)` recorre dos colecciones en paralelo.

El bucle `while` funciona como en Java y PHP. `break` sale del bucle y `continue` pasa a la siguiente vuelta. No existen los operadores `++` ni `--`: se escribe `contador += 1`.

```
# python_basico/control.py
incidencias = [
    ("Proyector del aula 12 no enciende", "alta"),
    ("Ratón roto en el puesto 7", "baja"),
    ("No funciona la wifi en la biblioteca", "alta"),
```

```
    ("Instalar Python 3.13 en el aula 15", "media"),
]

urgentes = 0
for numero, (titulo, prioridad) in enumerate(incidencias, start=1):
    if prioridad == "alta":
        marca = "!!"
        urgentes += 1
    elif prioridad == "media":
        marca = "! "
    else:
        marca = "  "
    print(f"{numero}. {marca} {titulo}")

print(f"Urgentes: {urgentes} de {len(incidencias)}")

intentos = 0
while intentos < 3:
    intentos += 1
print("Intentos:", intentos)

for n in range(10, 0, -3):
    print(n, end=" ")
print()
```

En `for numero, (titulo, prioridad) in enumerate(...)` se combinan dos desempaquetados: `enumerate` produce la pareja (posición, tupla) y la tupla se separa a su vez en `titulo` y `prioridad`. El parámetro `end=" "` de `print()` sustituye el salto de línea final por un espacio.

Ejercicio 1.4. Control de flujo

Objetivo: escribir condicionales y bucles con la sintaxis de Python.

1. Guardad el código anterior en `control.py` y ejecutadlo.
2. Añadid al final un bucle que recorra la lista `incidencias` y **se detenga** (`break`) en la primera incidencia cuyo título contenga la palabra `"wifi"`, imprimiendo `Primera de red:` seguido de su título.
3. Añadid una línea que imprima `Hay incidencias` o `No hay incidencias` con una expresión condicional de una línea, según la lista tenga elementos o no.

Verificación. La salida del código original es:

```
1. !! Proyector del aula 12 no enciende
2.    Ratón roto en el puesto 7
3. !! No funciona la wifi en la biblioteca
4. !  Instalar Python 3.13 en el aula 15
Urgentes: 2 de 4
Intentos: 3
10 7 4 1
```

Las líneas añadidas imprimen `Primera de red: No funciona la wifi en la biblioteca` y `Hay incidencias`. Si se vacía la lista (`incidencias = []`) antes de la línea del paso 3, imprime `No hay incidencias`.

1.5 Funciones

Una función se define con `def`, sus parámetros no llevan tipo (aunque se puede indicar, como se verá en el apartado 1.10) y devuelve un valor con `return`. Si no hay `return`, devuelve `None`.

Python permite dos formas de pasar argumentos, que se pueden combinar:

- **Por posición**, como en Java: `resumen("Ratón roto", "baja")`.
- **Por nombre** (*keyword arguments*): `resumen(prioridad="baja", titulo="Ratón roto")`. El orden da igual y la llamada se lee mejor. Django los usa constantemente: `models.CharField(max_length=120)`, `path("", vista, name="lista")`.

Un parámetro puede tener un **valor por defecto** (`prioridad="media"`); si se omite en la llamada, toma ese valor. Los parámetros escritos después de un asterisco solitario (`*`) solo se pueden pasar por nombre.

```python
# python_basico/funciones.py
def resumen(titulo, prioridad="media", *, max_longitud=20):
    """Devuelve una línea corta que resume una incidencia."""
    if len(titulo) > max_longitud:
        titulo = titulo[: max_longitud - 3] + "..."
    return f"[{prioridad.upper()}] {titulo}"


print(resumen("Ratón roto"))
print(resumen("Ratón roto", "baja"))
print(resumen(prioridad="alta", titulo="Proyector del aula 12 no enciende"))
print(resumen("Proyector del aula 12 no enciende", max_longitud=12))


def contar(*valores, **opciones):
    print(len(valores), opciones)


contar(1, 2, 3, orden="desc")
```

Elemento	Significado
`"""..."""` tras `def`	Cadena de documentación (*docstring*): describe la función; la muestran los editores y `help(resumen)`
`*valores`	Recoge en una **tupla** todos los argumentos posicionales sobrantes

Elemento	Significado
`**opciones`	Recoge en un **diccionario** todos los argumentos por nombre sobrantes

La notación `*args, **kwargs` aparece a menudo en el código de Django, por ejemplo al redefinir un método de una clase del framework y pasarle los mismos argumentos que se reciben: `super().save(*args, **kwargs)`.

Las funciones son objetos: se pueden guardar en variables y pasar como argumento a otras funciones, sin paréntesis. Es lo que se hace en `path("hola/", views.hola)`: se pasa la función, no el resultado de llamarla. Para funciones muy cortas que solo se usan como argumento existe la expresión `lambda`: `lambda i: i["horas"]` es una función anónima que recibe `i` y devuelve `i["horas"]`.

> ***Cuidado con los valores por defecto mutables.*** *El valor por defecto se crea* ***una sola vez****, al definir la función. Si se escribe* `def agregar(x, lista=[])`*, todas las llamadas comparten la misma lista. Lo correcto es* `def agregar(x, lista=None)` *y, dentro,* `if lista is None: lista = []`*.*

Ejercicio 1.5. Funciones y argumentos

Objetivo: definir funciones con valores por defecto y llamarlas por posición y por nombre.

1. Guardad el código anterior en `funciones.py` y ejecutadlo.
2. Añadid una función `estimar_horas(prioridad, tecnicos=1)` que devuelva `8` horas para `"alta"`, `16` para `"media"` y `40` para cualquier otro valor, divididas entre el número de técnicos y redondeadas a un decimal con `round(valor, 1)`.
3. Imprimid `estimar_horas("alta")`, `estimar_horas("media", 3)` y `estimar_horas(tecnicos=4, prioridad="baja")`.

Verificación. La salida del código original es:

```
[MEDIA] Ratón roto
[BAJA] Ratón roto
[ALTA] Proyector del aul...
[MEDIA] Proyector...
3 {'orden': 'desc'}
```

Las tres líneas añadidas imprimen `8.0`, `5.3` y `10.0`. Si se llama `estimar_horas()` sin argumentos, Python muestra `TypeError: estimar_horas() missing 1 required positional argument: 'prioridad'`.

1.6 Comprensiones

Una **comprensión** construye una colección nueva a partir de otra en una sola expresión, combinando un recorrido y, opcionalmente, un filtro. Sustituye a los bucles que solo sirven para rellenar una lista, y es una de las construcciones más características de Python.

Tipo	Sintaxis	Resultado
De lista	`[expresion for x in coleccion if condicion]`	`list`
De diccionario	`{clave: valor for x in coleccion if condicion}`	`dict`
De conjunto	`{expresion for x in coleccion}`	`set`
Expresión generadora	`(expresion for x in coleccion)`	Produce los valores de uno en uno, sin crear la lista

Una expresión generadora se suele pasar directamente a funciones que recorren valores: `sum()`, `min()`, `max()`, `any()` (¿alguno es verdadero?) y `all()` (¿todos lo son?). La función `sorted()` acepta un parámetro `key` con la función que da el criterio de orden, y `reverse=True` para invertirlo.

```
# python_basico/comprensiones.py
incidencias = [
    {"titulo": "Proyector del aula 12 no enciende", "prioridad": "alta", "horas":
 ↪  2},
    {"titulo": "Ratón roto en el puesto 7", "prioridad": "baja", "horas": 1},
    {"titulo": "No funciona la wifi en la biblioteca", "prioridad": "alta", "horas":
 ↪  5},
    {"titulo": "Instalar Python 3.13 en el aula 15", "prioridad": "media", "horas":
 ↪  3},
]

urgentes = [i["titulo"] for i in incidencias if i["prioridad"] == "alta"]
print(urgentes)

largas = {n: i["titulo"] for n, i in enumerate(incidencias, start=1) if i["horas"] >
 ↪  2}
print(largas)

prioridades = {i["prioridad"] for i in incidencias}
print(sorted(prioridades))

print(sum(i["horas"] for i in incidencias))
print(any(i["prioridad"] == "baja" for i in incidencias))
print(sorted(incidencias, key=lambda i: i["horas"], reverse=True)[0]["titulo"])
```

El equivalente con un bucle de la primera comprensión ocupa cuatro líneas:

```
urgentes = []
for i in incidencias:
    if i["prioridad"] == "alta":
        urgentes.append(i["titulo"])
```

Ejercicio 1.6. Comprensiones

Objetivo: sustituir bucles por comprensiones y expresiones generadoras.

1. Guardad el código anterior en `comprensiones.py` y ejecutadlo.
2. Añadid, usando una sola expresión en cada caso:
    1. Una lista con los títulos en mayúsculas de las incidencias que **no** son de prioridad alta.
    2. Un diccionario que asocie cada título con sus horas multiplicadas por 60 (minutos), solo para las incidencias de 3 horas o más.
    3. La media de horas, calculada con `sum()` y `len()`.

Verificación. La salida del código original es:

```
['Proyector del aula 12 no enciende', 'No funciona la wifi en la biblioteca']
{3: 'No funciona la wifi en la biblioteca', 4: 'Instalar Python 3.13 en el aula 15'}
['alta', 'baja', 'media']
11
True
No funciona la wifi en la biblioteca
```

Las líneas añadidas muestran `['RATÓN ROTO EN EL PUESTO 7', 'INSTALAR PYTHON 3.13 EN EL AULA 15']`, `{'No funciona la wifi en la biblioteca': 300, 'Instalar Python 3.13 en el aula 15': 180}` y `2.75`.

1.7 Clases, objetos y herencia

Las clases de Python tienen los mismos conceptos que las de Java (atributos, métodos, herencia), con una sintaxis más breve y algunas diferencias importantes:

Concepto	Java	Python
Constructor	Método con el nombre de la clase	Método `__init__(self, ...)`
Referencia al objeto	`this` (implícito)	`self`, **primer parámetro explícito** de cada método
Declaración de atributos	En el cuerpo de la clase, con tipo	Se crean al asignarlos: `self.titulo = titulo`

Concepto	Java	Python
Crear un objeto	`new Incidencia("...")`	`Incidencia("...")` (sin `new`)
Herencia	`class B extends A`	`class B(A):`
Llamar al padre	`super(...)`, `super.metodo()`	`super().__init__(...)`, `super().metodo()`
Visibilidad	`private`, `protected`, `public`	Todo es público; por convenio, un nombre que empieza por `_` es interno
Representación como texto	`toString()`	`__str__(self)`
Atributo de clase	`static`	Variable definida en el cuerpo de la clase

Los métodos con dos guiones bajos a cada lado (`__init__`, `__str__`) se llaman **métodos especiales**: Python los invoca en situaciones concretas. `__str__` se usa al hacer `print(objeto)` o `str(objeto)`; Django lo usa para mostrar un objeto en el panel de administración y en los formularios.

```
# python_basico/clases.py
class Incidencia:
    total = 0

    def __init__(self, titulo, prioridad="media"):
        self.titulo = titulo
        self.prioridad = prioridad
        self.estado = "abierta"
        Incidencia.total += 1

    def resolver(self):
        self.estado = "resuelta"

    def __str__(self):
        return f"{self.titulo} ({self.estado})"


class IncidenciaHardware(Incidencia):
    def __init__(self, titulo, equipo, prioridad="media"):
        super().__init__(titulo, prioridad)
        self.equipo = equipo

    def __str__(self):
        return f"{super().__str__()} [equipo {self.equipo}]"


a = Incidencia("No funciona la wifi", "alta")
b = IncidenciaHardware("Pantalla rota", "PC-07")
b.resolver()
print(a)
print(b)
print(isinstance(b, Incidencia), isinstance(a, IncidenciaHardware))
```

```
print(Incidencia.total)
```

1.7.1 Las clases en Django

Django se programa casi siempre **heredando de clases del framework** y declarando atributos de clase. Este fragmento, que se escribirá en el apartado 5, ya se puede leer con lo visto hasta ahora:

```
class Categoria(models.Model):
    nombre = models.CharField(max_length=50, unique=True)

    class Meta:
        ordering = ["nombre"]

    def __str__(self):
        return self.nombre
```

- `Categoria` hereda de `models.Model`, la clase de Django que sabe guardar objetos en la base de datos.
- `nombre` es un **atributo de clase** que contiene un objeto `CharField` creado con argumentos por nombre. Django lee esos atributos para saber qué columnas tiene la tabla. Este estilo, en el que se **declara** qué se quiere en lugar de programar cómo se hace, se llama **declarativo**.
- `Meta` es una **clase anidada** que agrupa opciones del modelo.
- `__str__` indica cómo se muestra cada categoría como texto.

Ejercicio 1.7. Clases y herencia

Objetivo: definir clases con constructor, métodos y herencia.

1. Guardad el código anterior en `clases.py` y ejecutadlo.
2. Añadid a `Incidencia` un método `escalar(self)` que suba la prioridad un nivel (`baja` pasa a `media` y `media` a `alta`; `alta` se queda igual). Usad la tupla `NIVELES = ("baja", "media", "alta")` como atributo de clase y el método `index()` de las tuplas.
3. Cread una clase `IncidenciaRed(Incidencia)` con un atributo adicional `ubicacion` y un `__str__` que añada `@ ubicacion` al texto de la clase padre.
4. Probad: cread `r = IncidenciaRed("Sin wifi", "biblioteca", "baja")`, llamad dos veces a `r.escalar()` e imprimid `r` y `r.prioridad`.

Verificación. La salida del código original es:

```
No funciona la wifi (abierta)
Pantalla rota (resuelta) [equipo PC-07]
True False
2
```

Las líneas del paso 4 imprimen `Sin wifi (abierta) @ biblioteca` y `alta`. Si se llama una tercera vez a `escalar()`, la prioridad sigue siendo `alta` y no se produce ningún error.

1.8 Módulos y paquetes

Un **módulo** es un fichero `.py`; su nombre es el del fichero sin la extensión. Un **paquete** es una carpeta que agrupa módulos y contiene un fichero `__init__.py` (que puede estar vacío). Un proyecto Django es exactamente eso: `config` e `incidencias` son paquetes, y `models.py`, `views.py` y `urls.py` son módulos.

Forma de importar	Uso posterior
`import json`	`json.dumps(...)`
`from pathlib import Path`	`Path(...)`
`from django.db import models`	`models.Model`
`from . import views`	`views.hola` (el punto significa «del mismo paquete»)
`from .models import Categoria, Incidencia`	`Categoria`, `Incidencia`
`import datetime as dt`	`dt.date.today()` (alias)

Las importaciones que empiezan por punto son **relativas**: buscan el módulo dentro del paquete actual. Son las que se usan dentro de una aplicación Django (`from .models import Incidencia`). Por convenio, las importaciones van al principio del fichero, en tres grupos separados por una línea en blanco: biblioteca estándar, paquetes de terceros (como Django) y módulos del propio proyecto.

1.8.1 Ejecución como programa principal

Al importar un módulo, Python **ejecuta todo su código**. Para que un fichero pueda usarse a la vez como programa y como módulo importable, el código del programa principal se escribe dentro de esta condición, que solo es cierta cuando el fichero se ejecuta directamente con `python fichero.py`:

```
def main():
    ...


if __name__ == "__main__":
    main()
```

1.8.2 La biblioteca estándar

Python incluye una biblioteca estándar muy amplia. Algunos módulos útiles en una aplicación web:

Módulo	Para qué sirve
`datetime`	Fechas y horas: `date(2026, 10, 21)`, `datetime.now()`, `timedelta(days=7)`

Módulo	Para qué sirve
`pathlib`	Rutas de ficheros independientes del sistema operativo: `Path("datos") / "fichero.json"`
`json`	Convertir entre texto JSON y diccionarios o listas: `json.dumps()`, `json.loads()`
`logging`	Registro de mensajes (se usará en el apartado 4)
`time`	Medición de tiempos: `time.perf_counter()`

Los paquetes que no están en la biblioteca estándar, como Django, se instalan con pip, preferiblemente dentro de un entorno virtual.

1.9 Excepciones

Los errores se gestionan con excepciones, como en Java, con otras palabras clave:

Java	Python
`try { ... }`	`try:`
`catch (IOException e) { ... }`	`except OSError as e:`
`finally { ... }`	`finally:`
`throw new MiError("...")`	`raise MiError("...")`
`class MiError extends Exception`	`class MiError(Exception):`
No existe	`else:` (se ejecuta solo si el bloque `try` no ha lanzado ninguna excepción)

Python no tiene excepciones comprobadas (*checked exceptions*): ninguna función declara qué excepciones lanza ni obliga a capturarlas. Algunas excepciones frecuentes son `ValueError` (valor incorrecto, como `int("dos")`), `TypeError` (tipo incorrecto), `KeyError` (clave inexistente en un diccionario), `IndexError` (posición inexistente en una lista) y `AttributeError` (atributo inexistente). Django define las suyas: por ejemplo, `Incidencia.DoesNotExist` cuando se busca un objeto que no está en la base de datos.

```python
# python_basico/excepciones.py
class PrioridadNoValida(Exception):
    pass


PRIORIDADES = ("baja", "media", "alta")


def comprobar_prioridad(valor):
    if valor not in PRIORIDADES:
        raise PrioridadNoValida(f"'{valor}' no es una prioridad válida")
    return valor
```

```
for texto in ["alta", "urgente"]:
    try:
        print("Correcta:", comprobar_prioridad(texto))
    except PrioridadNoValida as error:
        print("Error:", error)

try:
    horas = int("dos")
except ValueError as error:
    print("No es un número:", error)
else:
    print("Horas:", horas)
finally:
    print("Fin de la comprobación")
```

La palabra clave `pass` no hace nada: se usa cuando la sintaxis exige un bloque pero no hay nada que escribir, como en la clase `PrioridadNoValida`, que hereda todo de `Exception`.

Ejercicio 1.8. Excepciones

Objetivo: lanzar y capturar excepciones propias y del lenguaje.

1. Guardad el código anterior en `excepciones.py` y ejecutadlo.
2. Cambiad `"dos"` por `"2"` y volved a ejecutarlo.
3. Escribid una función `leer_horas(texto)` que devuelva el número entero contenido en `texto` y, si no es un número o es negativo, lance `ValueError` con el mensaje `Horas no válidas:` seguido del texto recibido. Probadla con `"3"`, `"-1"` y `"tres"` dentro de un `try`.

Verificación. La salida del código original es:

```
Correcta: alta
Error: 'urgente' no es una prioridad válida
No es un número: invalid literal for int() with base 10: 'dos'
Fin de la comprobación
```

Tras el paso 2, las dos últimas líneas pasan a ser `Horas: 2` y `Fin de la comprobación`: el bloque `else` se ejecuta y `finally` se ejecuta siempre. En el paso 3, `"3"` devuelve `3`, y `"-1"` y `"tres"` producen los mensajes `Horas no válidas: -1` y `Horas no válidas: tres`.

1.10 Tipado opcional

Desde Python 3.5 se pueden **anotar** los tipos de los parámetros, del valor devuelto y de las variables. Las anotaciones se llaman *type hints* (indicaciones de tipo):

```python
# python_basico/tipos.py
def media_horas(horas: list[int]) -> float | None:
    if not horas:
        return None
    return sum(horas) / len(horas)


print(media_horas([2, 1, 5, 3]))
print(media_horas([]))
print(media_horas.__annotations__)
```

Anotación	Significado
`x: int`	Un entero
`-> str`	La función devuelve una cadena
`list[str]`	Lista de cadenas
`dict[str, int]`	Diccionario con claves de texto y valores enteros
`str | None`	Una cadena o `None` (desde Python 3.10)
`-> None`	La función no devuelve nada

A diferencia de Java, **Python no comprueba las anotaciones al ejecutar**: `media_horas("hola")` no da error por el tipo del argumento, sino al intentar sumar las letras. Las anotaciones sirven como documentación y para que el editor (la extensión Python de Visual Studio Code) o herramientas como *mypy* avisen de errores antes de ejecutar. Django no obliga a usarlas; en este módulo se usarán cuando aclaren el código.

Ejercicio 1.9. Tipado opcional

Objetivo: anotar funciones y comprobar que las anotaciones no se verifican al ejecutar.

1. Guardad el código anterior en `tipos.py` y ejecutadlo.
2. Añadid la línea `print(media_horas((4, 6)))` (una tupla en lugar de una lista) y ejecutad de nuevo.

Verificación. La salida del código original es:

```
2.75
None
{'horas': list[int], 'return': float | None}
```

La línea añadida imprime `5.0` sin ningún error: la anotación dice `list[int]`, pero Python no la comprueba. En Visual Studio Code, con la comprobación de tipos activada (*Python, Analysis: Type Checking Mode* en `basic`), el editor subraya el argumento.

1.11 Ejercicio integrador

Este apartado reúne todo lo anterior en un programa pequeño organizado en un paquete. Es, a escala, lo que hará Django: clases que representan los datos, un módulo que los almacena y los consulta, excepciones para los casos de error y conversión a JSON.

La estructura es la siguiente:

```
python_basico/
    principal.py
    gestor/
        __init__.py      vacío: convierte la carpeta en un paquete
        modelos.py       clase Incidencia y excepción propia
        almacen.py       clase Almacen: crear, obtener, filtrar y guardar
```

```python
# python_basico/gestor/modelos.py
from datetime import date

PRIORIDADES = ("baja", "media", "alta")


class IncidenciaNoEncontrada(Exception):
    pass


class Incidencia:
    def __init__(self, id: int, titulo: str, prioridad: str = "media") -> None:
        if prioridad not in PRIORIDADES:
            raise ValueError(f"Prioridad no válida: {prioridad}")
        self.id = id
        self.titulo = titulo
        self.prioridad = prioridad
        self.estado = "abierta"
        self.creada = date(2026, 10, 21)

    def a_diccionario(self) -> dict:
        return {
            "id": self.id,
            "titulo": self.titulo,
            "prioridad": self.prioridad,
            "estado": self.estado,
            "creada": self.creada.isoformat(),
        }

    def __str__(self) -> str:
        return f"#{self.id} {self.titulo} [{self.prioridad}, {self.estado}]"
```

```python
# python_basico/gestor/almacen.py
import json
from pathlib import Path

from .modelos import Incidencia, IncidenciaNoEncontrada
```

```
class Almacen:
    def __init__(self) -> None:
        self._datos: dict[int, Incidencia] = {}
        self._siguiente = 1

    def crear(self, titulo: str, prioridad: str = "media") -> Incidencia:
        incidencia = Incidencia(self._siguiente, titulo, prioridad)
        self._datos[incidencia.id] = incidencia
        self._siguiente += 1
        return incidencia

    def obtener(self, id: int) -> Incidencia:
        try:
            return self._datos[id]
        except KeyError:
            raise IncidenciaNoEncontrada(f"No existe la incidencia {id}") from None

    def filtrar(self, estado: str | None = None) -> list[Incidencia]:
        return [i for i in self._datos.values() if estado is None or i.estado ==
        ↪  estado]

    def guardar(self, ruta: Path) -> None:
        datos = [i.a_diccionario() for i in self._datos.values()]
        ruta.write_text(json.dumps(datos, ensure_ascii=False, indent=2),
↪   encoding="utf-8")
```

```
# python_basico/principal.py
from pathlib import Path

from gestor.almacen import Almacen
from gestor.modelos import IncidenciaNoEncontrada


def main() -> None:
    almacen = Almacen()
    almacen.crear("Proyector del aula 12 no enciende", "alta")
    almacen.crear("Ratón roto en el puesto 7", "baja")
    almacen.crear("No funciona la wifi en la biblioteca", "alta")
    almacen.obtener(2).estado = "resuelta"

    for incidencia in almacen.filtrar("abierta"):
        print(incidencia)

    try:
        almacen.obtener(7)
    except IncidenciaNoEncontrada as error:
        print("Error:", error)

    try:
        almacen.crear("Teclado sin teclas", "urgentisima")
    except ValueError as error:
        print("Error:", error)
```

```
    almacen.guardar(Path("incidencias.json"))
    print("Guardadas:", len(almacen.filtrar()))


if __name__ == "__main__":
    main()
```

Algunos detalles nuevos:

- `raise ... from None` lanza la excepción propia ocultando la original (`KeyError`), para que el mensaje de error sea más claro.
- `self._datos` empieza por guion bajo: por convenio, indica que es un atributo interno que no debe usarse desde fuera de la clase.
- `json.dumps(..., ensure_ascii=False, indent=2)` genera el JSON con las tildes tal cual y con sangría de dos espacios.

Ejercicio 1.10. Gestor de incidencias en memoria

Objetivo: organizar un programa en un paquete con clases, excepciones y anotaciones de tipo.

1. Cread la carpeta `gestor` con los tres ficheros y `principal.py` fuera de ella, y ejecutad `python principal.py`.
2. Abrid el fichero `incidencias.json` generado.
3. Añadid a `Almacen` un método `contar_por_prioridad(self) -> dict[str, int]` que devuelva cuántas incidencias hay de cada prioridad, usando una comprensión de diccionario sobre `PRIORIDADES` (importadla de `.modelos`). Imprimid su resultado al final de `main()`.
4. Añadid a `Almacen` un método `eliminar(self, id: int) -> None` que borre una incidencia y lance `IncidenciaNoEncontrada` si no existe (reutilizad `obtener`).

Verificación. La salida del programa original es:

```
#1 Proyector del aula 12 no enciende [alta, abierta]
#3 No funciona la wifi en la biblioteca [alta, abierta]
Error: No existe la incidencia 7
Error: Prioridad no válida: urgentisima
Guardadas: 3
```

`incidencias.json` contiene una lista con tres objetos; el primero tiene `"id": 1`, `"estado": "abierta"` y `"creada": "2026-10-21"`, y el segundo, `"estado": "resuelta"`. El método del paso 3 devuelve `{'baja': 1, 'media': 0, 'alta': 2}`. Tras `almacen.eliminar(1)`, `almacen.obtener(1)` lanza `IncidenciaNoEncontrada` con el mensaje `No existe la incidencia 1`.

1.12 Resumen comparativo

Necesidad	PHP	Java	Python
Mostrar por pantalla	`echo "Hola $n";`	`System.out.println("Hola " + n);`	`print(f"Hola {n}")`
Lista	`$l = [1, 2];`	`List<Integer> l = new ArrayList<>();`	`l = [1, 2]`
Diccionario	`$d = ["a" => 1];`	`Map<String, Integer> d = new HashMap<>();`	`d = {"a": 1}`
Recorrer	`foreach ($l as $x)`	`for (int x : l)`	`for x in l:`
Longitud	`count($l)`, `strlen($s)`	`l.size()`, `s.length()`	`len(l)`, `len(s)`
Función	`function f($a, $b = 2)`	`int f(int a, int b)`	`def f(a, b=2):`
Clase	`class A extends B`	`class A extends B`	`class A(B):`
Constructor	`__construct()`	`A()`	`__init__(self)`
Objeto actual	`$this`	`this`	`self`
Excepción	`throw new E();` / `catch (E $e)`	`throw new E();` / `catch (E e)`	`raise E()` / `except E as e:`
Importar	`require`, `use`	`import`	`import`, `from ... import`

Para reflexionar. *Python no comprueba los tipos al ejecutar y obliga a sangrar el código. ¿Qué ventajas e inconvenientes tiene cada una de esas decisiones para un equipo que mantiene una aplicación grande durante años?*

Capítulo 2

Estructura de un proyecto Django

Para empezar

1. En la Unidad 1, ¿qué ficheros generó Django con `startproject` y cuáles tuvisteis que escribir vosotros?
2. Si una aplicación web tiene una parte de usuarios, otra de incidencias y otra de informes, ¿cómo repartiríais el código en carpetas?
3. ¿Qué datos de configuración de una aplicación web (contraseñas, rutas, idioma…) no deberían estar escritos dentro del código?

2.1 Proyecto y aplicaciones

En la Unidad 1 se distinguieron dos niveles en Django, que conviene recordar porque organizan todo lo que viene después:

- El **proyecto** es la configuración global del sitio. Lo crea `django-admin startproject` y contiene el paquete de configuración (en este módulo se llama siempre `config`) y la utilidad `manage.py`.
- Una **aplicación** (*app*) es un paquete de Python que agrupa una funcionalidad: sus modelos, vistas, rutas, plantillas, formularios y pruebas. La crea `python manage.py startapp`. Un proyecto tiene una o varias aplicaciones, propias o de terceros: el panel de administración (`django.contrib.admin`) o Django REST Framework (`rest_framework`) son aplicaciones que se instalan en el proyecto.

Una aplicación debe tener una responsabilidad clara. En un proyecto grande habría aplicaciones como `usuarios`, `incidencias` o `inventario`; en este módulo bastará con una, `incidencias`.

2.2 Creación del proyecto «Incidencias»

El proyecto de ejemplo de toda la unidad se crea con los mismos pasos de la Unidad 1. Se resumen aquí para tenerlos a mano.

Ejercicio 2.1. Creación del proyecto y de la aplicación

Objetivo: crear el proyecto `incidencias` con su entorno virtual, su paquete de configuración `config` y su aplicación `incidencias`.

1. Cread la carpeta y el entorno virtual, e instalad Django:

```
> cd $HOME\proyectos\pauf
> mkdir incidencias
> cd incidencias
> python -m venv .venv
> .\.venv\Scripts\Activate.ps1
> python -m pip install --upgrade pip
> python -m pip install "django~=5.2.0"
```

2. Cread el proyecto (con el punto final) y la aplicación:

```
> django-admin startproject config .
> python manage.py startapp incidencias
```

3. Guardad las dependencias y cread las tablas iniciales:

```
> pip freeze > requirements.txt
> python manage.py migrate
```

4. Abrid la carpeta en Visual Studio Code con `code .`. Si el editor pregunta qué intérprete de Python usar, elegid el de `.venv` (también se puede elegir con `Ctrl + Mayús + P`, *Python: Select Interpreter*).

Verificación. `python -m django --version` muestra `5.2.x`. En la carpeta `incidencias` hay exactamente estos elementos: `.venv`, `config`, `incidencias`, `db.sqlite3`, `manage.py` y `requirements.txt`. El nombre `incidencias` aparece dos veces en la ruta (`incidencias\incidencias\`): la carpeta exterior es la del proyecto y la interior es el paquete de la aplicación.

2.3 La estructura generada

Tras el ejercicio anterior, el proyecto tiene esta estructura:

```
incidencias/                carpeta del proyecto
    .venv/                  entorno virtual (no se entrega ni se sube a Git)
    config/                 paquete de configuración del proyecto
        __init__.py
        settings.py         configuración
        urls.py             tabla de rutas principal
        asgi.py             punto de entrada para servidores ASGI
        wsgi.py             punto de entrada para servidores WSGI
    incidencias/            paquete de la aplicación
        __init__.py
```

```
    admin.py            registro de modelos en el panel de administración
    apps.py             configuración de la aplicación
    migrations/         migraciones de la base de datos
        __init__.py
    models.py           modelos
    tests.py            pruebas automatizadas
    views.py            vistas
db.sqlite3              base de datos SQLite de desarrollo
manage.py               utilidad de administración
requirements.txt        dependencias
```

A lo largo de la unidad, la aplicación irá recibiendo ficheros nuevos que no crea `startapp`, pero que siguen convenios de nombre que Django y los desarrolladores reconocen:

Fichero o carpeta	Contenido	Apartado
`incidencias/urls.py`	Rutas de la aplicación	3
`incidencias/templates/incidencias/`	Plantillas HTML	3
`incidencias/middleware.py`	*Middleware* propio	4
`incidencias/forms.py`	Formularios	6
`incidencias/management/commands/`	Órdenes propias de `manage.py`	6
`incidencias/serializers.py`	Serializadores de la API	7
`incidencias/api.py`	Vistas de la API	7
`incidencias/permissions.py`	Permisos de la API	8

Los nombres de `models.py`, `admin.py` y `apps.py` son obligatorios, porque Django los busca por su nombre. Los demás son convenios: se podrían llamar de otra forma, pero cualquier desarrollador de Django espera encontrarlos así.

2.4 El fichero de configuración `settings.py`

`config/settings.py` es un módulo de Python normal: cada ajuste es una variable en mayúsculas. Django lo lee al arrancar. Los ajustes más importantes son estos:

Ajuste	Valor generado	Función
`BASE_DIR`	`Path(__file__).resolve().parent.parent`	Carpeta del proyecto, calculada con `pathlib`. Se usa para construir otras rutas: `BASE_DIR / "db.sqlite3"`

Ajuste	Valor generado	Función
`SECRET_KEY`	Cadena aleatoria que empieza por `django-insecure-`	Clave para firmar cookies, sesiones y tokens CSRF. En producción debe ser secreta y no estar en el código (Unidad 4)
`DEBUG`	`True`	Modo de depuración: páginas de error detalladas y recarga automática. Nunca en producción
`ALLOWED_HOSTS`	`[]`	Nombres de dominio a los que responde el sitio. Con `DEBUG = True` y la lista vacía, se admiten `localhost` y `127.0.0.1`
`INSTALLED_APPS`	Seis aplicaciones de `django.contrib`	Aplicaciones activas del proyecto
`MIDDLEWARE`	Siete componentes	Capas que procesan cada petición y cada respuesta (apartado 4)
`ROOT_URLCONF`	`"config.urls"`	Módulo con la tabla de rutas principal
`TEMPLATES`	Motor de plantillas de Django, `APP_DIRS: True`	Dónde y cómo se buscan las plantillas (apartado 3)
`DATABASES`	SQLite en `BASE_DIR / "db.sqlite3"`	Conexión a la base de datos. En la Unidad 4 será PostgreSQL
`AUTH_PASSWORD_VALIDATORS`	Cuatro validadores	Reglas para las contraseñas de los usuarios (apartado 8)
`LANGUAGE_CODE`	`"en-us"`	Idioma de los textos del framework (mensajes de error, panel de administración)
`TIME_ZONE`	`"UTC"`	Zona horaria con la que se muestran las fechas
`USE_TZ`	`True`	Las fechas se guardan en UTC en la base de datos y se convierten a `TIME_ZONE` al mostrarlas

Ajuste	Valor generado	Función
`STATIC_URL`	`"static/"`	Prefijo de las direcciones de los ficheros estáticos (CSS, JavaScript, imágenes)
`DEFAULT_AUTO_FIELD`	`"django.db.models.BigAutoField"`	Tipo de la clave primaria que Django añade a cada modelo

2.4.1 Registrar la aplicación

Una aplicación creada con `startapp` **no está activa** hasta que se añade a `INSTALLED_APPS`. Mientras no lo esté, Django no busca sus modelos ni sus plantillas ni sus órdenes de `manage.py`. Basta con añadir el nombre del paquete:

```
# config/settings.py (fragmento)
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "incidencias",
]
```

También se puede escribir la ruta completa de la clase de configuración que hay en `incidencias/apps.py` (`"incidencias.apps.IncidenciasConfig"`); el resultado es el mismo. Las aplicaciones de `django.contrib` cubren tareas comunes ya estudiadas en la Unidad 1:

Aplicación	Función
`django.contrib.admin`	Panel de administración
`django.contrib.auth`	Usuarios, grupos, permisos y autenticación
`django.contrib.contenttypes`	Registro de los modelos del proyecto (lo usan `auth` y `admin`)
`django.contrib.sessions`	Sesiones de usuario
`django.contrib.messages`	Mensajes de un solo uso entre peticiones («Incidencia guardada»)
`django.contrib.staticfiles`	Gestión de ficheros estáticos

2.4.2 Idioma y zona horaria

Para que los textos del framework aparezcan en español y las horas en hora peninsular española:

```
# config/settings.py (fragmento)
LANGUAGE_CODE = "es-es"

TIME_ZONE = "Europe/Madrid"
```

El fichero generado por Django usa comillas simples; en este documento se usan comillas dobles, como recomienda la guía de estilo de Django. Las dos formas son equivalentes en Python.

Ejercicio 2.2. Configuración inicial del proyecto

Objetivo: registrar la aplicación y configurar el idioma y la zona horaria.

1. Añadid `"incidencias"` al final de `INSTALLED_APPS`.
2. Cambiad `LANGUAGE_CODE` a `"es-es"` y `TIME_ZONE` a `"Europe/Madrid"`.
3. Ejecutad `python manage.py check`.
4. Arrancad el servidor con `python manage.py runserver` y abrid http://127.0.0.1:8000/admin/.

Verificación. `check` muestra `System check identified no issues (0 silenced).`. El navegador redirige a `/admin/login/?next=/admin/` y muestra el formulario de acceso en español, con el título de pestaña **«Iniciar sesión | Sitio de administración de Django»** y los campos «Nombre de usuario» y «Contraseña». Si `LANGUAGE_CODE` sigue en `"en-us"`, el mismo formulario aparece en inglés («Log in | Django site admin»).

2.5 La utilidad `manage.py`

`manage.py` ejecuta las **órdenes de administración** (*management commands*) del proyecto. Cada aplicación instalada puede añadir las suyas. Las que se usarán en la unidad son estas:

Orden	Función	Apartado
`runserver [puerto]`	Arranca el servidor de desarrollo (por defecto en el puerto 8000)	2
`check`	Comprueba la configuración y el código sin arrancar el servidor	2
`startapp nombre`	Crea una aplicación	2
`makemigrations`	Genera las migraciones a partir de los cambios en los modelos	5
`migrate`	Aplica las migraciones a la base de datos	5

Orden	Función	Apartado
`showmigrations`	Muestra qué migraciones están aplicadas	5
`sqlmigrate app número`	Muestra el SQL de una migración	5
`createsuperuser`	Crea un usuario administrador	6
`shell`	Abre el intérprete de Python con el proyecto cargado	6
`changepassword usuario`	Cambia la contraseña de un usuario	8
`check --deploy`	Comprueba los ajustes de seguridad para producción	8
`help [orden]`	Lista todas las órdenes o explica una	2

2.5.1 El servidor de desarrollo

`runserver` arranca un servidor web pensado **solo para desarrollar**: atiende las peticiones de una en una, sirve los ficheros estáticos y **se reinicia solo** cada vez que se guarda un fichero `.py`. Al arrancar, muestra la versión de Django, el módulo de configuración y la dirección:

```
> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
October 28, 2026 - 09:15:02
Django version 5.2.x, using settings 'config.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Cada petición atendida aparece en la terminal con la fecha, el método, la ruta, el código de estado y el tamaño de la respuesta, por ejemplo `"GET /admin/login/?next=/admin/ HTTP/1.1" 200 4173`. Esa terminal es la primera herramienta de depuración: si una página falla, el error completo aparece allí.

Si un cambio en el código introduce un error de sintaxis, el servidor lo muestra en la terminal y espera a que se corrija. No hace falta pararlo y volver a arrancarlo, salvo al instalar paquetes nuevos o al crear ficheros que Django solo lee al arrancar.

Ejercicio 2.3. Órdenes de administración

Objetivo: usar `manage.py` para obtener información del proyecto.

1. Ejecutad `python manage.py help` y localizad en la lista la sección `[incidencias]`. ¿Aparece?

2. Ejecutad `python manage.py help migrate` y leed qué argumentos admite.
3. Arrancad el servidor en otro puerto con `python manage.py runserver 8080` y abrid http://127.0.0.1:8080/. Detenedlo con `Ctrl + C`.
4. Ejecutad `python manage.py diffsettings` y observad su salida.

Verificación. En el paso 1 la sección `[incidencias]` **no** aparece, porque la aplicación todavía no define órdenes propias (se añadirá una en el apartado 6); sí aparecen `[auth]`, `[contenttypes]`, `[django]`, `[sessions]` y `[staticfiles]`. En el paso 3 se ve la página del cohete «The install worked successfully!» en el puerto 8080. `diffsettings` muestra solo los ajustes que difieren de los valores por defecto de Django; entre ellos, `LANGUAGE_CODE = 'es-es'` y `TIME_ZONE = 'Europe/Madrid'`.

2.6 El recorrido de una petición

Con la estructura del proyecto a la vista, se puede describir qué ocurre desde que el navegador pide una dirección hasta que recibe la respuesta. Cada paso corresponde a uno de los apartados de esta unidad:

1. El **servidor** (en desarrollo, `runserver`; en producción, Gunicorn) recibe la petición HTTP y la entrega a Django a través de `wsgi.py`. Django construye un objeto `HttpRequest`.
2. La petición atraviesa, en orden, las capas de **middleware** de la lista `MIDDLEWARE` (apartado 4).
3. El **sistema de rutas** compara la dirección con los patrones de `config/urls.py` y de los `urls.py` incluidos, y elige una **vista** (apartado 3).
4. La vista consulta o modifica datos mediante los **modelos** y el ORM (apartados 5 y 6).
5. La vista construye la respuesta: una página HTML a partir de una **plantilla** (apartado 3) o un documento JSON en una **API** (apartado 7).
6. La respuesta atraviesa el *middleware* en orden inverso y vuelve al servidor, que la envía al navegador.

En todos los pasos intervienen las **medidas de seguridad** del framework (apartado 8).

2.7 Control de versiones

Un proyecto Django se guarda en un repositorio Git, pero no todo lo que hay en la carpeta debe subirse. El fichero `.gitignore`, en la carpeta del proyecto, indica qué se excluye:

```
# .gitignore
.venv/
__pycache__/
*.pyc
db.sqlite3
.vscode/
```

Excluido	Motivo
`.venv/`	Se reconstruye con `pip install -r requirements.txt`
`__pycache__/`, `*.pyc`	*Bytecode* que Python genera al ejecutar; se regenera solo
`db.sqlite3`	Datos locales de cada desarrollador; la estructura de la base de datos ya está en las migraciones
`.vscode/`	Preferencias personales del editor

Para reconstruir el proyecto en otro equipo bastan tres órdenes: crear el entorno virtual, `pip install -r requirements.txt` y `python manage.py migrate`.

Ejercicio 2.4. Repositorio del proyecto

Objetivo: preparar el proyecto para el control de versiones sin subir ficheros que no corresponden.

1. Cread el fichero `.gitignore` con el contenido anterior.
2. Si tenéis Git instalado, ejecutad en la carpeta del proyecto:

```
> git init
> git add .
> git status
```

3. Si no tenéis Git, comprimid la carpeta del proyecto en un `.zip` **excluyendo** `.venv` y `db.sqlite3`, y comprobad su tamaño.

Verificación. `git status` lista como ficheros nuevos `.gitignore`, `config/...`, `incidencias/...`, `manage.py` y `requirements.txt`, y **no** lista `.venv`, `db.sqlite3` ni ninguna carpeta `__pycache__`. El `.zip` sin entorno virtual ocupa menos de 50 KB; con `.venv` ocuparía decenas de megabytes.

> ***Para reflexionar.*** *`settings.py` contiene a la vez ajustes que son iguales en todos los equipos (`INSTALLED_APPS`) y otros que deberían cambiar en producción (`DEBUG`, `SECRET_KEY`, `DATABASES`). ¿Cómo separaríais unos de otros?*

Capítulo 3

Rutas, vistas y plantillas

Para empezar

1. En una aplicación PHP sin framework, ¿cómo se decide qué fichero atiende la dirección `/incidencias.php?id=3`? ¿Y en Spring Boot, qué indica la anotación `@GetMapping("/incidencias/{id}")`?
2. ¿Qué ventajas tiene una dirección como `/incidencias/3/` frente a `/ver.php?id=3`?
3. Si veinte páginas comparten la misma cabecera y el mismo pie, ¿cómo evitaríais copiar ese HTML veinte veces?

3.1 El sistema de rutas

Una **ruta** (*URL pattern*) asocia un patrón de dirección con la vista que la atiende. En el patrón MVC, el conjunto de rutas y vistas cumple el papel del **controlador**: recibe la petición, decide qué hacer y devuelve la respuesta. Django busca la tabla de rutas en el módulo indicado por `ROOT_URLCONF` (`config/urls.py`), que contiene una lista llamada `urlpatterns`.

Al llegar una petición, Django recorre `urlpatterns` **en orden** y se queda con el **primer** patrón que coincide. La comparación se hace solo con la ruta: no intervienen el dominio, el método HTTP ni los parámetros de consulta (lo que va detrás de `?`). Si ningún patrón coincide, Django responde con un error 404.

3.1.1 La función `path()`

```
path(ruta, vista, name="nombre")
```

Argumento	Significado
`ruta`	Patrón de la dirección, **sin barra inicial** y, por convenio, con barra final: `"incidencias/"`

Argumento	Significado
`vista`	La función que atiende la petición (sin paréntesis), o `Clase.as_view()` si es una vista basada en clase, o `include(...)` para delegar en otra tabla de rutas
`name`	Nombre de la ruta, para referirse a ella sin escribir la dirección

3.1.2 Convertidores de ruta

Una parte de la ruta puede ser variable. Se escribe entre ángulos con la forma `<convertidor:nombre>`. Django comprueba que el fragmento encaja con el convertidor, lo convierte al tipo correspondiente y lo pasa a la vista como **argumento por nombre**:

Convertidor	Admite	Valor que recibe la vista	Ejemplo
`str`	Cualquier texto sin `/` (es el convertidor por defecto)	`str`	`<str:usuario>` o `<usuario>`
`int`	Cero o un entero positivo	`int`	`<int:pk>`
`slug`	Letras, números, guiones y guiones bajos	`str`	`<slug:categoria>`
`uuid`	Un UUID con guiones	`uuid.UUID`	`<uuid:codigo>`
`path`	Cualquier texto, incluido `/`	`str`	`<path:ruta>`

Si el fragmento no encaja (por ejemplo, `/incidencias/abc/` con `<int:pk>`), el patrón **no coincide** y Django sigue buscando; si ninguno coincide, devuelve 404. Así la vista nunca recibe un valor con un tipo inesperado. Para patrones más complejos existe `re_path()`, que usa expresiones regulares; no se necesitará en este módulo.

En Django, el identificador de un objeto se suele llamar `pk` (*primary key*, clave primaria). Se evita llamar a un parámetro `id` porque es el nombre de una función integrada de Python.

3.1.3 Inclusión de rutas y espacios de nombres

Como se vio en la Unidad 1, cada aplicación define sus rutas en su propio `urls.py` y el proyecto las incluye con `include()`. El prefijo de `include()` se antepone a todas las rutas incluidas: con `path("gestion/", include("incidencias.urls"))`, la ruta `"incidencias/<int:pk>/"` de la aplicación respondería en `/gestion/incidencias/3/`.

Si en el `urls.py` de la aplicación se define la variable `app_name`, sus nombres de ruta quedan dentro de un **espacio de nombres** y se escriben con la forma `aplicacion:nombre` (`incidencias:detalle`). Así dos aplicaciones pueden tener una ruta llamada `lista` sin confundirse.

```
# config/urls.py
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("incidencias.urls")),
]
```

```
# incidencias/urls.py
from django.urls import path

from . import views

app_name = "incidencias"

urlpatterns = [
    path("", views.lista, name="lista"),
    path("incidencias/<int:pk>/", views.detalle, name="detalle"),
]
```

3.1.4 Inversión de rutas

Escribir direcciones a mano dentro del código es frágil: si cambia una ruta, hay que buscar todas sus apariciones. Django permite **invertir** una ruta: a partir de su nombre y sus parámetros, calcula la dirección. En el código Python se usa `reverse()`, y en las plantillas, la etiqueta `{% url %}` (apartado 3.5):

```
>>> from django.urls import reverse
>>> reverse("incidencias:lista")
'/'
>>> reverse("incidencias:detalle", args=[3])
'/incidencias/3/'
>>> reverse("incidencias:detalle", kwargs={"pk": 3})
'/incidencias/3/'
```

Si se cambia la ruta en `urls.py`, todos los enlaces generados con `reverse()` o `{% url %}` cambian solos.

3.2 Vistas de función

Una **vista** recibe un objeto `HttpRequest` y devuelve un objeto `HttpResponse`. Si la ruta tiene parámetros, la vista los recibe como argumentos adicionales con el mismo nombre que en el patrón.

3.2.1 La petición

Atributo	Contenido	Ejemplo
`request.method`	Método HTTP, en mayúsculas	`"GET"`, `"POST"`
`request.path`	Ruta pedida	`"/incidencias/3/"`
`request.GET`	Parámetros de consulta (detrás de `?`), como un diccionario	`request.GET.get("estado")`
`request.POST`	Datos de un formulario enviado por POST	`request.POST["titulo"]`
`request.headers`	Cabeceras de la petición	`request.headers["User-Agent"]`
`request.COOKIES`	Cookies recibidas	
`request.user`	Usuario autenticado, o `AnonymousUser` (lo añade un *middleware*, apartado 4)	`request.user.is_authenticated`

`request.GET` y `request.POST` son objetos `QueryDict`, que funcionan como diccionarios. Conviene leerlos con `.get("clave")`, que devuelve `None` si el parámetro no existe, en lugar de con corchetes, que lanzan un error.

3.2.2 Las respuestas

Clase o función	Respuesta	Código
`HttpResponse("texto")`	Texto o HTML	200
`HttpResponse("...", status=201)`	Con otro código de estado	El indicado
`JsonResponse(diccionario)`	Documento JSON con `Content-Type: application/json`	200
`render(request, plantilla, contexto)`	HTML generado con una plantilla (apartado 3.5)	200
`redirect("app:nombre", pk=3)`	Redirección a otra ruta, por nombre o por dirección	302
`raise Http404("mensaje")`	No es una respuesta: es una excepción que Django convierte en una página 404	404

`render()` y `redirect()` son **atajos** (*shortcuts*) del módulo `django.shortcuts`: funciones que resumen en una línea tareas habituales.

3.2.3 Primeras vistas de la aplicación

Hasta que se definan los modelos (apartado 5), las vistas trabajarán con una lista de diccionarios guardada en memoria. Es una solución provisional que permite estudiar las rutas y las plantillas por separado de la base de datos.

```
# incidencias/views.py
from django.http import Http404, JsonResponse
from django.shortcuts import redirect, render

INCIDENCIAS = [
    {"id": 1, "titulo": "Proyector del aula 12 no enciende", "prioridad": "alta",
 ↪  "estado": "abierta"},
    {"id": 2, "titulo": "Ratón roto en el puesto 7", "prioridad": "baja", "estado":
 ↪  "resuelta"},
    {"id": 3, "titulo": "No funciona la wifi en la biblioteca", "prioridad": "alta",
 ↪  "estado": "en_curso"},
]


def buscar(pk):
    for incidencia in INCIDENCIAS:
        if incidencia["id"] == pk:
            return incidencia
    raise Http404("La incidencia no existe")


def lista(request):
    estado = request.GET.get("estado")
    incidencias = [i for i in INCIDENCIAS if estado is None or i["estado"] ==
 ↪  estado]
    contexto = {"incidencias": incidencias, "estado": estado}
    return render(request, "incidencias/lista.html", contexto)


def detalle(request, pk):
    return render(request, "incidencias/detalle.html", {"incidencia": buscar(pk)})


def detalle_json(request, pk):
    return JsonResponse(buscar(pk), json_dumps_params={"ensure_ascii": False})


def ultima(request):
    return redirect("incidencias:detalle", pk=INCIDENCIAS[-1]["id"])
```

Las vistas `lista` y `detalle` necesitan dos plantillas que se escribirán en el apartado 3.5. Para probar las rutas antes, el ejercicio siguiente empieza con respuestas de texto.

Ejercicio 3.1. Rutas y vistas de función

Objetivo: definir rutas con parámetros y comprobar el comportamiento de los convertidores.

1. Escribid en `incidencias/views.py` la lista `INCIDENCIAS` y la función `buscar` anteriores, y estas dos vistas provisionales:

```
# incidencias/views.py (vistas provisionales)
from django.http import HttpResponse


def lista(request):
    titulos = ", ".join(i["titulo"] for i in INCIDENCIAS)
    return HttpResponse(f"{len(INCIDENCIAS)} incidencias: {titulos}")


def detalle(request, pk):
    return HttpResponse(buscar(pk)["titulo"])
```

2. Cread `incidencias/urls.py` y modificad `config/urls.py` como en el apartado 3.1.

3. Arrancad el servidor y abrid en el navegador: `/`, `/incidencias/2/`, `/incidencias/9/` y `/incidencias/abc/`.

Verificación.

- `/` muestra `3 incidencias: Proyector del aula 12 no enciende, Ratón roto en el puesto 7, No funciona la wifi en la biblioteca`.
- `/incidencias/2/` muestra `Ratón roto en el puesto 7`.
- `/incidencias/9/` devuelve 404 con una página de depuración que muestra el mensaje «**La incidencia no existe**»: la ruta coincide, pero la vista lanza `Http404`.
- `/incidencias/abc/` devuelve 404 con la lista de patrones probados (`admin/`, la ruta vacía e `incidencias/<int:pk>/`): ningún patrón coincide, porque `abc` no es un entero, y la vista ni siquiera se ejecuta.

Ejercicio 3.2. Parámetros de consulta, JSON y redirecciones

Objetivo: leer parámetros de consulta y devolver respuestas JSON y redirecciones.

1. Modificad la vista provisional `lista` para que filtre por el parámetro `estado`, manteniendo la respuesta de texto:

```
# incidencias/views.py (vista provisional)
def lista(request):
    estado = request.GET.get("estado")
    incidencias = [i for i in INCIDENCIAS if estado is None or i["estado"] ==
↪   estado]
    titulos = ", ".join(i["titulo"] for i in incidencias)
    return HttpResponse(f"{len(incidencias)} incidencias: {titulos}")
```

2. Añadid las vistas `detalle_json` y `ultima` y sus rutas:

```
# incidencias/urls.py (rutas añadidas a urlpatterns)
    path("incidencias/<int:pk>/json/", views.detalle_json,
↪   name="detalle_json"),
    path("incidencias/ultima/", views.ultima, name="ultima"),
```

3. Haced estas peticiones desde otra terminal:

```
> curl.exe "http://127.0.0.1:8000/?estado=alta"
> curl.exe "http://127.0.0.1:8000/?estado=abierta"
> curl.exe -i http://127.0.0.1:8000/incidencias/2/json/
> curl.exe -i http://127.0.0.1:8000/incidencias/ultima/
```

En PowerShell, una dirección con `?` o `&` debe ir entre comillas.

Verificación.

- `?estado=alta` responde `0 incidencias:` (ningún estado se llama `alta`; es una prioridad).
- `?estado=abierta` responde `1 incidencias: Proyector del aula 12 no enciende`.
- `/incidencias/2/json/` responde `HTTP/1.1 200 OK`, `Content-Type: application/json` y el cuerpo `{"id": 2, "titulo": "Ratón roto en el puesto 7", "prioridad": "baja", "estado": "resuelta"}`. Si la terminal muestra mal las tildes (`RatÃ³n`), es un problema de codificación de la consola: en el navegador se ven correctamente.
- `/incidencias/ultima/` responde `HTTP/1.1 302 Found` con la cabecera `Location: /incidencias/3/`.

Ejercicio 3.3. Nombres de ruta y `reverse()`

Objetivo: calcular direcciones a partir del nombre de las rutas.

1. Abrid la consola de Django con `python manage.py shell` (es el intérprete de Python con el proyecto cargado; se estudiará en el apartado 6).

2. Ejecutad:

```
>>> from django.urls import reverse
>>> reverse("incidencias:detalle", args=[2])
>>> reverse("incidencias:detalle_json", kwargs={"pk": 2})
>>> reverse("incidencias:detalles", args=[2])
```

3. Salid con `exit()`. En `config/urls.py`, cambiad `path("", include(...))` por `path("gestion/", include(...))` y repetid el paso 2. Después, dejadlo como estaba.

Verificación. Las dos primeras órdenes devuelven `'/incidencias/2/'` y `'/incidencias/2/json/'`. La tercera lanza `NoReverseMatch: Reverse for 'detalles' not found. 'detalles' is not a valid view function or pattern name.`. Con el prefijo `gestion/`, las direcciones pasan a ser `'/gestion/incidencias/2/'` y `'/gestion/incidencias/2/json/'` sin haber cambiado nada más.

3.3 Vistas basadas en clase

Una **vista basada en clase** (*class-based view*, CBV) es una clase que hereda de `django.views.View` y define un método por cada método HTTP que atiende: `get()`, `post()`, `put()`, `delete()`… Django llama al método que corresponde a la petición; si la clase no lo define, responde con **405 Method Not Allowed**.

En `urls.py` no se pasa la clase, sino el resultado de `as_view()`, que construye una función vista a partir de ella:

```
# incidencias/views.py (añadido)
from django.views import View
from django.views.generic import TemplateView


class EstadoView(View):
    def get(self, request):
        return JsonResponse({"aplicacion": "incidencias", "total":
        ↪ len(INCIDENCIAS)})


class AcercaView(TemplateView):
    template_name = "incidencias/acerca.html"
```

```
# incidencias/urls.py (rutas añadidas a urlpatterns)
    path("estado/", views.EstadoView.as_view(), name="estado"),
    path("acerca/", views.AcercaView.as_view(), name="acerca"),
```

Las vistas basadas en clase tienen dos ventajas:

- **Separan los métodos HTTP** sin cadenas de `if request.method == "POST"`.
- **Se reutilizan por herencia.** Django incluye **vistas genéricas** que ya resuelven las tareas más comunes y que solo hay que configurar con atributos de clase:

Vista genérica	Tarea	Apartado
`TemplateView`	Mostrar una plantilla	3
`RedirectView`	Redirigir a otra dirección	3
`ListView`	Listar objetos de un modelo	6
`DetailView`	Mostrar un objeto	6

Vista genérica	Tarea	Apartado
`CreateView`, `UpdateView`, `DeleteView`	Crear, modificar y borrar con un formulario	6

`AcercaView` es un ejemplo: no tiene ningún método, solo el atributo `template_name`; `TemplateView` ya sabe atender `GET` mostrando esa plantilla.

	Vista de función	Vista basada en clase
Definición	`def vista(request, ...)`	`class Vista(View)` con métodos `get()`, `post()`…
En `urls.py`	`views.vista`	`views.Vista.as_view()`
Métodos HTTP	Con `if request.method == ...`	Un método por cada uno; 405 automático para el resto
Reutilización	Llamando a funciones auxiliares	Herencia y vistas genéricas
Cuándo usarla	Lógica sencilla o muy particular	Operaciones habituales sobre modelos

Ejercicio 3.4. Vistas basadas en clase

Objetivo: crear una vista basada en clase y comprobar qué métodos HTTP admite.

1. Añadid `EstadoView` y su ruta (`AcercaView` se completará en el ejercicio 3.6, cuando exista su plantilla).
2. Haced estas peticiones:

```
> curl.exe http://127.0.0.1:8000/estado/
> curl.exe -i -X OPTIONS http://127.0.0.1:8000/estado/
> curl.exe -i -X POST http://127.0.0.1:8000/estado/
```

Verificación. La primera petición devuelve `{"aplicacion": "incidencias", "total": 3}`. La segunda devuelve `HTTP/1.1 200 OK` con la cabecera `Allow: GET, HEAD, OPTIONS`: la vista solo define `get()`, y Django añade `HEAD` y `OPTIONS` automáticamente. La tercera devuelve `HTTP/1.1 403 Forbidden` y no 405: antes de llegar a la vista, el *middleware* de protección CSRF rechaza cualquier POST que no lleve un token válido (apartados 4 y 8).

3.4 Plantillas

Una **plantilla** (*template*) es un fichero de texto, normalmente HTML, con marcas especiales que Django sustituye por datos al generar la respuesta. En el patrón MTV es la capa de presentación. La vista prepara los datos en un diccionario, el **contexto**, y la función `render()` combina plantilla y contexto:

```
return render(request, "incidencias/lista.html", {"incidencias": incidencias,
 ↪  "estado": estado})
```

3.4.1 Dónde se guardan

Con `APP_DIRS: True` en el ajuste `TEMPLATES` (es el valor generado), Django busca las plantillas en la carpeta `templates` de cada aplicación instalada. Por convenio, dentro de `templates` se crea otra carpeta con el nombre de la aplicación:

```
incidencias/
    templates/
        incidencias/
            base.html
            lista.html
            detalle.html
```

La carpeta intermedia evita conflictos: Django une las carpetas `templates` de todas las aplicaciones, y si dos aplicaciones tuvieran una `lista.html` en la raíz, solo se usaría una. Por eso el nombre que se pasa a `render()` es `"incidencias/lista.html"`.

3.4.2 Sintaxis del lenguaje de plantillas

Marca	Función	Ejemplo
`{{ variable }}`	Muestra un valor del contexto	`{{ incidencia.titulo }}`
`{{ variable|filtro }}`	Muestra un valor transformado por un filtro	`{{ incidencia.titulo|upper }}`
`{% etiqueta %}`	Ejecuta lógica: condicionales, bucles, herencia	`{% if incidencias %}...{% endif %}`
`{# comentario #}`	Comentario de una línea, no aparece en el HTML generado	`{# lista.html #}`

El **punto** sirve para todo tipo de accesos. Ante `{{ incidencia.titulo }}`, Django prueba, por este orden, la clave de diccionario `incidencia["titulo"]`, el atributo `incidencia.titulo` y el índice de lista. Si el resultado es un método, lo llama sin argumentos (en las plantillas no se escriben paréntesis). Si la variable no existe, muestra una cadena vacía en lugar de dar un error.

El lenguaje de plantillas es intencionadamente limitado: no permite ejecutar código Python arbitrario. La lógica va en la vista; la plantilla solo presenta.

3.4.3 Etiquetas principales

Etiqueta	Uso
`{% if c %} ... {% elif c %} ... {% else %} ... {% endif %}`	Condicional. Admite `and`, `or`, `not`, `==`, `!=`, `<`, `in`
`{% for x in lista %} ... {% empty %} ... {% endfor %}`	Bucle; el bloque `empty` se muestra si la lista está vacía
`{% url "app:nombre" arg %}`	Dirección de una ruta por su nombre (la versión de `reverse()` para plantillas)
`{% extends "base.html" %}`	La plantilla hereda de otra (apartado 3.6)
`{% block nombre %} ... {% endblock %}`	Zona que una plantilla hija puede sustituir
`{% include "fragmento.html" %}`	Inserta otra plantilla
`{% load static %}` y `{% static "ruta" %}`	Dirección de un fichero estático
`{% csrf_token %}`	Campo oculto con el token CSRF, obligatorio en los formularios POST (apartado 6)

Dentro de un `{% for %}`, la variable `forloop` informa de la vuelta actual: `forloop.counter` (empieza en 1), `forloop.first`, `forloop.last`.

3.4.4 Filtros más usados

Filtro	Ejemplo	Resultado
`upper`, `lower`, `capfirst`	`{{ "alta"|upper }}`	`ALTA`
`length`	`{{ incidencias|length }}`	`3`
`default`	`{{ incidencia.autor|default:"anónimo" }}`	El valor, o `anónimo` si está vacío
`pluralize`	`incidencia{{ total|pluralize }}`	`incidencia` si `total` vale 1; `incidencias` en otro caso
`truncatechars`	`{{ titulo|truncatechars:15 }}`	Corta el texto a 15 caracteres con …
`date`	`{{ fecha|date:"d/m/Y H:i" }}`	`28/10/2026 09:15`
`linebreaksbr`	`{{ descripcion|linebreaksbr }}`	Convierte los saltos de línea en ` `

Filtro	Ejemplo	Resultado
`yesno`	`{{ activo|yesno:"sí,no" }}`	`sí` o `no`
`safe`	`{{ html|safe }}`	Desactiva el escapado automático (peligroso, apartado 8)

Los filtros se pueden encadenar: `{{ incidencias|length|pluralize }}`.

Ejercicio 3.5. Primeras plantillas

Objetivo: generar las páginas de listado y de detalle con plantillas y enlaces por nombre de ruta.

1. Volved a poner en `views.py` las versiones definitivas de `lista` y `detalle` (las que usan `render()`) y borrad las provisionales.
2. Cread `incidencias/templates/incidencias/lista.html` con un documento HTML completo (`<!DOCTYPE html>`, `<head>` con `<title>Incidencias</title>` y `<body>`) cuyo cuerpo sea:

```
<h1>Incidencias{% if estado %} ({{ estado }}){% endif %}</h1>
<p>{{ incidencias|length }} incidencia{{ incidencias|length|pluralize }}</p>
<ul>
  {% for incidencia in incidencias %}
    <li class="{{ incidencia.prioridad }}">
      {{ forloop.counter }}.
      <a href="{% url 'incidencias:detalle' incidencia.id %}">{{
      ↪  incidencia.titulo }}</a>
      ({{ incidencia.estado }})
    </li>
  {% empty %}
    <li>No hay incidencias.</li>
  {% endfor %}
</ul>
```

3. Cread `detalle.html` de la misma forma, con el título de la incidencia en `<h1>`, la prioridad en mayúsculas y el estado en un párrafo, y un enlace al listado hecho con `{% url 'incidencias:lista' %}`.
4. Abrid `/`, `/?estado=abierta`, `/?estado=cerrada` y pulsad en una de las incidencias.

Si al abrir una página aparece `TemplateDoesNotExist`, comprobad la ruta de la carpeta (`incidencias\templates\incidencias\`), que la aplicación está en `INSTALLED_APPS` y reiniciad el servidor: Django no detecta las carpetas `templates` creadas con el servidor en marcha.

Verificación. `/` muestra el encabezado «Incidencias», el párrafo «3 incidencias» y una lista numerada del 1 al 3 con los títulos enlazados. `/?estado=abierta` muestra «Incidencias

(abierta)» y «1 incidencia» (en singular). `/?estado=cerrada` muestra «0 incidencias» y «No hay incidencias.». Al pulsar en «Ratón roto en el puesto 7», la dirección es `/incidencias/2/` y la página muestra `Prioridad: BAJA`. El código fuente de la página (`Ctrl + U`) muestra los enlaces ya resueltos: `href="/incidencias/2/"`.

3.5 Herencia de plantillas y ficheros estáticos

Las páginas de una aplicación comparten estructura: cabecera, menú, pie, hojas de estilo. La **herencia de plantillas** evita repetirla. Una plantilla **base** define el esqueleto y marca con `{% block %}` las zonas que cambian; cada página **hereda** de la base con `{% extends %}` y redefine solo esos bloques.

```
{# incidencias/templates/incidencias/base.html #}
{% load static %}
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>{% block titulo %}Incidencias{% endblock %}</title>
  <link rel="stylesheet" href="{% static 'incidencias/estilos.css' %}">
</head>
<body>
  <header>
    <a href="{% url 'incidencias:lista' %}">Incidencias</a>
    | <a href="{% url 'incidencias:acerca' %}">Acerca de</a>
  </header>
  <main>
    {% block contenido %}{% endblock %}
  </main>
  <footer>IES Hontanilla · Departamento de Informática</footer>
</body>
</html>
```

```
{# incidencias/templates/incidencias/lista.html #}
{% extends "incidencias/base.html" %}

{% block titulo %}Incidencias{% endblock %}

{% block contenido %}
<h1>Incidencias{% if estado %} ({{ estado }}){% endif %}</h1>
<p>{{ incidencias|length }} incidencia{{ incidencias|length|pluralize }}</p>
<ul>
  {% for incidencia in incidencias %}
    <li class="{{ incidencia.prioridad }}">
      {{ forloop.counter }}.
      <a href="{% url 'incidencias:detalle' incidencia.id %}">{{ incidencia.titulo
      ↪ }}</a>
      ({{ incidencia.estado }})
    </li>
  {% empty %}
    <li>No hay incidencias.</li>
```

```
  {% endfor %}
</ul>
{% endblock %}
```

```
{# incidencias/templates/incidencias/detalle.html #}
{% extends "incidencias/base.html" %}

{% block titulo %}{{ incidencia.titulo }} | {{ block.super }}{% endblock %}

{% block contenido %}
<h1>{{ incidencia.titulo }}</h1>
<p>Prioridad: {{ incidencia.prioridad|upper }} · Estado: {{ incidencia.estado }}</p>
<p><a href="{% url 'incidencias:lista' %}">Volver al listado</a></p>
{% endblock %}
```

Reglas de la herencia:

- `{% extends %}` debe ser **la primera etiqueta** de la plantilla hija (solo puede ir precedida de comentarios `{# #}`).
- Todo lo que la hija escriba fuera de un `{% block %}` se ignora.
- Un bloque que la hija no redefine conserva el contenido de la base.
- `{{ block.super }}` inserta el contenido que el bloque tenía en la plantilla padre. En `detalle.html`, el título de la pestaña es el de la incidencia seguido de `| Incidencias`.
- Puede haber varios niveles de herencia: una plantilla puede extender a otra que a su vez extiende a la base.

3.5.1 Ficheros estáticos

Los **ficheros estáticos** son los que se sirven tal cual, sin procesar: hojas de estilo, scripts de JavaScript, imágenes. Con la aplicación `django.contrib.staticfiles` (instalada por defecto), Django los busca en la carpeta `static` de cada aplicación, con la misma subcarpeta con el nombre de la aplicación que las plantillas:

```
/* incidencias/static/incidencias/estilos.css */
body { font-family: sans-serif; max-width: 50rem; margin: 0 auto; }
.alta { color: darkred; font-weight: bold; }
.baja { color: gray; }
```

En la plantilla se carga la librería de etiquetas `static` con `{% load static %}` y se obtiene la dirección con `{% static 'incidencias/estilos.css' %}`, que da como resultado `/static/incidencias/estilos.css` (el prefijo es el ajuste `STATIC_URL`). El servidor de desarrollo los sirve automáticamente mientras `DEBUG = True`; en producción los servirá Nginx (Unidad 4).

Ejercicio 3.6. Plantilla base y estilos

Objetivo: aplicar la herencia de plantillas y servir una hoja de estilos.

1. Cread `base.html`, `estilos.css` y una plantilla `acerca.html` que extienda la base, con el título de pestaña «Acerca de» y un párrafo que describa la aplicación.
2. Reescribid `lista.html` y `detalle.html` para que hereden de la base, como en los ejemplos anteriores.
3. Añadid `AcercaView` y su ruta `acerca/`.
4. **Reiniciad el servidor** (la carpeta `static` es nueva) y abrid `/`, `/incidencias/2/` y `/acerca/`.

Verificación.

- Las tres páginas tienen la misma cabecera con los enlaces «Incidencias» y «Acerca de» y el mismo pie.
- En el listado, las incidencias de prioridad alta aparecen en rojo oscuro y en negrita, y la de prioridad baja, en gris.
- La pestaña de `/incidencias/2/` muestra «Ratón roto en el puesto 7 | Incidencias».
- `curl.exe -I http://127.0.0.1:8000/static/incidencias/estilos.css` devuelve `HTTP/1.1 200 OK` con `Content-Type: text/css`.
- El código fuente de las páginas no contiene ninguna marca `{% %}` ni `{{ }}`: Django las ha procesado todas.

3.6 El contexto de las plantillas

Además de las variables que pasa la vista, todas las plantillas reciben automáticamente algunas variables de los **procesadores de contexto** (*context processors*) configurados en `TEMPLATES`:

Procesador	Variables que añade
`django.template.context_processors.request`	`request`: la petición actual (`{{ request.path }}`)
`django.contrib.auth.context_processors.auth`	`user`: el usuario actual (`{% if user.is_authenticated %}`), y `perms`: sus permisos
`django.contrib.messages.context_processors.messages`	`messages`: los mensajes pendientes de mostrar

Así, la plantilla base puede mostrar el nombre del usuario conectado sin que todas las vistas tengan que pasarlo (se hará en el apartado 8).

Ejercicio 3.7. Contexto, filtros y enlaces con parámetros

Objetivo: usar el contexto automático y combinar etiquetas y filtros.

1. En `lista.html`, añadid antes de la lista tres enlaces para filtrar por estado: «Abiertas», «En curso» y «Resueltas». Cada uno debe construirse con `{% url 'incidencias:lista' %}` seguido de `?estado=...`.

2. Añadid un cuarto enlace, «Todas», que solo aparezca si hay un filtro aplicado (usad la variable `estado` del contexto).
3. En `base.html`, añadid al pie la ruta de la página actual con `{{ request.path }}`.
4. En `detalle.html`, mostrad el título con `truncatechars:20` dentro del `<h1>` y el título completo en un párrafo aparte.

Verificación. En `/` aparecen los enlaces «Abiertas», «En curso» y «Resueltas», pero no «Todas». Al pulsar «En curso», la dirección es `/?estado=en_curso`, la página muestra «Incidencias (en_curso)» y «1 incidencia», y aparece el enlace «Todas». El pie muestra `/` en el listado y `/incidencias/3/` en el detalle de la tercera incidencia. En esa página, el `<h1>` muestra «No funciona la wifi…» (19 caracteres más los puntos suspensivos, 20 en total).

> ***Para reflexionar.*** *El lenguaje de plantillas de Django no permite escribir código Python arbitrario, mientras que PHP permite mezclar cualquier código con el HTML. ¿Qué problemas evita esa limitación? ¿Qué inconvenientes tiene?*

Capítulo 4

Middleware

Para empezar

1. Si quisierais anotar en un registro cuánto tarda **cada** petición de la aplicación, ¿en qué vistas escribiríais ese código?
2. En el Ejercicio 3.4, un POST a `/estado/` devolvió 403 sin llegar a la vista. ¿Quién creéis que lo rechazó?
3. Las respuestas de Django incluyen cabeceras como `X-Frame-Options: DENY` que ninguna vista ha escrito. ¿De dónde salen?

4.1 Qué es un middleware

Un **middleware** es un componente que se sitúa **entre el servidor y las vistas** y procesa **todas** las peticiones y todas las respuestas de la aplicación. Sirve para las tareas transversales, las que afectan a toda la aplicación y no a una vista concreta: seguridad, sesiones, autenticación, registro de peticiones, compresión, cabeceras comunes…

Django organiza el middleware como **capas** alrededor de la vista. Cada petición atraviesa las capas de fuera hacia dentro, en el orden de la lista `MIDDLEWARE` de `settings.py`; la respuesta las atraviesa en orden inverso, de dentro hacia fuera:

```
petición                                                    respuesta
   |                                                           ^
   v                                                           |
SecurityMiddleware ..................................... (7)
   SessionMiddleware ................................... (6)
      CommonMiddleware ................................. (5)
         CsrfViewMiddleware ............................ (4)
            AuthenticationMiddleware ................... (3)
               MessageMiddleware ....................... (2)
                  XFrameOptionsMiddleware .............. (1)
                     vista  -->  construye la respuesta
```

Cualquier capa puede:

- **Modificar la petición** antes de pasarla a la siguiente (por ejemplo, `AuthenticationMiddleware` añade `request.user`).
- **Cortar el recorrido** y devolver una respuesta sin llamar a las capas interiores ni a la vista (por ejemplo, `CsrfViewMiddleware` devuelve 403 ante un POST sin token). En ese caso, la respuesta solo atraviesa de vuelta las capas **exteriores** a la que cortó.
- **Modificar la respuesta** al volver (por ejemplo, `XFrameOptionsMiddleware` añade la cabecera `X-Frame-Options`).

Este mecanismo es también un ejemplo de inversión de control: el desarrollador escribe la capa y el framework decide cuándo llamarla.

En otros frameworks el concepto existe con nombres parecidos: *middleware* en Laravel y en Express, *filtros* (*filters*) e interceptores en Spring Boot.

4.2 El middleware de serie

El proyecto generado por `startproject` activa siete componentes:

Middleware	Qué hace	Cómo se observa
`SecurityMiddleware`	Añade cabeceras de seguridad y, en producción, puede redirigir a HTTPS	Cabeceras `X-Content-Type-Options: nosniff`, `Referrer-Policy: same-origin`, `Cross-Origin-Opener-Policy: same-origin`
`SessionMiddleware`	Carga y guarda la **sesión** del usuario (datos que se conservan entre peticiones)	`request.session` en las vistas; cookie `sessionid`
`CommonMiddleware`	Normaliza las direcciones: añade la barra final (`APPEND_SLASH`) o el `www`	Redirección 301 de `/acerca` a `/acerca/`
`CsrfViewMiddleware`	Rechaza las peticiones POST, PUT, PATCH y DELETE sin un token CSRF válido	Error 403 «Prohibido (403)» (apartado 8)
`AuthenticationMiddleware`	Identifica al usuario a partir de la sesión	`request.user` en las vistas y `user` en las plantillas
`MessageMiddleware`	Gestiona los mensajes de un solo uso entre peticiones	`messages` en las plantillas

Middleware	Qué hace	Cómo se observa
`XFrameOptionsMiddleware`	Impide que otras webs muestren la aplicación dentro de un `<iframe>` (protección frente al *clickjacking*)	Cabecera `X-Frame-Options: DENY`

4.2.1 Orden de los componentes

Algunos componentes dependen de otros, y por eso la lista tiene un orden concreto:

- `AuthenticationMiddleware` necesita la sesión para saber quién es el usuario: debe ir **después** de `SessionMiddleware`.
- `MessageMiddleware` guarda los mensajes en la sesión: también va después de `SessionMiddleware`.
- `SecurityMiddleware` va el primero para que sus redirecciones a HTTPS ocurran antes de hacer cualquier otro trabajo.
- Cualquier middleware propio que use `request.user` **antes** de llamar a la vista debe ir después de `AuthenticationMiddleware`.

La documentación oficial indica el orden recomendado de todos los componentes de Django.

Ejercicio 4.1. Identificar el middleware en las respuestas

Objetivo: reconocer en las peticiones y respuestas HTTP el trabajo de los componentes de serie.

1. Con el servidor en marcha, haced estas peticiones:

```
> curl.exe -i http://127.0.0.1:8000/estado/
> curl.exe -i http://127.0.0.1:8000/acerca
```

2. Comentad la línea de `XFrameOptionsMiddleware` en `MIDDLEWARE` (escribiendo `#` delante), guardad y repetid la primera petición. Después, quitad el comentario.

3. Cambiad temporalmente `APPEND_SLASH = False` (al final de `settings.py`) y repetid la segunda petición. Después, borrad esa línea.

Verificación.

- La primera respuesta incluye `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: same-origin` y `Cross-Origin-Opener-Policy: same-origin`, que ninguna vista ha escrito.

- La segunda devuelve `HTTP/1.1 301 Moved Permanently` con `Location: /acerca/`: es `CommonMiddleware`.
- Con `XFrameOptionsMiddleware` comentado, la cabecera `X-Frame-Options` desaparece y las demás se mantienen.
- Con `APPEND_SLASH = False`, `/acerca` devuelve `HTTP/1.1 404 Not Found`.

4.3 Middleware propio

Un middleware es una **fábrica**: Django la llama una sola vez al arrancar, pasándole `get_response`, que es la función que invoca a la siguiente capa (o a la vista, si es la última). La fábrica devuelve otra función, la que se ejecuta en cada petición. Se puede escribir como función o como clase.

4.3.1 Middleware como función

```
# incidencias/middleware.py (versión con funciones)
import time


def tiempo_respuesta(get_response):
    # Código de inicialización: se ejecuta una vez, al arrancar.

    def middleware(request):
        # Antes de la vista
        inicio = time.perf_counter()
        response = get_response(request)
        # Después de la vista
        ms = (time.perf_counter() - inicio) * 1000
        response["X-Tiempo-Respuesta"] = f"{ms:.1f}ms"
        return response

    return middleware
```

4.3.2 Middleware como clase

La forma más habitual es una clase: el constructor `__init__` recibe `get_response` (inicialización) y el método especial `__call__` se ejecuta en cada petición. `__call__` permite que un objeto se use como si fuera una función.

```
# incidencias/middleware.py
import logging
import time

from django.conf import settings
from django.http import HttpResponse

logger = logging.getLogger("incidencias")
```

```
class TiempoRespuestaMiddleware:
    """Mide cuánto tarda cada petición y lo anota en el registro y en una
    ↪ cabecera."""

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        inicio = time.perf_counter()
        response = self.get_response(request)
        ms = (time.perf_counter() - inicio) * 1000
        response["X-Tiempo-Respuesta"] = f"{ms:.1f}ms"
        logger.info("%s %s %s %.1fms", request.method, request.path,
↪   response.status_code, ms)
        return response
```

- `time.perf_counter()` devuelve un instante en segundos con mucha precisión; la diferencia entre dos llamadas es el tiempo transcurrido.
- `response["X-Tiempo-Respuesta"] = ...` añade una cabecera a la respuesta. Por convenio, las cabeceras propias de una aplicación empiezan por `X-`.
- `logger.info(...)` escribe un mensaje en el **registro** (*log*) con el módulo `logging` de la biblioteca estándar. Los `%s` se sustituyen por los argumentos siguientes.

El middleware se activa añadiendo la ruta completa de la clase a `MIDDLEWARE`:

```
# config/settings.py (fragmento)
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
    "incidencias.middleware.TiempoRespuestaMiddleware",
]
```

4.3.3 El registro de la aplicación

Para que los mensajes de `logger.info()` aparezcan en la terminal, hay que configurar el registro con el ajuste `LOGGING`. Esta configuración envía a la consola los mensajes de nivel `INFO` o superior del registrador `incidencias`:

```
# config/settings.py (añadido al final)
LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "handlers": {"consola": {"class": "logging.StreamHandler"}},
    "loggers": {"incidencias": {"handlers": ["consola"], "level": "INFO"}},
}
```

Nivel	Uso
`DEBUG`	Información detallada para depurar
`INFO`	Funcionamiento normal (una petición atendida)
`WARNING`	Algo inesperado que no impide seguir
`ERROR`	Un error que ha impedido completar una operación
`CRITICAL`	Un error grave que puede detener la aplicación

Un registro con niveles es preferible a `print()`: se puede filtrar, enviar a un fichero o a un servicio externo, y desactivar en producción sin tocar el código.

Ejercicio 4.2. Middleware que mide el tiempo de respuesta

Objetivo: escribir, activar y probar un middleware propio.

1. Cread `incidencias/middleware.py` con la clase `TiempoRespuestaMiddleware`.
2. Añadidla al final de `MIDDLEWARE` y añadid el ajuste `LOGGING`.
3. Haced peticiones a `/`, `/estado/` y `/incidencias/9/` con `curl.exe -i` y mirad la terminal del servidor.

Verificación. Todas las respuestas, también la 404, incluyen una cabecera como `X-Tiempo-Respuesta: 1.0ms` (el número varía). En la terminal del servidor, antes de la línea habitual de cada petición, aparece otra con el formato del registro, por ejemplo:

```
GET /estado/ 200 1.0ms
[28/Oct/2026 09:40:12] "GET /estado/ HTTP/1.1" 200 41
```

Si se escribe mal la ruta de la clase en `MIDDLEWARE`, el servidor no arranca y muestra `ImportError` o `ModuleNotFoundError` con el nombre que no ha encontrado.

4.4 Middleware que corta el recorrido

Un middleware puede responder **sin llamar** a `get_response`. Así funciona este segundo ejemplo, que pone la aplicación «en mantenimiento» cuando el ajuste `MANTENIMIENTO` vale `True`, salvo para el panel de administración:

```
# incidencias/middleware.py (añadido al final)
class MantenimientoMiddleware:
    """Si el ajuste MANTENIMIENTO vale True, responde 503 salvo en el panel de
    ↪ administración."""

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
```

```
        if getattr(settings, "MANTENIMIENTO", False) and not
        ↪ request.path.startswith("/admin/"):
            return HttpResponse("Aplicación en mantenimiento. Volved más tarde.",
            ↪ status=503)
        return self.get_response(request)
```

- `from django.conf import settings` da acceso a los ajustes del proyecto desde cualquier módulo. Es la forma correcta de leerlos (no se importa `config.settings` directamente).
- `getattr(settings, "MANTENIMIENTO", False)` lee un ajuste propio y devuelve `False` si no está definido.
- El código **503 Service Unavailable** indica que el servicio no está disponible temporalmente.

Ejercicio 4.3. Orden del middleware y respuestas cortadas

Objetivo: comprobar cómo influye el orden de `MIDDLEWARE` cuando una capa corta el recorrido.

1. Añadid `MantenimientoMiddleware` a `middleware.py` y a `MIDDLEWARE`, **después** de `TiempoRespuestaMiddleware`.
2. Añadid al final de `settings.py` la línea `MANTENIMIENTO = True` y pedid `curl.exe -i http://127.0.0.1:8000/estado/` y `curl.exe -I http://127.0.0.1:8000/admin/login/`.
3. Intercambiad el orden de los dos middleware propios en `MIDDLEWARE` y repetid la primera petición.
4. Dejad `MANTENIMIENTO = False` y el orden del paso 1 (`TiempoRespuestaMiddleware` antes que `MantenimientoMiddleware`).

Verificación.

- En el paso 2, `/estado/` devuelve `HTTP/1.1 503 Service Unavailable`, el texto «Aplicación en mantenimiento. Volved más tarde.» **y** la cabecera `X-Tiempo-Respuesta`; `/admin/login/` devuelve `HTTP/1.1 200 OK`.
- En el paso 3, `/estado/` sigue devolviendo 503, pero **sin** la cabecera `X-Tiempo-Respuesta` y sin línea de registro: `MantenimientoMiddleware`, ahora exterior, corta el recorrido antes de llegar a la capa que mide el tiempo. Las cabeceras `X-Frame-Options` y `X-Content-Type-Options` siguen apareciendo porque sus componentes son exteriores a los dos.
- En el paso 4, `/estado/` vuelve a responder 200.

4.5 Middleware y `request.user`

Las capas de serie dejan información en la petición para las capas interiores y para las vistas. La más usada es `request.user`, que añade `AuthenticationMiddleware`. Un middleware propio que la lea **antes** de llamar a `get_response` tiene que estar situado después de `AuthenticationMiddleware` en la lista; si está antes, el atributo todavía no existe.

Ejercicio 4.4. Dependencias entre capas

Objetivo: comprobar qué ocurre cuando un middleware usa información que añade otra capa posterior.

1. Añadid a `middleware.py` esta clase de prueba:

```python
# incidencias/middleware.py (clase de prueba)
class UsuarioMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        print("Usuario:", request.user)
        return self.get_response(request)
```

2. Añadidla al **principio** de `MIDDLEWARE` y pedid `/estado/`.
3. Movedla al **final** de `MIDDLEWARE` y repetid la petición.
4. Borrad la clase y su línea en `MIDDLEWARE`.

Verificación. En el paso 2, la petición devuelve `HTTP/1.1 500 Internal Server Error` y la terminal muestra `AttributeError: 'WSGIRequest' object has no attribute 'user'`. En el paso 3, devuelve 200 y la terminal muestra `Usuario: AnonymousUser`, porque todavía no hay nadie autenticado (apartado 8).

4.6 Otros métodos del middleware

Además de `__call__`, una clase de middleware puede definir métodos que Django llama en momentos concretos:

Método	Cuándo se llama	Uso típico
`process_view(request, vista, args, kwargs)`	Justo antes de llamar a la vista, cuando ya se sabe cuál es	Comprobar permisos según la vista
`process_exception(request, excepcion)`	Si la vista lanza una excepción	Registrar errores, devolver una página propia
`process_template_response(request, response)`	Si la respuesta es una plantilla que aún no se ha generado	Añadir datos al contexto

`CsrfViewMiddleware`, por ejemplo, hace su comprobación en `process_view`, porque necesita saber si la vista está marcada como exenta.

Cuando una tarea afecta solo a algunas vistas, no se usa middleware sino **decoradores**: funciones que envuelven una vista concreta. Django incluye, entre otros, `@login_required` (exige usuario autenticado, apartado 8), `@require_POST` (solo admite POST) y `@csrf_exempt` (desactiva la protección CSRF en una vista, algo que casi nunca debe hacerse).

	Middleware	Decorador
Alcance	Todas las peticiones	Una vista concreta
Dónde se activa	Lista `MIDDLEWARE` de `settings.py`	Encima de la función vista (`@nombre`) o con un *mixin* en las vistas de clase
Ejemplos	Sesiones, autenticación, CSRF, cabeceras, registro	`@login_required`, `@require_POST`

Para reflexionar. *El middleware se ejecuta en todas las peticiones, también en las que acaban en 404 o en error. ¿Qué cuidado hay que tener con lo que se programa en él? ¿Qué pasaría si un middleware hiciera una consulta lenta a la base de datos?*

Capítulo 5

Modelos y migraciones

Para empezar

1. En los proyectos de PHP o de Java que habéis hecho, ¿cómo se creaban las tablas de la base de datos? ¿Cómo se enteraba un compañero de que habíais añadido una columna?
2. ¿Qué diferencias hay entre escribir `SELECT * FROM incidencia WHERE estado = 'abierta'` en una cadena de texto y llamar a un método como `filtrar(estado="abierta")`?
3. Si una categoría tiene incidencias asociadas, ¿qué debería pasar al intentar borrarla?

5.1 El ORM de Django

Un **ORM** (*Object-Relational Mapping*, mapeo objeto-relacional) es una capa que traduce entre los **objetos** del lenguaje de programación y las **tablas** de una base de datos relacional. El desarrollador trabaja con clases y objetos; el ORM genera y ejecuta el SQL.

Concepto en Python	Concepto en la base de datos
Clase de modelo (`Incidencia`)	Tabla (`incidencias_incidencia`)
Atributo de clase con un campo (`titulo = models.CharField(...)`)	Columna (`titulo varchar(120)`)
Objeto (`Incidencia(titulo="...")`)	Fila
`incidencia.save()`	`INSERT` o `UPDATE`
`Incidencia.objects.filter(...)`	`SELECT ... WHERE ...`
`ForeignKey`	Clave ajena (`REFERENCES`)

Ya conocéis ORM de otros lenguajes: **JPA con Hibernate** en Java (`@Entity`, `@Column`, `@ManyToOne`) y **Eloquent** en Laravel. El de Django sigue la misma idea con una diferencia de enfoque: el modelo es la **única fuente de verdad**. Las tablas no se crean a mano: se describen en `models.py` y Django genera las **migraciones** que las crean y las modifican.

Ventajas de trabajar con un ORM:

- **Independencia del gestor.** El mismo código funciona con SQLite en desarrollo y con PostgreSQL en producción (Unidad 4).
- **Seguridad.** Los valores se pasan siempre como parámetros, nunca concatenados en el SQL, lo que evita la inyección SQL (apartado 8).
- **Productividad.** Los modelos alimentan también el panel de administración, los formularios y los serializadores de la API.

5.2 Definición de modelos

Un **modelo** es una clase que hereda de `django.db.models.Model`. Cada atributo de clase que es un **campo** (*field*) se convierte en una columna. Django añade automáticamente una clave primaria llamada `id` (de tipo `BigAutoField`, según el ajuste `DEFAULT_AUTO_FIELD`), accesible también como `pk`.

5.2.1 Tipos de campo

Campo	Columna	Uso
`CharField(max_length=n)`	`varchar(n)`	Texto corto; `max_length` es obligatorio
`TextField()`	`text`	Texto largo
`IntegerField()`, `PositiveIntegerField()`	`integer`	Números enteros (el segundo no admite negativos)
`DecimalField(max_digits, decimal_places)`	`decimal`	Importes y cantidades exactas
`BooleanField()`	`bool`	Verdadero o falso
`DateField()`, `TimeField()`, `DateTimeField()`	`date`, `time`, `datetime`	Fechas y horas
`EmailField()`, `URLField()`, `SlugField()`	`varchar`	Texto con validación de formato
`ForeignKey(Modelo, on_delete=...)`	`modelo_id` con `REFERENCES`	Relación «muchos a uno»
`OneToOneField(Modelo, on_delete=...)`	Clave ajena única	Relación «uno a uno»
`ManyToManyField(Modelo)`	Tabla intermedia	Relación «muchos a muchos»

5.2.2 Opciones de los campos

Opción	Significado
`max_length=120`	Longitud máxima (en `CharField`, obligatoria)

Opción	Significado
`null=True`	La columna admite `NULL` en la base de datos
`blank=True`	El campo puede dejarse vacío en los formularios (validación)
`default=valor`	Valor por defecto
`unique=True`	No puede haber dos filas con el mismo valor
`choices=...`	Lista cerrada de valores permitidos
`verbose_name="título"`	Nombre legible del campo (se usa en formularios y en el panel de administración); también se puede pasar como primer argumento posicional
`help_text="..."`	Texto de ayuda en los formularios
`auto_now_add=True`	Fecha que se rellena sola al **crear** el objeto (`DateField`, `DateTimeField`)
`auto_now=True`	Fecha que se actualiza sola cada vez que se **guarda** el objeto

`null` afecta a la base de datos y `blank` a la validación. En los campos de texto, el convenio de Django es **no** usar `null=True`: un texto vacío se guarda como cadena vacía (`""`), y así no hay dos formas distintas de representar «sin datos». Por eso `descripcion = models.TextField(blank=True)` es la forma correcta de un texto opcional.

5.2.3 Opciones cerradas

Cuando un campo solo puede tomar unos valores concretos, se definen con una **enumeración** que hereda de `models.TextChoices`. Cada opción tiene un **valor**, que se guarda en la base de datos, y una **etiqueta**, que se muestra al usuario:

```
class Estado(models.TextChoices):
    ABIERTA = "abierta", "Abierta"
    EN_CURSO = "en_curso", "En curso"
    RESUELTA = "resuelta", "Resuelta"
```

`Estado.EN_CURSO` vale `"en_curso"` y `Estado.choices` es la lista de parejas `[("abierta", "Abierta"), ("en_curso", "En curso"), ("resuelta", "Resuelta")]`. Para cada campo con `choices`, Django añade al modelo un método `get_<campo>_display()` que devuelve la etiqueta: `incidencia.get_estado_display()` devuelve `"En curso"`.

5.2.4 Claves ajenas

Una **clave ajena** (`ForeignKey`) relaciona cada objeto con **uno** de otro modelo: cada incidencia pertenece a una categoría y una categoría tiene muchas incidencias. En la tabla se crea la columna `categoria_id`.

El argumento obligatorio `on_delete` indica qué hacer con las incidencias cuando se borra la categoría a la que apuntan:

`on_delete`	Al borrar el objeto referenciado…
`models.CASCADE`	Se borran también los objetos que apuntan a él
`models.PROTECT`	Se impide el borrado (excepción `ProtectedError`)
`models.RESTRICT`	Se impide el borrado, salvo que se borren en la misma operación por otra relación en cascada
`models.SET_NULL`	La clave ajena se pone a `NULL` (requiere `null=True`)
`models.SET_DEFAULT`	La clave ajena toma su valor por defecto
`models.DO_NOTHING`	Django no hace nada (la base de datos puede dar error)

En «Incidencias», borrar una categoría con incidencias se **impide** (`PROTECT`): no se quiere perder información por un descuido. En cambio, si se borra un usuario, se borran sus incidencias (`CASCADE`).

El argumento `related_name` da nombre a la relación **inversa**: con `related_name="incidencias"`, desde una categoría se obtienen sus incidencias con `categoria.incidencias.all()`. Si no se indica, el nombre por defecto es `incidencia_set`.

5.2.5 El modelo de usuario

Django incluye un modelo de usuario (`django.contrib.auth.models.User`) con nombre de usuario, contraseña cifrada, correo, nombre, apellidos y los indicadores `is_staff` (acceso al panel de administración), `is_superuser` e `is_active`. Para relacionar un modelo con los usuarios no se importa `User` directamente, sino que se usa el ajuste `settings.AUTH_USER_MODEL`: así el código sigue funcionando si un proyecto sustituye el modelo de usuario por uno propio.

5.2.6 La clase `Meta`

La clase anidada `Meta` reúne opciones del modelo que no son campos:

Opción	Significado
`ordering = ["-creada"]`	Orden por defecto de las consultas; el signo `-` indica orden descendente
`verbose_name`, `verbose_name_plural`	Nombre legible del modelo en singular y en plural (por defecto, el plural añade una `s`)
`db_table = "nombre"`	Nombre de la tabla, si no se quiere el generado (`aplicacion_modelo`)
`constraints = [...]`	Restricciones de la base de datos, como `models.UniqueConstraint(fields=[...], name="...")` o `models.CheckConstraint(condition=models.Q(...), name="...")`

5.2.7 Los modelos de «Incidencias»

```python
# incidencias/models.py
from django.conf import settings
from django.db import models
from django.urls import reverse


class Categoria(models.Model):
    nombre = models.CharField(max_length=50, unique=True)
    descripcion = models.TextField(blank=True)

    class Meta:
        ordering = ["nombre"]
        verbose_name_plural = "categorías"

    def __str__(self):
        return self.nombre


class Incidencia(models.Model):
    class Prioridad(models.TextChoices):
        BAJA = "baja", "Baja"
        MEDIA = "media", "Media"
        ALTA = "alta", "Alta"

    class Estado(models.TextChoices):
        ABIERTA = "abierta", "Abierta"
        EN_CURSO = "en_curso", "En curso"
        RESUELTA = "resuelta", "Resuelta"

    titulo = models.CharField(max_length=120)
    descripcion = models.TextField()
    prioridad = models.CharField(
        max_length=10, choices=Prioridad.choices, default=Prioridad.MEDIA
    )
    estado = models.CharField(
        max_length=10, choices=Estado.choices, default=Estado.ABIERTA
    )
    categoria = models.ForeignKey(
        Categoria, on_delete=models.PROTECT, related_name="incidencias"
    )
    autor = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="incidencias",
    )
    creada = models.DateTimeField(auto_now_add=True)
    actualizada = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ["-creada"]

    def __str__(self):
        return self.titulo
```

```
    def get_absolute_url(self):
        return reverse("incidencias:detalle", args=[self.pk])
```

- `__str__` devuelve el texto con el que se muestra cada objeto: en el panel de administración, en las listas desplegables de los formularios y en la consola.
- `get_absolute_url` devuelve la dirección del detalle de cada incidencia. Es un convenio de Django: las vistas genéricas de creación lo usan para saber adónde redirigir después de guardar (apartado 6), y el panel de administración muestra un botón «Ver en el sitio».
- Las enumeraciones `Prioridad` y `Estado` están **dentro** de la clase `Incidencia`: se accede a ellas como `Incidencia.Estado.RESUELTA`.

5.3 Migraciones

Una **migración** es un fichero de Python que describe un cambio en la estructura de la base de datos: crear una tabla, añadir una columna, cambiar un tipo. Las migraciones se guardan en la carpeta `migrations` de cada aplicación, se numeran (`0001_initial.py`, `0002_...`) y forman un **historial** que se sube al repositorio junto con el código. Así cualquier miembro del equipo, o el servidor de producción, puede llevar su base de datos exactamente al mismo estado.

El ciclo de trabajo siempre es el mismo:

1. Se modifica `models.py`.
2. `python manage.py makemigrations`: Django compara los modelos con el estado que resulta de las migraciones existentes y **genera** un fichero de migración con las diferencias. No toca la base de datos.
3. `python manage.py migrate`: Django **aplica** a la base de datos las migraciones pendientes, en orden, y anota cada una en la tabla `django_migrations`.

Orden	Función
`makemigrations [app]`	Genera migraciones a partir de los cambios en los modelos
`migrate`	Aplica todas las migraciones pendientes de todas las aplicaciones
`migrate app número`	Lleva la aplicación hasta esa migración, **deshaciendo** las posteriores si están aplicadas
`migrate app zero`	Deshace todas las migraciones de la aplicación
`showmigrations [app]`	Lista las migraciones: `[X]` aplicada, `[ ]` pendiente
`sqlmigrate app número`	Muestra el SQL que ejecutaría una migración, sin ejecutarlo
`makemigrations --check`	Termina con error si hay cambios en los modelos sin migración (útil en integración continua, Unidad 4)

Ejercicio 5.1. Primer modelo y primera migración

Objetivo: crear un modelo, generar su migración, aplicarla y ver el SQL generado.

1. Escribid en `incidencias/models.py` solo las importaciones y el modelo `Categoria`.
2. Ejecutad:

```
> python manage.py makemigrations
> python manage.py migrate incidencias
> python manage.py sqlmigrate incidencias 0001
```

3. Abrid el fichero `incidencias/migrations/0001_initial.py` y buscad la operación `migrations.CreateModel`.

Verificación. `makemigrations` muestra:

```
Migrations for 'incidencias':
  incidencias\migrations\0001_initial.py
    + Create model Categoria
```

`migrate incidencias` termina con `Applying incidencias.0001_initial... OK`. `sqlmigrate` muestra, entre `BEGIN;` y `COMMIT;`, la sentencia:

```
CREATE TABLE "incidencias_categoria" ("id" integer NOT NULL PRIMARY KEY
AUTOINCREMENT, "nombre" varchar(50) NOT NULL UNIQUE, "descripcion" text NOT NULL);
```

La tabla se llama `incidencias_categoria` (aplicación y modelo en minúsculas), tiene la columna `id` que no se ha escrito, `nombre` es `UNIQUE` y `descripcion` es `NOT NULL` aunque tenga `blank=True`: `blank` solo afecta a la validación.

Ejercicio 5.2. Modelo con relaciones

Objetivo: añadir un modelo con claves ajenas y opciones cerradas, y comprobar el estado de las migraciones.

1. Añadid el modelo `Incidencia` completo.
2. Ejecutad `python manage.py makemigrations` y, **antes** de aplicar, `python manage.py showmigrations incidencias`.
3. Aplicad con `python manage.py migrate` y repetid `showmigrations incidencias`.
4. Ejecutad `python manage.py sqlmigrate incidencias 0002` y localizad las claves ajenas.
5. Ejecutad otra vez `python manage.py makemigrations`.

Verificación.

- El paso 2 genera `incidencias\migrations\0002_incidencia.py` con `+ Create model Incidencia`, y `showmigrations` muestra `[X] 0001_initial` y `[ ] 0002_incidencia`.
- Tras `migrate` (que muestra `Applying incidencias.0002_incidencia... OK`), las dos aparecen con `[X]`.
- El SQL contiene `"autor_id" integer NOT NULL REFERENCES "auth_user" ("id")` y `"categoria_id" bigint NOT NULL REFERENCES "incidencias_categoria" ("id")`, además de dos sentencias `CREATE INDEX`: Django crea un índice para cada clave ajena. Los campos con `choices` son `varchar(10)` normales: la lista de valores la controla Django, no la base de datos.
- El paso 5 responde `No changes detected`.

5.4 Modificar modelos existentes

Los modelos cambian durante la vida de un proyecto. Cada cambio genera una nueva migración que se aplica sobre la anterior, sin perder los datos existentes.

5.4.1 Campos nuevos en tablas con datos

Al añadir un campo a una tabla que ya existe, la base de datos necesita un valor para las filas que ya hay. Si el campo no admite valores vacíos y no tiene `default`, `makemigrations` lo pregunta:

```
It is impossible to add a non-nullable field 'codigo' to incidencia without
 ↪  specifying a
default. This is because the database needs something to populate existing rows.
Please select a fix:
 1) Provide a one-off default now (will be set on all existing rows with a null
  ↪  value
    for this column)
 2) Quit and manually define a default value in models.py.
Select an option:
```

La opción 1 pide un valor que se usará solo para las filas existentes; la opción 2 sale sin generar la migración, para que se añada `default=...` (o `blank=True`, en los textos) al campo. La opción 2 suele ser la más clara.

En SQLite, muchos cambios en una tabla se hacen creando una tabla nueva, copiando los datos y renombrándola, porque SQLite tiene un `ALTER TABLE` muy limitado. Django lo hace automáticamente, como se ve en el SQL del ejercicio siguiente.

5.4.2 Deshacer una migración

Para volver atrás, se ejecuta `migrate` indicando la última migración que se quiere conservar. Django ejecuta las operaciones inversas de las migraciones posteriores. Después se puede borrar el fichero de la migración deshecha, **siempre que no se haya compartido** con nadie: una migración que ya está en el repositorio o aplicada en producción no se borra ni se modifica; se crea otra migración que corrija la anterior.

Ejercicio 5.3. Añadir un campo y deshacer el cambio

Objetivo: generar una migración de modificación, aplicarla y deshacerla.

1. Añadid a `Incidencia`, después de `descripcion`, el campo `ubicacion = models.CharField(max_length=
   blank=True)`.
2. Ejecutad `makemigrations`, `sqlmigrate incidencias 0003` y `migrate`.
3. Deshaced la migración con `python manage.py migrate incidencias 0002` y comprobadlo con `showmigrations incidencias`.
4. Borrad el fichero `0003_incidencia_ubicacion.py` y el campo `ubicacion` de `models.py`, y ejecutad `makemigrations`.
5. Ahora añadid el campo `codigo = models.CharField(max_length=10)` (sin `blank` ni `default`), ejecutad `makemigrations`, elegid la opción `2` y borrad el campo.

Verificación.

- El paso 2 genera `0003_incidencia_ubicacion.py` con `+ Add field ubicacion to incidencia`. El SQL empieza por `CREATE TABLE "new__incidencias_incidencia"`, copia los datos con `INSERT INTO ... SELECT ..., '' FROM "incidencias_incidencia"` (la cadena vacía es el valor para las filas existentes), borra la tabla antigua y renombra la nueva.
- El paso 3 muestra `Unapplying incidencias.0003_incidencia_ubicacion... OK`, y `showmigrations` muestra `[ ] 0003_incidencia_ubicacion`.
- El paso 4 responde `No changes detected`: los modelos vuelven a coincidir con las migraciones 0001 y 0002.
- El paso 5 muestra el mensaje `It is impossible to add a non-nullable field 'codigo' to incidencia without specifying a default` y, al elegir `2`, termina sin crear ningún fichero.

5.5 Buenas prácticas con las migraciones

Práctica	Motivo
Ejecutar `makemigrations` tras cada cambio en los modelos y revisar el fichero generado	La migración es código: hay que entender qué hace
Subir las migraciones al repositorio junto con el cambio del modelo	El resto del equipo y el servidor necesitan aplicarlas
No editar ni borrar migraciones ya compartidas o aplicadas en producción	Las bases de datos de otros quedarían en un estado distinto del historial
No subir `db.sqlite3`	Cada desarrollador reconstruye su base de datos con `migrate`
Dar nombre a las migraciones importantes con `makemigrations --name descripcion`	El historial se lee mejor

Práctica	Motivo
Para modificar **datos** (no estructura), usar migraciones de datos con `migrations.RunPython`	Los cambios de datos quedan también en el historial

Si en desarrollo la base de datos queda en un estado confuso, se puede empezar de cero: borrar `db.sqlite3` y ejecutar `migrate`. Se pierden los datos, pero no la estructura, que está en las migraciones.

Ejercicio 5.4. Historial de migraciones

Objetivo: consultar el historial completo de migraciones del proyecto.

1. Ejecutad `python manage.py showmigrations` (sin indicar aplicación).
2. Ejecutad `python manage.py sqlmigrate auth 0001` y buscad la tabla de usuarios.
3. Con el servidor de desarrollo parado, borrad `db.sqlite3`, ejecutad `python manage.py migrate` y repetid el paso 1.

Verificación. El paso 1 lista, por orden alfabético, las aplicaciones `admin`, `auth`, `contenttypes`, `incidencias` y `sessions`, con todas sus migraciones marcadas con `[X]`; `incidencias` tiene `0001_initial` y `0002_incidencia`. El SQL del paso 2 contiene `CREATE TABLE "auth_user"` con las columnas `password`, `username`, `is_staff`, `is_superuser` e `is_active`, entre otras. Tras el paso 3, la base de datos se ha reconstruido y el resultado de `showmigrations` es idéntico (aunque vacía de datos).

> ***Para reflexionar.*** *El ORM genera el SQL por vosotros. ¿Sigue siendo necesario saber SQL para trabajar con Django? ¿En qué situaciones os haría falta?*

Capítulo 6

Consultas, administración y formularios

Para empezar

1. ¿Cómo obtendríais en SQL las incidencias abiertas de la categoría «Hardware», ordenadas de la más reciente a la más antigua?
2. Si una página muestra 100 incidencias con el nombre de su categoría, ¿cuántas consultas a la base de datos creéis que hacen falta? ¿Cuántas debería hacer?
3. ¿Qué comprobaciones haríais sobre los datos de un formulario de alta de incidencias antes de guardarlos?

6.1 La consola de Django

`python manage.py shell` abre el intérprete interactivo de Python con el proyecto cargado: los ajustes, las aplicaciones y la conexión a la base de datos. Es la herramienta para probar consultas antes de escribirlas en una vista. Desde Django 5.2, la consola **importa automáticamente** los modelos de todas las aplicaciones instaladas, de modo que `Categoria`, `Incidencia` y `User` están disponibles sin escribir ningún `import`:

```
> python manage.py shell
8 objects imported automatically (use -v 2 for details).

>>>
```

6.1.1 El gestor `objects`

Cada modelo tiene un **gestor** (*manager*), el atributo de clase `objects`, que es la puerta de entrada a la base de datos. Sus métodos devuelven objetos o **QuerySets**:

Operación	Código	SQL equivalente
Crear y guardar en un paso	`Categoria.objects.create(nombre="Red")`	`INSERT`
Crear y guardar en dos pasos	`c = Categoria(nombre="Red")` y `c.save()`	`INSERT`
Modificar	`c.descripcion = "Wifi y cables"` y `c.save()`	`UPDATE`
Borrar	`c.delete()`	`DELETE`
Obtener o crear	`c, creada = Categoria.objects.get_or_create(nombre="Red")`	`SELECT` y, si no existe, `INSERT`

Hasta que se llama a `save()`, un objeto creado con `Incidencia(...)` solo existe en memoria y su `pk` vale `None`. Los usuarios se crean con `User.objects.create_user(nombre, password=...)`, que guarda la contraseña cifrada (apartado 8); nunca se asigna la contraseña directamente al atributo `password`.

Ejercicio 6.1. Primeros objetos desde la consola

Objetivo: crear, relacionar y guardar objetos con el ORM, y comprobar las restricciones del modelo.

1. Abrid la consola con `python manage.py shell` y ejecutad, una a una:

```
>>> ana = User.objects.create_user("ana", password="Clave.2026")
>>> hw = Categoria.objects.create(nombre="Hardware", descripcion="Equipos,
 ↪  periféricos y proyectores")
>>> i = Incidencia(titulo="Proyector del aula 12 no enciende", descripcion="No
 ↪  enciende ni con el mando ni con el botón.", prioridad="alta", categoria=hw,
 ↪  autor=ana)
>>> print(i.pk)
>>> i.save()
>>> i.pk
>>> i.estado == "abierta"
>>> i.get_prioridad_display()
>>> hw.incidencias.all()
>>> Incidencia.objects.count()
```

2. Intentad crear otra categoría con el mismo nombre: `Categoria.objects.create(nombre="Hardware")`.
3. Intentad borrar la categoría: `hw.delete()`.
4. Salid con `exit()`.

Verificación. `print(i.pk)` muestra `None`; después de `save()`, `i.pk` vale `1`. `i.estado == "abierta"` da `True` (es el valor por defecto) y `get_prioridad_display()` devuelve `'Alta'`.

`hw.incidencias.all()` devuelve `<QuerySet [<Incidencia: Proyector del aula 12 no enciende>]>`, usando el `related_name` y el `__str__` del modelo, y `count()` devuelve `1`. El paso 2 lanza `IntegrityError: UNIQUE constraint failed: incidencias_categoria.nombre` y el paso 3, `ProtectedError: ("Cannot delete some instances of model 'Categoria' because they are referenced through protected foreign keys: 'Incidencia.categoria'.", ...)`.

6.1.2 Órdenes de administración propias

Para que todos trabajen con los mismos datos, se puede programar una **orden de administración propia**. Django busca las órdenes en la carpeta `management/commands` de cada aplicación instalada: cada fichero es una orden con el mismo nombre, que define una clase `Command` heredera de `BaseCommand` con un método `handle()`.

```
incidencias/
    management/
        __init__.py
        commands/
            __init__.py
            cargar_datos.py
```

```python
# incidencias/management/commands/cargar_datos.py
from django.contrib.auth.models import User
from django.core.management.base import BaseCommand

from incidencias.models import Categoria, Incidencia

CATEGORIAS = {
    "Hardware": "Equipos, periféricos y proyectores",
    "Software": "Programas, licencias e instalaciones",
    "Red": "Wifi, cableado y acceso a Internet",
}

INCIDENCIAS = [
    ("Proyector del aula 12 no enciende", "Hardware", "alta", "abierta", "ana"),
    ("Ratón roto en el puesto 7", "Hardware", "baja", "resuelta", "luis"),
    ("No funciona la wifi en la biblioteca", "Red", "alta", "en_curso", "ana"),
    ("Instalar Python 3.13 en el aula 15", "Software", "media", "abierta", "luis"),
    ("Licencia del antivirus caducada", "Software", "alta", "resuelta", "ana"),
    ("Impresora de la sala de profesores atascada", "Hardware", "media", "abierta",
 ↪  "luis"),
]


class Command(BaseCommand):
    help = "Borra las incidencias y carga datos de prueba"

    def handle(self, *args, **options):
        usuarios = {}
        for nombre in ["ana", "luis"]:
            usuario, creado = User.objects.get_or_create(username=nombre)
```

```
            if creado:
                usuario.set_password("Clave.2026")
                usuario.save()
            usuarios[nombre] = usuario

        categorias = {}
        for nombre, descripcion in CATEGORIAS.items():
            categorias[nombre], _ = Categoria.objects.get_or_create(
                nombre=nombre, defaults={"descripcion": descripcion}
            )

        Incidencia.objects.all().delete()
        for titulo, categoria, prioridad, estado, autor in INCIDENCIAS:
            Incidencia.objects.create(
                titulo=titulo,
                descripcion=f"Descripción de: {titulo}.",
                categoria=categorias[categoria],
                prioridad=prioridad,
                estado=estado,
                autor=usuarios[autor],
            )
        self.stdout.write(self.style.SUCCESS(f"Cargadas {Incidencia.objects.count()}
        ↪ incidencias"))
```

- `get_or_create` devuelve una tupla (objeto, creado): si el objeto ya existe, lo devuelve sin crearlo. Así la orden se puede ejecutar varias veces sin duplicar usuarios ni categorías.
- `defaults` indica los valores que se usan **solo** si hay que crearlo.
- `set_password()` cifra la contraseña antes de guardarla.
- `self.stdout.write(self.style.SUCCESS(...))` escribe en la terminal un mensaje en verde.
- La variable `_` es un convenio de Python para un valor que no se va a usar.

Ejercicio 6.2. Orden `cargar_datos`

Objetivo: crear una orden de administración propia y cargar con ella los datos de prueba.

1. Cread la estructura de carpetas `management/commands` con sus dos `__init__.py` vacíos y el fichero `cargar_datos.py`.
2. Ejecutad `python manage.py help` y buscad la sección `[incidencias]`.
3. Ejecutad dos veces `python manage.py cargar_datos`.

Verificación. En el paso 2 aparece la sección `[incidencias]` con la orden `cargar_datos`, y `python manage.py help cargar_datos` muestra el texto `Borra las incidencias y carga datos de prueba`. Cada ejecución del paso 3 muestra `Cargadas 6 incidencias`: la segunda vez no se duplica nada. Los identificadores (`id`) de las incidencias cambian en cada ejecución, porque se borran y se crean de nuevo; por eso los ejercicios siguientes se refieren a ellas por su título.

6.2 Consultas con QuerySets

Un **QuerySet** es una consulta a la base de datos, construida con métodos encadenables. Representa «un conjunto de objetos de un modelo que cumplen unas condiciones».

6.2.1 Los métodos básicos

Método	Devuelve	Equivalente SQL
`all()`	QuerySet con todos los objetos	`SELECT * FROM ...`
`filter(condiciones)`	QuerySet con los que **cumplen** las condiciones	`WHERE ...`
`exclude(condiciones)`	QuerySet con los que **no** las cumplen	`WHERE NOT ...`
`get(condiciones)`	**Un** objeto; error si hay cero o más de uno	`WHERE ...` (un resultado)
`order_by("campo", "-otro")`	QuerySet ordenado (`-` descendente)	`ORDER BY ...`
`[:5]`, `[5:10]`	QuerySet limitado	`LIMIT ... OFFSET ...`
`count()`	Número de resultados	`SELECT COUNT(*)`
`exists()`	`True` si hay algún resultado	`SELECT ... LIMIT 1`
`first()`, `last()`	El primer o el último objeto, o `None`	`ORDER BY ... LIMIT 1`
`values("campo", ...)`	QuerySet de diccionarios	`SELECT campo, ...`
`values_list("campo", flat=True)`	QuerySet de valores sueltos	`SELECT campo`
`distinct()`	Sin repetidos	`SELECT DISTINCT`
`update(campo=valor)`	Número de filas modificadas	`UPDATE ... WHERE ...`
`delete()`	Número de objetos borrados	`DELETE ... WHERE ...`

Los métodos que devuelven un QuerySet se pueden **encadenar**: `Incidencia.objects.filter(prioridad="al`
Varias condiciones dentro del mismo `filter()`, separadas por comas, se combinan con `AND`.

`get()` lanza `Incidencia.DoesNotExist` si no hay ningún resultado y `Incidencia.MultipleObjectsReturned` si hay más de uno. Se usa cuando se espera exactamente un objeto, normalmente al buscar por clave primaria o por un campo único.

6.2.2 Evaluación perezosa

Los QuerySets son **perezosos** (*lazy*): construirlos no consulta la base de datos. La consulta se ejecuta solo cuando se necesitan los datos: al recorrerlos con `for`, al convertirlos con `list()`, al pedir `len()`, al mostrarlos en la consola o al usarlos en una plantilla. Por eso se pueden construir por partes, como en las vistas de este apartado, sin coste:

```
consulta = Incidencia.objects.all()          # todavía no se ha consultado nada
if estado:
    consulta = consulta.filter(estado=estado)  # tampoco
for incidencia in consulta:                  # aquí se ejecuta una única consulta
    ...
```

El atributo `query` muestra el SQL que generará un QuerySet: `print(Incidencia.objects.filter(estado="ab`

6.2.3 Búsquedas por campo (*lookups*)

Las condiciones de `filter()`, `exclude()` y `get()` se escriben como argumentos por nombre. Por defecto comparan por igualdad; para otras comparaciones se añade al nombre del campo un **sufijo de búsqueda** separado por **dos guiones bajos**:

Búsqueda	Significado	SQL aproximado
`titulo="Red"` o `titulo__exact="Red"`	Igual a	`= 'Red'`
`titulo__iexact="red"`	Igual, sin distinguir mayúsculas	`LIKE 'red'`
`titulo__contains="aula"`	Contiene	`LIKE '%aula%'`
`titulo__icontains="AULA"`	Contiene, sin distinguir mayúsculas	`LIKE '%aula%'`
`titulo__startswith="Ratón"`	Empieza por	`LIKE 'Ratón%'`
`prioridad__in=["alta", "media"]`	Está en la lista	`IN ('alta', 'media')`
`creada__gte=fecha`, `__gt`, `__lte`, `__lt`	Mayor o igual, mayor, menor o igual, menor	`>=`, `>`, `<=`, `<`
`creada__date=fecha`, `creada__year=2026`	Parte de una fecha	
`descripcion__isnull=True`	Es nulo	`IS NULL`

6.2.4 Consultas a través de relaciones

Los dos guiones bajos también sirven para **recorrer relaciones**: `categoria__nombre="Hardware"` significa «cuya categoría tiene el nombre Hardware». Django genera el `JOIN` necesario. Se pueden encadenar varias relaciones y terminar con un sufijo de búsqueda: `autor__username__startswith="a"`.

Las relaciones también se recorren en sentido inverso, con el `related_name`: `Categoria.objects.filter(incid` devuelve las categorías que tienen alguna incidencia de prioridad alta.

Ejercicio 6.3. Consultas básicas

Objetivo: escribir consultas con filtros, exclusiones, búsquedas, relaciones y ordenación.

Con los datos de la orden `cargar_datos`, abrid la consola y escribid una consulta para cada apartado. Comprobad el resultado antes de pasar al siguiente.

1. Número total de incidencias.
2. Número de incidencias abiertas.
3. Títulos (con `values_list(..., flat=True)`) de las incidencias de prioridad alta que no están resueltas.
4. Incidencias de la categoría «Hardware» que siguen abiertas, contadas.
5. Número de incidencias cuyo título contiene «aula», sin distinguir mayúsculas.
6. Número de incidencias de la usuaria `ana`.
7. Las dos primeras incidencias por orden alfabético del título.
8. La incidencia cuyo título empieza por «Ratón», obtenida con `get()`, y el nombre de su categoría, su autor y la etiqueta de su estado.
9. Un `get()` con `estado="abierta"`. ¿Qué ocurre?
10. El SQL de la consulta del apartado 2.

Verificación.

1. `Incidencia.objects.count()` da `6`.
2. `Incidencia.objects.filter(estado="abierta").count()` da `3`.
3. `Incidencia.objects.filter(prioridad="alta").exclude(estado="resuelta").values_list("tit` `flat=True)` devuelve `<QuerySet ['No funciona la wifi en la biblioteca', 'Proyector del aula 12 no enciende']>` (en orden de creación descendente, por el `ordering` del modelo).
4. `Incidencia.objects.filter(categoria__nombre="Hardware", estado="abierta").count()` da `2`.
5. `filter(titulo__icontains="aula")` da `2`: el proyector y la instalación de Python.
6. `filter(autor__username="ana").count()` da `3`.
7. `order_by("titulo")[:2]` devuelve `<QuerySet [<Incidencia: Impresora de la sala de profesores atascada>, <Incidencia: Instalar Python 3.13 en el aula 15>]>`.
8. `i = Incidencia.objects.get(titulo__startswith="Ratón");` `i.categoria`, `i.autor` e `i.get_estado_display()` muestran `Hardware`, `luis` y `'Resuelta'`.
9. Se lanza `MultipleObjectsReturned: get() returned more than one Incidencia -- it returned 3!`.
10. `print(...query)` muestra un `SELECT` de todas las columnas de `incidencias_incidencia` con `WHERE "incidencias_incidencia"."estado" = abierta ORDER BY "incidencias_incidencia".` `DESC`. (El SQL real envía `abierta` como parámetro; `query` lo muestra sin comillas solo como referencia.)

6.2.5 Objetos Q y F

Para combinar condiciones con `OR` o con `NOT` se usan objetos `Q`, que se unen con los operadores `|` (o), `&` (y) y `~` (no):

```
from django.db.models import Q

Incidencia.objects.filter(Q(prioridad="alta") | Q(estado="en_curso"))
Incidencia.objects.filter(~Q(categoria__nombre="Red") & Q(estado="abierta"))
```

Un objeto `F` representa **el valor de otro campo de la misma fila**. Permite comparar dos columnas entre sí o actualizar un campo a partir de su valor actual sin traer los datos a Python:

```
from django.db.models import F

Incidencia.objects.filter(actualizada__gt=F("creada"))   # modificadas después de
↪ crearse
```

6.2.6 Agregaciones y anotaciones

Una **agregación** calcula un valor sobre un conjunto de filas (`COUNT`, `SUM`, `AVG`, `MAX`, `MIN`):

- `aggregate()` devuelve **un diccionario** con el resultado sobre todo el QuerySet.
- `annotate()` añade un valor calculado **a cada objeto** del QuerySet; con una relación inversa, equivale a un `GROUP BY`.

```
from django.db.models import Count

Incidencia.objects.aggregate(total=Count("id"))
# {'total': 6}

Categoria.objects.annotate(total=Count("incidencias")).order_by("-total")
# cada categoría lleva un atributo .total con su número de incidencias

Incidencia.objects.values("estado").annotate(n=Count("id")).order_by("estado")
# número de incidencias por estado
```

`Count` admite un argumento `filter` para contar solo algunas filas: `Count("incidencias", filter=Q(incidencias__estado="abierta"))`.

6.2.7 El problema de las N+1 consultas

Al acceder a una clave ajena de un objeto (`incidencia.categoria`), Django hace **una consulta adicional** para traer la categoría, salvo que ya la tenga. Si se recorren 100 incidencias mostrando su categoría, se hacen 1 + 100 consultas: el llamado **problema de las N+1 consultas**, la causa más frecuente de lentitud en las aplicaciones con ORM.

`select_related("categoria")` resuelve el problema: trae las incidencias **y** sus categorías en **una sola consulta** con `JOIN`. Se usa con `ForeignKey` y `OneToOneField`. Para relaciones inversas o de muchos a muchos existe `prefetch_related()`, que hace una segunda consulta para todos los objetos relacionados a la vez.

Con `DEBUG = True`, Django guarda las consultas ejecutadas en `django.db.connection.queries`, lo que permite contarlas desde la consola.

Ejercicio 6.4. Consultas avanzadas y número de consultas

Objetivo: usar `Q`, `F`, agregaciones y `select_related`, y medir el número de consultas.

1. En la consola, importad `Q`, `F` y `Count` de `django.db.models` y obtened:
    1. El número de incidencias que son de prioridad alta **o** están en curso.
    2. Una lista de parejas (nombre de categoría, número de incidencias), ordenada de más a menos incidencias y, a igualdad, por nombre.
    3. El número de incidencias de cada estado, con `values()` y `annotate()`.
    4. Las categorías que tienen alguna incidencia de prioridad alta, sin repetidos.
2. Medid las consultas de un recorrido sin y con `select_related`:

```
>>> from django.db import connection, reset_queries
>>> reset_queries()
>>> for i in Incidencia.objects.all(): print(i.titulo, "-", i.categoria.nombre)
>>> len(connection.queries)
>>> reset_queries()
>>> for i in Incidencia.objects.select_related("categoria"): print(i.titulo,
 ↪  "-", i.categoria.nombre)
>>> len(connection.queries)
```

3. Marcad como resueltas todas las incidencias abiertas de la categoría «Software» con una sola llamada a `update()` y volved a ejecutar `python manage.py cargar_datos` al terminar.

Verificación.

- 1.1: `Incidencia.objects.filter(Q(prioridad="alta") | Q(estado="en_curso")).count()` da `3` (la incidencia en curso también es de prioridad alta).
- 1.2: `list(Categoria.objects.annotate(total=Count("incidencias")).order_by("-total", "nombre").values_list("nombre", "total"))` da `[('Hardware', 3), ('Software', 2), ('Red', 1)]`.
- 1.3: `[{'estado': 'abierta', 'n': 3}, {'estado': 'en_curso', 'n': 1}, {'estado': 'resuelta', 'n': 2}]`.
- 1.4: `Categoria.objects.filter(incidencias__prioridad="alta").distinct()` devuelve `<QuerySet [<Categoria: Hardware>, <Categoria: Red>, <Categoria: Software>]>`.
- Paso 2: el primer recorrido hace `7` consultas (una para las incidencias y una por cada categoría); el segundo, `1`.
- Paso 3: `Incidencia.objects.filter(categoria__nombre="Software", estado="abierta").update(` devuelve `1` (el número de filas modificadas).

6.3 El panel de administración

La aplicación `django.contrib.admin` genera automáticamente una **interfaz de administración** completa a partir de los modelos: listados, buscador, filtros, formularios de alta y modificación,

borrado con confirmación e historial de cambios. Está pensada para el personal que gestiona los datos (en «Incidencias», el equipo de mantenimiento), no para el público.

Para acceder hace falta un usuario con `is_staff = True`. El primero se crea con `createsuperuser`, que pide nombre de usuario, correo (opcional) y contraseña dos veces. Si la contraseña es demasiado corta, común o parecida al nombre, la orden avisa y pregunta si se quiere usar de todos modos; en desarrollo se puede aceptar, pero no en producción.

6.3.1 Registrar los modelos

Un modelo aparece en el panel cuando se **registra** en el `admin.py` de su aplicación. El registro con una clase `ModelAdmin` permite configurar cómo se muestra:

```
# incidencias/admin.py
from django.contrib import admin

from .models import Categoria, Incidencia


@admin.register(Categoria)
class CategoriaAdmin(admin.ModelAdmin):
    list_display = ["nombre", "descripcion"]
    search_fields = ["nombre"]


@admin.register(Incidencia)
class IncidenciaAdmin(admin.ModelAdmin):
    list_display = ["titulo", "categoria", "prioridad", "estado", "autor", "creada"]
    list_filter = ["estado", "prioridad", "categoria"]
    search_fields = ["titulo", "descripcion"]
    list_select_related = ["categoria", "autor"]
```

`@admin.register(Modelo)` es un **decorador**: se escribe encima de la clase y equivale a llamar a `admin.site.register(Modelo, ClaseAdmin)` después de definirla.

Atributo de `ModelAdmin`	Efecto
`list_display`	Columnas del listado
`list_filter`	Filtros del lateral derecho
`search_fields`	Campos en los que busca la caja de búsqueda (con `icontains`)
`list_select_related`	Aplica `select_related` al listado (evita las N+1 consultas)

Atributo de `ModelAdmin`	Efecto
`ordering`	Orden del listado, si no se quiere el del modelo
`readonly_fields`	Campos que se muestran pero no se pueden editar
`list_editable`	Campos editables directamente en el listado
`list_per_page`	Elementos por página (100 por defecto)

Cada filtro y cada búsqueda del panel es un QuerySet: el panel es también una forma de practicar consultas sin escribirlas.

Ejercicio 6.5. Panel de administración

Objetivo: configurar el panel de administración de los dos modelos y gestionar datos con él.

1. Cread un superusuario con `python manage.py createsuperuser` (nombre `admin` y una contraseña que recordéis).
2. Escribid `incidencias/admin.py` como en el ejemplo, arrancad el servidor y entrad en http://127.0.0.1:8000/admin/.
3. Entrad en «Incidencias», usad el filtro «Por estado» para ver las abiertas y buscad «wifi».
4. Cread una categoría «Mobiliario» y una incidencia en ella desde el panel.
5. Intentad borrar desde el panel la categoría «Red».
6. Abrid una incidencia y observad que las fechas no aparecen: los campos con `auto_now` y `auto_now_add` no son editables y el panel los oculta. Añadid `readonly_fields = ["creada", "actualizada"]` a `IncidenciaAdmin` y volved a abrirla.

Verificación.

- La página principal del panel muestra la sección **«Incidencias»** con «Categorías» e «Incidencias» (gracias a `verbose_name_plural`), y la sección «Autenticación y autorización» con «Grupos» y «Usuarios».
- La pestaña del listado muestra «Seleccione incidencia a modificar | Sitio de administración de Django». El filtro «Por estado: Abierta» deja 3 incidencias y la búsqueda «wifi», 1.
- La incidencia creada en el paso 4 aparece la primera del listado (es la más reciente) y su página tiene el botón «Ver en el sitio», que lleva a su detalle gracias a `get_absolute_url`.
- El paso 5 no borra nada: el panel muestra que no se puede eliminar la categoría porque lo impiden objetos protegidos relacionados, y enumera la incidencia de «Red».
- En el paso 6, el formulario de una incidencia muestra las fechas de creación y actualización como texto, no como campos.

6.4 Vistas que consultan la base de datos

Con los modelos y las consultas ya se pueden sustituir los datos en memoria del apartado 3 por la base de datos.

6.4.1 Vistas de función con el ORM

El atajo `get_object_or_404(Modelo, condiciones)` hace un `get()` y, si el objeto no existe, lanza `Http404` en lugar de `DoesNotExist`, que provocaría un error 500:

```
# incidencias/views.py (versión con vistas de función)
from django.shortcuts import get_object_or_404, render

from .models import Incidencia


def lista(request):
    incidencias = Incidencia.objects.select_related("categoria")
    estado = request.GET.get("estado")
    if estado:
        incidencias = incidencias.filter(estado=estado)
    return render(request, "incidencias/lista.html", {"incidencias": incidencias,
    ↪  "estado": estado})


def detalle(request, pk):
    incidencia = get_object_or_404(Incidencia, pk=pk)
    return render(request, "incidencias/detalle.html", {"incidencia": incidencia})
```

6.4.2 Vistas genéricas de listado y detalle

Las vistas genéricas de Django resuelven estas mismas tareas con atributos de clase. Se redefinen solo los métodos cuyo comportamiento se quiere cambiar:

Elemento	`ListView`	`DetailView`
`model`	Modelo que se lista	Modelo que se muestra
`template_name`	Por defecto, `incidencias/incidencia_list.html`	Por defecto, `incidencias/incidencia_detail.html`
`context_object_name`	Nombre de la lista en la plantilla (por defecto, `object_list` e `incidencia_list`)	Nombre del objeto (por defecto, `object` e `incidencia`)
`get_queryset()`	Consulta que se lista (por defecto, `Modelo.objects.all()`)	Consulta en la que se busca el objeto
Parámetro de la ruta	No necesita	`pk` (o `slug`); responde 404 si no existe

Elemento	`ListView`	`DetailView`
`paginate_by`	Elementos por página; añade `page_obj` al contexto	No aplica
`get_context_data()`	Añade variables al contexto	Añade variables al contexto

Esta es la versión definitiva de las vistas de «Incidencias». Sustituye **por completo** el contenido de `views.py` del apartado 3 (incluidas la lista `INCIDENCIAS` y las vistas de prueba `detalle_json`, `ultima` y `EstadoView`):

```
# incidencias/views.py
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import CreateView, DetailView, ListView, TemplateView

from .forms import IncidenciaForm
from .models import Incidencia


class IncidenciaListView(ListView):
    model = Incidencia
    template_name = "incidencias/lista.html"
    context_object_name = "incidencias"
    paginate_by = 5

    def get_queryset(self):
        consulta = Incidencia.objects.select_related("categoria")
        estado = self.request.GET.get("estado")
        if estado:
            consulta = consulta.filter(estado=estado)
        return consulta

    def get_context_data(self, **kwargs):
        contexto = super().get_context_data(**kwargs)
        contexto["estado"] = self.request.GET.get("estado")
        return contexto


class IncidenciaDetailView(DetailView):
    model = Incidencia
    template_name = "incidencias/detalle.html"
    context_object_name = "incidencia"


class IncidenciaCreateView(LoginRequiredMixin, CreateView):
    model = Incidencia
    form_class = IncidenciaForm
    template_name = "incidencias/formulario.html"

    def form_valid(self, form):
        form.instance.autor = self.request.user
        return super().form_valid(form)
```

```
class AcercaView(TemplateView):
    template_name = "incidencias/acerca.html"
```

```
# incidencias/urls.py
from django.urls import path

from . import views

app_name = "incidencias"

urlpatterns = [
    path("", views.IncidenciaListView.as_view(), name="lista"),
    path("incidencias/<int:pk>/", views.IncidenciaDetailView.as_view(),
    ↪ name="detalle"),
    path("incidencias/nueva/", views.IncidenciaCreateView.as_view(), name="nueva"),
    path("acerca/", views.AcercaView.as_view(), name="acerca"),
]
```

`IncidenciaCreateView` y el formulario `IncidenciaForm` se explican en el apartado 6.5. Hasta escribir `forms.py`, se pueden dejar comentadas la vista, su importación y su ruta.

`get_context_data()` sigue el patrón habitual al redefinir un método de una clase del framework: se llama al método del padre con `super()`, se añade lo necesario al resultado y se devuelve.

6.4.3 Plantillas con objetos del modelo

En las plantillas, los objetos del modelo se usan como los diccionarios del apartado 3, con algunas posibilidades nuevas: los métodos `get_estado_display` y `get_absolute_url` (sin paréntesis), las relaciones (`incidencia.categoria`, `incidencia.autor.username`) y el objeto de paginación `page_obj`:

```
{# incidencias/templates/incidencias/lista.html #}
{% extends "incidencias/base.html" %}

{% block titulo %}Incidencias{% endblock %}

{% block contenido %}
<h1>Incidencias{% if estado %} ({{ estado }}){% endif %}</h1>
<p>
  <a href="{% url 'incidencias:lista' %}?estado=abierta">Abiertas</a>
  | <a href="{% url 'incidencias:lista' %}?estado=en_curso">En curso</a>
  | <a href="{% url 'incidencias:lista' %}?estado=resuelta">Resueltas</a>
  {% if estado %}| <a href="{% url 'incidencias:lista' %}">Todas</a>{% endif %}
</p>
<p>{{ page_obj.paginator.count }} incidencia{{ page_obj.paginator.count|pluralize
↪  }}</p>
<ul>
  {% for incidencia in incidencias %}
    <li class="{{ incidencia.prioridad }}">
```

```
      <a href="{{ incidencia.get_absolute_url }}">{{ incidencia.titulo }}</a>
      ({{ incidencia.get_estado_display }}, {{ incidencia.categoria }})
      {{ incidencia.creada|date:"d/m/Y H:i" }}
    </li>
  {% empty %}
    <li>No hay incidencias.</li>
  {% endfor %}
</ul>
{% if page_obj.has_other_pages %}
  <p>
    {% if page_obj.has_previous %}
      <a href="?page={{ page_obj.previous_page_number }}{% if estado
      ↪ %}&amp;estado={{ estado }}{% endif %}">Anterior</a>
    {% endif %}
    Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }}
    {% if page_obj.has_next %}
      <a href="?page={{ page_obj.next_page_number }}{% if estado %}&amp;estado={{
      ↪ estado }}{% endif %}">Siguiente</a>
    {% endif %}
  </p>
{% endif %}
{% endblock %}
```

```
{# incidencias/templates/incidencias/detalle.html #}
{% extends "incidencias/base.html" %}

{% block titulo %}{{ incidencia.titulo }} | {{ block.super }}{% endblock %}

{% block contenido %}
<h1>{{ incidencia.titulo }}</h1>
<p>Prioridad: {{ incidencia.get_prioridad_display }} · Estado: {{
↪ incidencia.get_estado_display }}</p>
<p>Categoría: {{ incidencia.categoria }} · Autor: {{ incidencia.autor.username
↪ }}</p>
<p>Creada: {{ incidencia.creada|date:"d/m/Y H:i" }} · Actualizada: {{
↪ incidencia.actualizada|date:"d/m/Y H:i" }}</p>
<p>{{ incidencia.descripcion|linebreaksbr }}</p>
<p><a href="{% url 'incidencias:lista' %}">Volver al listado</a></p>
{% endblock %}
```

Las fechas se guardan en UTC (`USE_TZ = True`), pero el filtro `date` las muestra en la zona horaria de `TIME_ZONE`, `Europe/Madrid`.

Ejercicio 6.6. Listado y detalle desde la base de datos

Objetivo: sustituir los datos en memoria por consultas con vistas genéricas y paginación.

1. Escribid las vistas, las rutas y las dos plantillas definitivas (dejando comentado de momento lo relativo a `IncidenciaCreateView`).
2. Abrid `/`, `/?page=2`, `/?estado=abierta`, `/?page=9` y el detalle de cualquier incidencia. Probad también `/incidencias/999999/`.

Verificación.

- `/` muestra «6 incidencias», las 5 más recientes (la primera, «Impresora de la sala de profesores atascada») y el texto «Página 1 de 2» con un enlace «Siguiente».
- `/?page=2` muestra una sola incidencia, «Proyector del aula 12 no enciende», y un enlace «Anterior».
- `/?estado=abierta` muestra «3 incidencias» y no muestra paginación.
- `/?page=9` y `/incidencias/999999/` devuelven 404.
- El detalle muestra la prioridad y el estado con su etiqueta («Alta», «En curso»), la categoría, el autor y las fechas en hora peninsular.

6.5 Formularios

Un **formulario** de Django es una clase que describe los campos que se esperan, sabe **generar** su HTML, **valida** los datos recibidos y los **convierte** a tipos de Python. Hay dos clases base:

- `forms.Form`: formulario cuyos campos se declaran uno a uno (un buscador, un formulario de contacto).
- `forms.ModelForm`: formulario **generado a partir de un modelo**, que además sabe guardar el objeto. Es el más habitual.

```
# incidencias/forms.py
from django import forms

from .models import Incidencia


class IncidenciaForm(forms.ModelForm):
    class Meta:
        model = Incidencia
        fields = ["titulo", "descripcion", "prioridad", "categoria"]
        labels = {
            "titulo": "Título",
            "descripcion": "Descripción",
            "categoria": "Categoría",
        }

    def clean_titulo(self):
        titulo = self.cleaned_data["titulo"].strip()
        if len(titulo) < 5:
            raise forms.ValidationError("El título debe tener al menos 5
            ↪ caracteres.")
        return titulo
```

- `fields` indica qué campos del modelo forman parte del formulario. **Nunca** se incluyen `autor`, `estado`, `creada` ni `actualizada`: los decide la aplicación, no el usuario. Escribir `fields = "__all__"` es cómodo pero peligroso, porque cualquier campo nuevo del modelo pasaría a ser editable por el usuario.

- `labels` cambia las etiquetas que se muestran; por defecto se usa el nombre del campo («Titulo», sin tilde).
- Los campos con `choices` se convierten en listas desplegables, y la clave ajena `categoria`, en una lista con las categorías, mostradas con su `__str__`.

6.5.1 El proceso de validación

Al llamar a `form.is_valid()`, Django valida el formulario en este orden:

1. Cada campo convierte y valida su valor según su tipo y sus opciones (obligatorio, `max_length`, valor de `choices`, que la categoría exista…).
2. Para cada campo, si existe, se llama al método `clean_<campo>()`, que puede comprobar algo más y devolver el valor limpio.
3. Se llama al método `clean()` del formulario, para reglas que afectan a **varios campos**.
4. En un `ModelForm`, se valida también el modelo: se llama a su método `clean()` y se comprueban las restricciones `unique`.

Si algo falla, `is_valid()` devuelve `False` y los errores quedan en `form.errors`; si todo es correcto, los valores convertidos están en el diccionario `form.cleaned_data`.

6.5.2 Validación en el modelo

Una regla que afecta a varios campos y que debe cumplirse **siempre**, se cree el objeto desde donde se cree, se escribe en el método `clean()` del **modelo**. Así la aplican tanto los formularios de la aplicación como el panel de administración. En «Incidencias», una incidencia de prioridad alta debe explicar bien el problema:

Se importa `ValidationError` al principio de `models.py` (`from django.core.exceptions import ValidationError`) y se añade el método a la clase `Incidencia`, después de `__str__`:

```
# incidencias/models.py (método añadido a la clase Incidencia)
    def clean(self):
        if self.prioridad == self.Prioridad.ALTA and len(self.descripcion) < 20:
            raise ValidationError(
                {"descripcion": "Las incidencias de prioridad alta necesitan una
 ↪  descripción de al menos 20 caracteres."}
            )
```

El diccionario asocia el error a un campo concreto, para que el formulario lo muestre junto a él. El método `clean()` del modelo **no** se llama al ejecutar `save()` ni `objects.create()`: lo llaman los formularios (y el método `full_clean()` del modelo). Por eso la orden `cargar_datos` no lo activa, y por eso la API del apartado 7 tendrá que llamarlo expresamente.

6.5.3 Vista de creación

Con una vista de función, el patrón clásico de un formulario es este:

```python
# incidencias/views.py (versión con vista de función, como referencia)
from django.contrib.auth.decorators import login_required
from django.shortcuts import redirect, render

from .forms import IncidenciaForm


@login_required
def nueva(request):
    if request.method == "POST":
        form = IncidenciaForm(request.POST)
        if form.is_valid():
            incidencia = form.save(commit=False)
            incidencia.autor = request.user
            incidencia.save()
            return redirect(incidencia)
    else:
        form = IncidenciaForm()
    return render(request, "incidencias/formulario.html", {"form": form})
```

1. Con `GET`, se muestra el formulario vacío.
2. Con `POST`, se construye el formulario con los datos recibidos y se valida.
3. Si es válido, `save(commit=False)` crea el objeto **sin guardarlo**, se completa el autor con el usuario conectado y se guarda. `redirect(incidencia)` redirige a su `get_absolute_url()`. Redirigir tras un POST correcto evita que, al recargar la página, el navegador reenvíe el formulario (patrón *Post/Redirect/Get*).
4. Si no es válido, se vuelve a mostrar el formulario, ahora con los datos introducidos y los mensajes de error.

`IncidenciaCreateView` hace exactamente lo mismo con una vista genérica: `form_valid()` se ejecuta cuando el formulario es válido, y ahí se asigna el autor antes de dejar que la clase padre guarde y redirija. `LoginRequiredMixin` es la versión para vistas de clase del decorador `@login_required` (apartado 8): un *mixin* es una clase pensada para combinarse con otras mediante herencia múltiple, y debe escribirse **a la izquierda** de la vista genérica.

6.5.4 La plantilla del formulario

```
{# incidencias/templates/incidencias/formulario.html #}
{% extends "incidencias/base.html" %}

{% block titulo %}Nueva incidencia{% endblock %}

{% block contenido %}
<h1>Nueva incidencia</h1>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Guardar</button>
</form>
```

```
{% endblock %}
```

- `{{ form.as_p }}` genera cada campo con su etiqueta dentro de un párrafo, junto con sus errores. También existen `as_div` y `as_ul`, o se puede escribir cada campo a mano con `{{ form.titulo.label_tag }}`, `{{ form.titulo }}` y `{{ form.titulo.errors }}`.
- `{% csrf_token %}` inserta un campo oculto con el **token CSRF**. Sin él, `CsrfViewMiddleware` rechaza el envío con un 403 (apartados 4 y 8).

Ejercicio 6.7. Formulario de alta de incidencias

Objetivo: crear incidencias desde la aplicación con un formulario validado.

1. Escribid `forms.py`, el método `clean()` del modelo y la plantilla `formulario.html`, y quitad los comentarios de `IncidenciaCreateView` y de su ruta.
2. Añadid a `estilos.css` la regla `.errorlist { color: darkred; }` (la clase que Django pone a las listas de errores).
3. Sin haber iniciado sesión, abrid http://127.0.0.1:8000/incidencias/nueva/.
4. Iniciad sesión en http://127.0.0.1:8000/admin/ con el superusuario y volved a abrir la dirección anterior.
5. Enviad el formulario: primero vacío; después con el título `abc`; después con prioridad alta y la descripción `No va`; por último, con el título `Cable de red cortado`, la descripción `En el aula 4`, prioridad baja y categoría «Red».
6. Probad la misma incidencia de prioridad alta con descripción corta desde el panel de administración.

Verificación.

- En el paso 3, el navegador es redirigido a `/accounts/login/?next=/incidencias/nueva/` y muestra un 404: todavía no existe la página de inicio de sesión de la aplicación (se creará en el apartado 8).
- En el paso 4 se ve el formulario con los campos «Título», «Descripción», «Prioridad» (con «Media» seleccionada) y «Categoría» (con las opciones «———», «Hardware», «Red» y «Software»).
- Envío vacío: el navegador no deja enviarlo, porque Django añade el atributo `required` a los campos obligatorios. Título `abc`: «El título debe tener al menos 5 caracteres.». Prioridad alta y descripción corta: «Las incidencias de prioridad alta necesitan una descripción de al menos 20 caracteres.», junto al campo «Descripción». En los dos casos se conservan los datos escritos.
- El último envío redirige al detalle de la nueva incidencia, que muestra «Autor: admin» y «Estado: Abierta», y aparece la primera en el listado.
- En el paso 6, el panel de administración muestra el mismo error del método `clean()` del modelo.
- El código fuente del formulario contiene `<input type="hidden" name="csrfmiddlewaretoken" value="...">`.

Al terminar, ejecutad `python manage.py cargar_datos` para volver a los datos de prueba.

Para reflexionar. *La regla de la descripción mínima se ha escrito en el modelo y no en el formulario. ¿Qué ocurriría si estuviera solo en* `IncidenciaForm` *y mañana se añadiera otra forma de crear incidencias?*

Capítulo 7

API REST con Django REST Framework

Para empezar

1. Si la aplicación «Incidencias» tuviera que usarse también desde una aplicación móvil o desde un cliente escrito en Vue, ¿le serviría que el servidor le enviara páginas HTML? ¿Qué formato le enviaríais?
2. ¿Qué diferencia hay entre los métodos HTTP `GET`, `POST`, `PUT`, `PATCH` y `DELETE`? ¿Cuáles habéis usado alguna vez?
3. ¿Qué significan los códigos de estado 200, 201, 204, 400, 401, 403 y 404?

7.1 Arquitectura REST

Hasta ahora, el servidor generaba **páginas HTML** completas. En una aplicación moderna es frecuente separar el cliente del servidor: el **cliente** (una aplicación Vue en el navegador, una aplicación móvil, otro servidor) se ocupa de la interfaz, y el **servidor** solo ofrece **datos** a través de una **API** (*Application Programming Interface*). Es la arquitectura que se usará en la Unidad 3, con un cliente Vue que consumirá la API de «Incidencias».

REST (*Representational State Transfer*) es un estilo de diseño de APIs sobre HTTP con estos principios:

- La API expone **recursos** (incidencias, categorías), cada uno con su dirección (URI): `/api/incidencias/` es la colección y `/api/incidencias/7/` es un elemento.
- Las operaciones se expresan con los **métodos HTTP**, no con el nombre de la dirección: no se escribe `/api/borrarIncidencia?id=7`, sino `DELETE /api/incidencias/7/`.
- Los datos viajan en una **representación** estándar, normalmente **JSON**.
- El servidor **no guarda estado** de la conversación con el cliente (*stateless*): cada petición lleva todo lo necesario, incluida la identificación del usuario (apartado 8).
- El resultado se comunica con los **códigos de estado** HTTP.

Una API que sigue estos principios se llama **API RESTful**.

7.1.1 Operaciones sobre un recurso

Método	Dirección	Operación	Código si va bien
`GET`	`/api/incidencias/`	Listar la colección	200 OK
`POST`	`/api/incidencias/`	Crear un elemento	201 Created
`GET`	`/api/incidencias/7/`	Obtener un elemento	200 OK
`PUT`	`/api/incidencias/7/`	Sustituir un elemento (se envían **todos** los campos)	200 OK
`PATCH`	`/api/incidencias/7/`	Modificar **algunos** campos	200 OK
`DELETE`	`/api/incidencias/7/`	Borrar un elemento	204 No Content

`GET` es un método **seguro** (no modifica nada); `GET`, `PUT` y `DELETE` son **idempotentes** (repetir la misma petición deja el servidor igual que hacerla una vez); `POST` no lo es: cada envío crea un elemento nuevo.

7.1.2 Códigos de estado

Código	Significado	Cuándo lo devuelve la API
200 OK	Correcto	Lectura o modificación correcta
201 Created	Creado	Alta correcta
204 No Content	Correcto, sin cuerpo	Borrado correcto
400 Bad Request	Petición incorrecta	Datos que no superan la validación, JSON mal formado
401 Unauthorized	No autenticado	Faltan las credenciales o no son válidas
403 Forbidden	Prohibido	El usuario está identificado pero no tiene permiso
404 Not Found	No encontrado	El elemento no existe
405 Method Not Allowed	Método no permitido	Por ejemplo, `DELETE` sobre la colección
409 Conflict	Conflicto	La operación choca con el estado actual (borrar una categoría con incidencias)
500 Internal Server Error	Error del servidor	Un fallo no controlado en el código: nunca debería ocurrir

7.2 Django REST Framework

Se podría construir una API con lo visto hasta ahora: vistas que devuelven `JsonResponse`. Pero habría que programar a mano la conversión entre modelos y JSON, la validación de los datos recibidos, los códigos de estado, la autenticación, los permisos y la paginación. **Django REST Framework** (DRF) es una aplicación de Django que resuelve todo eso. Sus piezas principales son:

Pieza	Función	Equivalente en las vistas HTML
Serializador (*serializer*)	Convierte objetos en datos JSON y valida los datos recibidos	Formulario (`ModelForm`)
ViewSet	Agrupa en una clase las operaciones de un recurso (listar, crear, obtener, modificar, borrar)	Vistas genéricas
Router	Genera automáticamente las rutas de los ViewSets	`urls.py` escrito a mano
Permisos y **autenticación**	Quién puede hacer qué (apartado 8)	`LoginRequiredMixin`
API navegable	Interfaz HTML para probar la API desde el navegador	

7.2.1 Instalación

```
> python -m pip install "djangorestframework~=3.16.0"
> pip freeze > requirements.txt
```

```
# config/settings.py (fragmento)
INSTALLED_APPS = [
    # ... aplicaciones de django.contrib ...
    "rest_framework",
    "incidencias",
]
```

```
REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.BasicAuthentication",
        "rest_framework.authentication.SessionAuthentication",
    ],
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticatedOrReadOnly",
    ],
}
```

La configuración de DRF se reúne en el diccionario `REST_FRAMEWORK`:

- `DEFAULT_AUTHENTICATION_CLASSES`: cómo se identifica el usuario. De momento, con **autenticación básica** (usuario y contraseña en cada petición, cómoda para probar con `curl.exe`) y con la **sesión** de Django (para la API navegable). En el apartado 8 la autenticación básica se sustituirá por **tokens**.
- `DEFAULT_PERMISSION_CLASSES`: `IsAuthenticatedOrReadOnly` permite leer a cualquiera y exige estar autenticado para crear, modificar o borrar.

No se configura paginación global: los listados de «Incidencias» se devuelven como **listas JSON** (arrays), que es lo que esperará el cliente Vue de la Unidad 3. La paginación se estudia en el apartado 7.6 sobre un listado concreto.

7.3 Serializadores

Un **serializador** hace dos trabajos opuestos:

- **Serializar**: convertir un objeto del modelo (o un QuerySet) en tipos básicos de Python (diccionarios, listas, cadenas) que se pueden escribir como JSON.
- **Deserializar**: recibir datos (por ejemplo, el JSON de un POST), **validarlos** y convertirlos en un objeto que se puede guardar.

`ModelSerializer` genera los campos a partir del modelo, igual que `ModelForm`:

```
# incidencias/serializers.py
import copy

from django.core.exceptions import ValidationError as DjangoValidationError
from rest_framework import serializers

from .models import Categoria, Incidencia


class CategoriaSerializer(serializers.ModelSerializer):
    class Meta:
        model = Categoria
        fields = ["id", "nombre", "descripcion"]
```

```python
class IncidenciaSerializer(serializers.ModelSerializer):
    autor = serializers.ReadOnlyField(source="autor.username")
    categoria_nombre = serializers.ReadOnlyField(source="categoria.nombre")

    class Meta:
        model = Incidencia
        fields = [
            "id", "titulo", "descripcion", "prioridad", "estado",
            "categoria", "categoria_nombre", "autor", "creada", "actualizada",
        ]
        read_only_fields = ["creada", "actualizada"]

    def validate_titulo(self, valor):
        valor = valor.strip()
        if len(valor) < 5:
            raise serializers.ValidationError("El título debe tener al menos 5
            ↪ caracteres.")
        return valor

    def validate(self, datos):
        # Reutiliza la validación del modelo (Incidencia.clean), que DRF no ejecuta
        ↪ por sí mismo.
        incidencia = copy.copy(self.instance) if self.instance else Incidencia()
        for campo, valor in datos.items():
            setattr(incidencia, campo, valor)
        try:
            incidencia.clean()
        except DjangoValidationError as error:
            # Errores asociados a campos (diccionario) o generales (lista)
            mensajes = error.message_dict if hasattr(error, "error_dict") else
↪   error.messages
            raise serializers.ValidationError(mensajes)
        return datos
```

Elemento	Significado
`fields`	Campos que se incluyen en el JSON, en ese orden. Como en los formularios, conviene enumerarlos en lugar de usar `"__all__"`
`read_only_fields`	Campos que se muestran pero que el cliente no puede enviar
`ReadOnlyField(source="autor.username")`	Campo de solo lectura cuyo valor se obtiene recorriendo la relación: el JSON muestra el nombre del autor en lugar de su `id`
`categoria`	Al ser una clave ajena, se representa con el `id` de la categoría; para crear una incidencia, el cliente envía ese `id`
`validate_<campo>()`	Validación de un campo, como `clean_<campo>()` en los formularios

Elemento	Significado
`validate()`	Validación de varios campos a la vez, como `clean()`

El campo `autor` es de solo lectura: el cliente **no** puede elegir el autor; lo asignará la vista con el usuario autenticado.

7.3.1 Los serializadores no llaman a `clean()` del modelo

A diferencia de `ModelForm`, un `ModelSerializer` **no** ejecuta el método `clean()` del modelo. Si no se hace nada, la regla «las incidencias de prioridad alta necesitan una descripción de al menos 20 caracteres» se cumpliría en los formularios y en el panel de administración, pero la API la saltaría. El método `validate()` del serializador anterior lo evita: construye un objeto `Incidencia` con los datos recibidos (sobre una copia del objeto existente, si es una modificación) y llama a su `clean()`, convirtiendo el error de Django en un error de DRF. Así la regla de negocio sigue escrita en un único sitio, el modelo.

Ejercicio 7.1. Serializadores desde la consola

Objetivo: instalar DRF y comprobar cómo serializa y valida un serializador.

1. Instalad Django REST Framework, añadid `"rest_framework"` a `INSTALLED_APPS` y el diccionario `REST_FRAMEWORK`.
2. Escribid `incidencias/serializers.py`.
3. En la consola de Django:

```
>>> from incidencias.serializers import IncidenciaSerializer
>>> i = Incidencia.objects.get(titulo__startswith="Ratón")
>>> IncidenciaSerializer(i).data
>>> s = IncidenciaSerializer(data={"titulo": "abc"})
>>> s.is_valid()
>>> s.errors
>>> s = IncidenciaSerializer(data={"titulo": "Teclado sin teclas",
 ↪  "descripcion": "Faltan teclas", "categoria": 1, "prioridad": "urgente"})
>>> s.is_valid()
>>> s.errors
```

Verificación. `IncidenciaSerializer(i).data` devuelve un diccionario con las claves `id`, `titulo`, `descripcion`, `prioridad`, `estado`, `categoria`, `categoria_nombre`, `autor`, `creada` y `actualizada`; en él, `'categoria': 1`, `'categoria_nombre': 'Hardware'`, `'autor': 'luis'` y las fechas en formato ISO 8601 con la zona horaria (`'2026-11-11T09:20:15.123456+01:00'`). El primer `is_valid()` da `False` y `errors` contiene tres errores: `'titulo'` con «El título debe tener al menos 5 caracteres.» y `'descripcion'` y `'categoria'` con «Este campo es requerido.». El segundo da `False` con el error `'"urgente" no es una elección válida.'` en `prioridad`.

7.4 ViewSets y routers

Un **ViewSet** es una clase que reúne todas las operaciones de un recurso. `ModelViewSet` implementa las seis operaciones de la tabla del apartado 7.1 a partir de dos atributos: la consulta (`queryset` o el método `get_queryset()`) y el serializador.

```
# incidencias/api.py
from django.db.models import ProtectedError
from rest_framework import status, viewsets
from rest_framework.response import Response

from .models import Categoria, Incidencia
from .serializers import CategoriaSerializer, IncidenciaSerializer


class CategoriaViewSet(viewsets.ModelViewSet):
    queryset = Categoria.objects.all()
    serializer_class = CategoriaSerializer

    def destroy(self, request, *args, **kwargs):
        try:
            return super().destroy(request, *args, **kwargs)
        except ProtectedError:
            return Response(
                {"detail": "La categoría tiene incidencias y no se puede borrar."},
                status=status.HTTP_409_CONFLICT,
            )


class IncidenciaViewSet(viewsets.ModelViewSet):
    serializer_class = IncidenciaSerializer

    def get_queryset(self):
        consulta = Incidencia.objects.select_related("categoria", "autor")
        estado = self.request.query_params.get("estado")
        prioridad = self.request.query_params.get("prioridad")
        if estado:
            consulta = consulta.filter(estado=estado)
        if prioridad:
            consulta = consulta.filter(prioridad=prioridad)
        return consulta

    def perform_create(self, serializer):
        serializer.save(autor=self.request.user)
```

Método de `ModelViewSet`	Petición	Acción
`list()`	`GET` a la colección	Lista de elementos (paginada si se configura paginación)
`create()`	`POST` a la colección	Valida con el serializador, guarda y responde 201
`retrieve()`	`GET` a un elemento	Un elemento, o 404
`update()`	`PUT` a un elemento	Sustitución completa
`partial_update()`	`PATCH` a un elemento	Modificación parcial
`destroy()`	`DELETE` a un elemento	Borrado y 204

Se redefinen solo los métodos que necesitan un comportamiento propio:

- `get_queryset()` aplica **filtros** a partir de los parámetros de consulta. En DRF se leen con `self.request.query_params`, que es lo mismo que `request.GET` con un nombre más claro.
- `perform_create()` se ejecuta al guardar un alta: asigna como autor al usuario autenticado.
- `destroy()` de `CategoriaViewSet` captura el `ProtectedError` que produce `on_delete=PROTECT` y responde **409 Conflict** con un mensaje, en lugar de dejar que se produzca un error 500.

Las respuestas de DRF se construyen con `Response(datos, status=...)`, que convierte los datos al formato que pida el cliente (JSON o la API navegable). El módulo `status` define constantes con nombre para todos los códigos HTTP (`status.HTTP_201_CREATED`, `status.HTTP_409_CONFLICT`…), más legibles que los números.

7.4.1 El router

Un **router** genera las rutas de los ViewSets registrados:

```
# config/urls.py
from django.contrib import admin
from django.urls import include, path
from rest_framework.routers import DefaultRouter

from incidencias.api import CategoriaViewSet, IncidenciaViewSet

router = DefaultRouter()
router.register("categorias", CategoriaViewSet)
```

```
router.register("incidencias", IncidenciaViewSet, basename="incidencia")

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include(router.urls)),
    path("api-auth/", include("rest_framework.urls")),
    path("", include("incidencias.urls")),
]
```

Ruta generada	Nombre	Métodos
`api/`	`api-root`	`GET`: índice de la API con enlaces a cada recurso
`api/categorias/`	`categoria-list`	`GET`, `POST`
`api/categorias/<pk>/`	`categoria-detail`	`GET`, `PUT`, `PATCH`, `DELETE`
`api/incidencias/`	`incidencia-list`	`GET`, `POST`
`api/incidencias/<pk>/`	`incidencia-detail`	`GET`, `PUT`, `PATCH`, `DELETE`

`register()` necesita un `basename` para dar nombre a las rutas. Si el ViewSet tiene el atributo `queryset`, lo deduce del modelo (`categoria`); `IncidenciaViewSet` usa `get_queryset()` en su lugar, y por eso se indica `basename="incidencia"`. La ruta `api-auth/` añade las páginas de inicio y cierre de sesión de la API navegable. La ruta vacía de la aplicación se deja la última, porque es la más general.

Ejercicio 7.2. Primera API

Objetivo: publicar los recursos de categorías e incidencias con ViewSets y un router.

1. Escribid `incidencias/api.py` y modificad `config/urls.py` como en los ejemplos.
2. Ejecutad `python manage.py cargar_datos` y haced estas peticiones:

   ```
   > curl.exe http://127.0.0.1:8000/api/
   > curl.exe http://127.0.0.1:8000/api/categorias/
   > curl.exe -i http://127.0.0.1:8000/api/incidencias/
   ```

3. Abrid http://127.0.0.1:8000/api/incidencias/ en el navegador.

Verificación.

- `/api/` devuelve `{"categorias":"http://127.0.0.1:8000/api/categorias/","incidencias":"http:`
- `/api/categorias/` devuelve una lista JSON de tres objetos, Hardware, Red y Software, en ese orden (el `ordering` del modelo): `[{"id":1,"nombre":"Hardware","descripcion":"Equipos, periféricos y proyectores"},{"id":3,"nombre":"Red",...},{"id":2,"nombre":"Software",...`
- `/api/incidencias/` devuelve `HTTP/1.1 200 OK`, `Content-Type: application/json` y la cabecera `Allow: GET, POST, HEAD, OPTIONS`; el cuerpo es una lista que empieza por `[` con las 6 incidencias, la primera «Impresora de la sala de profesores atascada» y la última «Proyector del aula 12 no enciende». En cada una, `categoria` es el `id` de la categoría y `autor`, el nombre de usuario.

- En el navegador se ve la **API navegable** de DRF: la misma respuesta con formato, la cabecera HTTP, botones para elegir el formato y un enlace «Log in» arriba a la derecha.

7.5 Operaciones de escritura

Para crear, modificar o borrar hay que estar autenticado (`IsAuthenticatedOrReadOnly`). Con la autenticación básica, `curl.exe` envía el usuario y la contraseña con la opción `-u usuario:contraseña`.

Los datos se pueden enviar de dos formas:

- **Como formulario**, con `-d "campo=valor&campo=valor"`. DRF los acepta igual que el JSON. Es lo más cómodo en PowerShell.
- **Como JSON**, escribiéndolo en un fichero (por ejemplo, `nueva.json`, guardado en UTF-8 con Visual Studio Code) y enviándolo con `-H "Content-Type: application/json" -d "@nueva.json"`. Escribir el JSON directamente en la línea de órdenes de PowerShell obliga a escapar las comillas y es fácil equivocarse.

Opción de `curl.exe`	Significado
`-i`	Muestra las cabeceras de la respuesta
`-X MÉTODO`	Método HTTP (`POST` es el implícito si se usa `-d`)
`-d "datos"` o `-d "@fichero"`	Cuerpo de la petición
`-H "Cabecera: valor"`	Añade una cabecera
`-u usuario:contraseña`	Autenticación básica

Ejercicio 7.3. Crear, modificar y borrar desde `curl.exe`

Objetivo: usar todos los métodos de la API y reconocer sus códigos de estado.

1. Comprobad en `/api/categorias/` el `id` de «Hardware» (si habéis seguido los ejercicios, es `1`).
2. Ejecutad, observando el código de estado de cada respuesta:

```
> curl.exe -i -X POST -d "titulo=Pantalla rota"
 ↪  http://127.0.0.1:8000/api/incidencias/
> curl.exe -i -u ana:mal http://127.0.0.1:8000/api/incidencias/
> curl.exe -i -u ana:Clave.2026 -d "titulo=abc"
 ↪  http://127.0.0.1:8000/api/incidencias/
> curl.exe -i -u ana:Clave.2026 -d "titulo=Pantalla rota&descripcion=No
 ↪  va&categoria=1&prioridad=alta" http://127.0.0.1:8000/api/incidencias/
> curl.exe -i -u ana:Clave.2026 -d "titulo=Pantalla rota en el aula
 ↪  3&descripcion=La pantalla del puesto 3 no muestra
 ↪  imagen&categoria=1&prioridad=alta" http://127.0.0.1:8000/api/incidencias/
```

3. Anotad el `id` de la incidencia creada (en los ejemplos, `N`) y ejecutad:

```
> curl.exe -i -u ana:Clave.2026 -X PATCH -d "estado=en_curso"
 ↪  http://127.0.0.1:8000/api/incidencias/N/
> curl.exe -i -u ana:Clave.2026 -X PATCH -d "estado=cerrada"
 ↪  http://127.0.0.1:8000/api/incidencias/N/
> curl.exe -i -u ana:Clave.2026 -X PUT -d "estado=resuelta"
 ↪  http://127.0.0.1:8000/api/incidencias/N/
> curl.exe -i -u ana:Clave.2026 -X DELETE
 ↪  http://127.0.0.1:8000/api/incidencias/N/
> curl.exe -i http://127.0.0.1:8000/api/incidencias/N/
> curl.exe -i -u ana:Clave.2026 -X DELETE
 ↪  http://127.0.0.1:8000/api/incidencias/
> curl.exe -i -u ana:Clave.2026 -X DELETE
 ↪  http://127.0.0.1:8000/api/categorias/1/
```

4. Cread `nueva.json` con el contenido `{"titulo": "Monitor del puesto 9 parpadea", "descripcion": "Parpadea al cabo de unos minutos", "categoria": 1}` y enviadlo con `curl.exe -i -u ana:Clave.2026 -H "Content-Type: application/json" -d "@nueva.json" http://127.0.0.1:8000/api/incidencias/`.

Verificación.

Petición	Respuesta
POST sin credenciales	`401 Unauthorized`, cabecera `WWW-Authenticate: Basic realm="api"` y `{"detail":"Las credenciales de autenticación no se proveyeron."}`
Contraseña incorrecta	`401 Unauthorized` y `{"detail":"Nombre de usuario/contraseña inválidos."}`
Título `abc`	`400 Bad Request` con errores en `titulo`, `descripcion` y `categoria`
Prioridad alta y descripción corta	`400 Bad Request` y `{"descripcion":["Las incidencias de prioridad alta necesitan una descripción de al menos 20 caracteres."]}`
Alta correcta	`201 Created`, el JSON de la incidencia con `"autor":"ana"` y `"estado":"abierta"`
PATCH `estado=en_curso`	`200 OK` con `"estado":"en_curso"` y la fecha `actualizada` cambiada
PATCH `estado=cerrada`	`400 Bad Request` y `{"estado":["\"cerrada\" no es una elección válida."]}`
PUT solo con el estado	`400 Bad Request`: PUT exige `titulo`, `descripcion` y `categoria` («Este campo es requerido.»)
DELETE del elemento	`204 No Content`, sin cuerpo

Petición	Respuesta
GET tras borrar	`404 Not Found` y `{"detail":"No Incidencia matches the given query."}`
DELETE a la colección	`405 Method Not Allowed` y `{"detail":"Método \"DELETE\" no permitido."}`
DELETE de la categoría 1	`409 Conflict` y `{"detail":"La categoría tiene incidencias y no se puede borrar."}`
POST con `nueva.json`	`201 Created` con `"prioridad":"media"` (el valor por defecto)

7.6 Filtros, paginación y formatos

7.6.1 Filtros por parámetros de consulta

Los filtros de `get_queryset()` permiten al cliente pedir solo lo que necesita: `/api/incidencias/?estado=abier`
Los parámetros que no se reconocen se ignoran. Para APIs con muchos filtros existe el paquete `django-filter`, que DRF integra; en este módulo basta con `get_queryset()`.

7.6.2 Paginación

La **paginación** divide un listado en páginas. Protege al servidor y al cliente: sin ella, un listado de cien mil incidencias se serializaría y se enviaría entero en cada petición. DRF la ofrece con varias clases; la más sencilla es `PageNumberPagination`, en la que el cliente pide la página con `?page=N`. Con ella, la respuesta de un listado **deja de ser una lista** y pasa a ser un objeto con cuatro claves:

Clave	Contenido
`count`	Número total de elementos, en todas las páginas
`next`	Dirección de la página siguiente, o `null`
`previous`	Dirección de la página anterior, o `null`
`results`	Los elementos de la página actual

La paginación se puede activar para **toda la API**, con `DEFAULT_PAGINATION_CLASS` y `PAGE_SIZE` en `REST_FRAMEWORK`, o para **un ViewSet concreto**, con su atributo `pagination_class`:

```
# incidencias/api.py (cambio temporal para el Ejercicio 7.4)
from rest_framework.pagination import PageNumberPagination


class PaginacionIncidencias(PageNumberPagination):
    page_size = 5
```

```
class IncidenciaViewSet(viewsets.ModelViewSet):
    serializer_class = IncidenciaSerializer
    pagination_class = PaginacionIncidencias
    # ... el resto, sin cambios
```

Cambiar la paginación modifica el **formato** de las respuestas, y por tanto el contrato con los clientes: un cliente que espera una lista deja de funcionar si recibe un objeto con `results`. Es una decisión que se toma al diseñar la API y que no se cambia a la ligera. En «Incidencias» se ha decidido **no paginar** (los listados son pequeños y el cliente de la Unidad 3 espera listas), por lo que la paginación del ejercicio siguiente se retira al terminarlo.

Ejercicio 7.4. Paginación de un listado

Objetivo: activar la paginación en un ViewSet, observar el cambio de formato de la respuesta y retirarla.

1. Añadid a `api.py` la clase `PaginacionIncidencias` y el atributo `pagination_class` de `IncidenciaViewSet`, como en el ejemplo, y ejecutad `python manage.py cargar_datos`.
2. Pedid:

   ```
   > curl.exe http://127.0.0.1:8000/api/incidencias/
   > curl.exe "http://127.0.0.1:8000/api/incidencias/?page=2"
   > curl.exe "http://127.0.0.1:8000/api/incidencias/?page=9"
   > curl.exe http://127.0.0.1:8000/api/categorias/
   ```

3. Borrad la clase `PaginacionIncidencias`, su importación y el atributo `pagination_class`, y repetid la primera petición.

Verificación.

- Con la paginación, `/api/incidencias/` devuelve un objeto que empieza por `{"count":6,"next":"http://127.0.0.1:8000/api/incidencias/?page=2","previous":null,"resu` con 5 incidencias, la primera «Impresora de la sala de profesores atascada».
- `?page=2` devuelve `"next":null`, `"previous":"http://127.0.0.1:8000/api/incidencias/"` y un único resultado, «Proyector del aula 12 no enciende». `?page=9` devuelve `404 Not Found` y `{"detail":"Página inválida."}`.
- `/api/categorias/` sigue devolviendo una lista: la paginación solo afecta al ViewSet que la declara.
- Tras el paso 3, `/api/incidencias/` vuelve a devolver una lista que empieza por `[` con las 6 incidencias, y `?page=9` se ignora (devuelve también las 6).

7.6.3 La API navegable

Cuando la petición la hace un navegador (que envía la cabecera `Accept: text/html`), DRF responde con la **API navegable**: una página HTML que muestra la respuesta JSON con formato y, si el usuario ha iniciado sesión, formularios para crear y modificar. Cualquier otro cliente, como `curl.exe` o el código JavaScript de la Unidad 3, recibe JSON. Elegir el formato de la respuesta según lo que pide el cliente se llama **negociación de contenido**. Se puede forzar el JSON en el navegador añadiendo `?format=json` a la dirección.

Ejercicio 7.5. Filtros y API navegable

Objetivo: usar los filtros y crear datos desde la API navegable.

1. Ejecutad `python manage.py cargar_datos` para partir de los datos de prueba. Pedid con `curl.exe` las incidencias abiertas, las de prioridad alta y las que cumplen las dos condiciones a la vez.
2. Abrid en el navegador http://127.0.0.1:8000/api/incidencias/ y pulsad «Log in». Entrad como `luis` (contraseña `Clave.2026`).
3. Usad el formulario HTML del final de la página para crear una incidencia de la categoría 3 (Red).
4. Abrid su detalle (`/api/incidencias/N/`) y pulsad el botón «DELETE».
5. Abrid http://127.0.0.1:8000/api/incidencias/?format=json.

Verificación.

- `?estado=abierta` devuelve una lista de 3 incidencias, `?prioridad=alta` otra de 3 y `?estado=abierta&prioridad=alta` una lista con 1 (el proyector). Para contarlas en PowerShell: `(curl.exe -s "http://127.0.0.1:8000/api/incidencias/?estado=abierta" | ConvertFrom-Json).Count`.
- Tras entrar, la esquina superior derecha muestra «luis» en lugar de «Log in» y aparece el formulario de alta. La incidencia creada tiene `"autor": "luis"` aunque el formulario no tenga ningún campo para el autor.
- El botón «DELETE» pide confirmación y, al aceptar, la página muestra `HTTP 204 No Content`.
- Con `?format=json` el navegador muestra el JSON sin formato, como lo recibiría un programa.

7.7 Pruebas de una API sin navegador

Además de `curl.exe` y la API navegable, para probar APIs se usan herramientas con interfaz gráfica, como la extensión **Thunder Client** o **REST Client** de Visual Studio Code, Postman o Bruno. Todas hacen lo mismo que `curl.exe`: construir una petición HTTP con su método, sus cabeceras y su cuerpo, y mostrar la respuesta. `curl.exe` tiene la ventaja de que las órdenes se pueden copiar, documentar y repetir exactamente; por eso se usa en las verificaciones de este documento.

La Unidad 6 automatizará estas comprobaciones con pruebas escritas en Python (pytest).

Para reflexionar. *El serializador muestra el autor como su nombre de usuario y la categoría como su* `id`*. ¿Qué ventajas e inconvenientes tiene cada representación para el cliente Vue que consumirá esta API en la Unidad 3?*

Capítulo 8

Seguridad de la aplicación

Para empezar

1. ¿Cómo guardaríais las contraseñas de los usuarios en la base de datos? ¿Qué pasaría si alguien robara esa tabla?
2. Si una página de otro sitio web contiene un formulario que envía datos a vuestra aplicación, y el usuario que la visita tiene la sesión iniciada en ella, ¿qué podría ocurrir?
3. ¿Qué pasaría si un usuario escribiera `<script>alert('hola')</script>` como título de una incidencia y la aplicación lo mostrara tal cual?

8.1 Autenticación y autorización

Dos conceptos que se confunden a menudo:

- **Autenticación**: comprobar **quién** es el usuario (con una contraseña, un token, un certificado…).
- **Autorización**: decidir **qué puede hacer** el usuario ya identificado (ver, crear, modificar, borrar…).

Django resuelve la autenticación con la aplicación `django.contrib.auth` y el *middleware* de sesiones y autenticación; la autorización, con los permisos de usuario, el decorador `login_required` y, en la API, las clases de permisos de DRF. Este apartado recorre además las protecciones frente a los ataques más comunes en aplicaciones web, recogidos en la lista **OWASP Top 10**, y cómo las aplica el framework.

Riesgo	Protección en Django	Apartado
Contraseñas robadas	Almacenamiento con *hash* y validadores de contraseña	8.1
Acceso sin autenticar	`login_required`, `LoginRequiredMixin`, permisos de DRF	8.2, 8.5
Falsificación de peticiones (CSRF)	`CsrfViewMiddleware` y `{% csrf_token %}`	8.3

Riesgo	Protección en Django	Apartado
Robo de credenciales en la API	Tokens en lugar de contraseñas	8.4
Datos de entrada maliciosos	Formularios y serializadores	8.6
Inyección SQL	Consultas parametrizadas del ORM	8.6
Inyección de código en las páginas (XSS)	Escapado automático de las plantillas	8.6
Peticiones desde otros orígenes	Política del mismo origen y CORS	8.7
Configuración insegura	`check --deploy`	8.8

8.2 Contraseñas

Django **nunca** guarda las contraseñas en claro. Guarda el resultado de aplicarles una **función hash**: una transformación de un solo sentido, que permite comprobar si una contraseña es correcta (se le aplica la misma función y se compara el resultado) pero no recuperar la contraseña a partir del resultado. Además, a cada contraseña se le añade una **sal** (*salt*) aleatoria antes de calcular el hash, de forma que dos usuarios con la misma contraseña tienen hashes distintos, y la función se aplica muchas veces seguidas para que probar contraseñas por fuerza bruta sea muy lento.

El campo `password` de un usuario tiene este formato:

```
pbkdf2_sha256$1000000$HpGF0tUu...$Wq3k...=
algoritmo    iteraciones  sal     hash
```

Django 5.2 usa por defecto el algoritmo **PBKDF2** con SHA-256 y un millón de iteraciones. Por eso las contraseñas se asignan siempre con `create_user()`, `set_password()` o los formularios de Django, y se comprueban con `check_password()`; nunca se escribe directamente en el atributo `password`.

8.2.1 Validadores de contraseñas

El ajuste `AUTH_PASSWORD_VALIDATORS` define las reglas que deben cumplir las contraseñas nuevas. Los cuatro validadores del proyecto generado rechazan las contraseñas:

Validador	Rechaza
`UserAttributeSimilarityValidator`	Las parecidas al nombre de usuario, al nombre o al correo

Validador	Rechaza
`MinimumLengthValidator`	Las de menos de 8 caracteres (configurable con `OPTIONS`)
`CommonPasswordValidator`	Las que están en una lista de 20 000 contraseñas comunes
`NumericPasswordValidator`	Las formadas solo por números

Los aplican `createsuperuser`, `changepassword` y los formularios de cambio de contraseña. **No** los aplica `create_user()`, que se usa en código y confía en quien lo llama.

Ejercicio 8.1. Hash y validación de contraseñas

Objetivo: comprobar cómo guarda Django las contraseñas y qué reglas les aplica.

1. En la consola de Django:

```
>>> ana = User.objects.get(username="ana")
>>> ana.password
>>> ana.check_password("Clave.2026")
>>> ana.check_password("clave.2026")
>>> from django.contrib.auth.password_validation import validate_password
>>> validate_password("1234")
>>> validate_password("password")
```

2. Salid de la consola y cambiad la contraseña de `luis` con `python manage.py changepassword luis`, probando primero con `12345678` y después con `Clave.2026`.

Verificación. `ana.password` empieza por `'pbkdf2_sha256$1000000$'` y no contiene `Clave.2026`. `check_password` da `True` con la contraseña correcta y `False` con la misma en minúsculas. `validate_password("1234")` lanza `ValidationError` con tres mensajes: «La contraseña es demasiado corta. Debe contener por lo menos 8 caracteres.», «Esta contraseña es demasiado común.» y «Esta contraseña es completamente numérica.». `"password"` produce solo «Esta contraseña es demasiado común.». En el paso 2, `12345678` se rechaza con los mensajes de contraseña común y numérica, y `Clave.2026` se acepta con `Password changed successfully for user 'luis'`.

8.3 Inicio de sesión y vistas protegidas

8.3.1 Las vistas de autenticación de Django

`django.contrib.auth` incluye las vistas de inicio de sesión, cierre de sesión y cambio y recuperación de contraseña. Se activan todas incluyendo sus rutas:

```
# config/urls.py (ruta añadida)
    path("cuentas/", include("django.contrib.auth.urls")),
```

Ruta	Nombre	Vista	Plantilla que busca
`cuentas/login/`	`login`	`LoginView`	`registration/login.html`
`cuentas/logout/`	`logout`	`LogoutView` (solo POST)	Ninguna: redirige
`cuentas/password_change/`	`password_change`	`PasswordChangeView`	`registration/password_change_form.html`
`cuentas/password_reset/`	`password_reset`	`PasswordResetView`	`registration/password_reset_form.html`

Django proporciona las vistas, pero no las plantillas: hay que escribir al menos `registration/login.html`. Se guarda en la carpeta `templates` de la aplicación, pero **sin** la subcarpeta `incidencias`, porque `LoginView` la busca con el nombre exacto `registration/login.html`:

```
{# incidencias/templates/registration/login.html #}
{% extends "incidencias/base.html" %}

{% block titulo %}Entrar{% endblock %}

{% block contenido %}
<h1>Entrar</h1>
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <input type="hidden" name="next" value="{{ next }}">
  <button type="submit">Entrar</button>
</form>
{% endblock %}
```

El campo oculto `next` conserva la página a la que el usuario intentaba entrar, para volver a ella tras identificarse.

Tres ajustes indican adónde se redirige:

```
# config/settings.py (añadido al final)
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "incidencias:lista"
LOGOUT_REDIRECT_URL = "incidencias:lista"
```

Ajuste	Uso	Valor por defecto
`LOGIN_URL`	Adónde se envía a un usuario no autenticado que pide una vista protegida	`/accounts/login/`

Ajuste	Uso	Valor por defecto
`LOGIN_REDIRECT_URL`	Adónde se va tras iniciar sesión, si no hay `next`	`/accounts/profile/`
`LOGOUT_REDIRECT_URL`	Adónde se va tras cerrar sesión	Ninguno

Los tres admiten un nombre de ruta en lugar de una dirección.

Desde Django 5.0, el cierre de sesión **solo admite POST**: un enlace normal (`GET`) responde 405. Así se evita que otra web pueda cerrar la sesión del usuario con una simple imagen o un enlace. Por eso la plantilla base usa un pequeño formulario con su token CSRF:

```
{# incidencias/templates/incidencias/base.html (el resto, como en el apartado 3) #}
  <header>
    <a href="{% url 'incidencias:lista' %}">Incidencias</a>
    | <a href="{% url 'incidencias:acerca' %}">Acerca de</a>
    {% if user.is_authenticated %}
      | <a href="{% url 'incidencias:nueva' %}">Nueva incidencia</a>
      | {{ user.username }}
      <form action="{% url 'logout' %}" method="post" class="en-linea">
        {% csrf_token %}
        <button type="submit">Salir</button>
      </form>
    {% else %}
      | <a href="{% url 'login' %}">Entrar</a>
    {% endif %}
  </header>
```

La clase `en-linea` se define en `estilos.css` con `.en-linea { display: inline; }`, para que el botón quede en la misma línea que los enlaces.

8.3.2 Proteger vistas

Tipo de vista	Protección	Ejemplo
Vista de función	Decorador `@login_required`	`@login_required` encima de `def nueva(request):`
Vista basada en clase	*Mixin* `LoginRequiredMixin`, a la izquierda	`class IncidenciaCreateView(LoginRequir CreateView)`

Tipo de vista	Protección	Ejemplo
Con un permiso concreto	`@permission_required("app.permiso")` o `PermissionRequiredMixin`	`permission_required = "incidencias.change_incidencia"`

Si el usuario no está autenticado, la vista no se ejecuta: se redirige a `LOGIN_URL` con la página pedida en el parámetro `next`. Django crea automáticamente cuatro permisos por cada modelo (`add_`, `change_`, `delete_` y `view_`), que se asignan a usuarios o grupos desde el panel de administración.

Ocultar un enlace en la plantilla (`{% if user.is_authenticated %}`) **no** protege nada: cualquiera puede escribir la dirección a mano. La protección real está siempre en la vista.

Ejercicio 8.2. Inicio y cierre de sesión

Objetivo: añadir el inicio y el cierre de sesión a la aplicación y comprobar la protección de la vista de alta.

1. Añadid la ruta `cuentas/`, los tres ajustes de redirección, la plantilla `registration/login.html`, la nueva `base.html` y la clase `en-linea`.
2. Cerrad la sesión del panel de administración, si la teníais abierta, y abrid http://127.0.0.1:8000/incidencias/nueva/.
3. Entrad con `ana` y una contraseña incorrecta; después, con la correcta.
4. Pulsad «Salir». Abrid después http://127.0.0.1:8000/cuentas/logout/ escribiendo la dirección.

Verificación.

- En el paso 2 el navegador va a `/cuentas/login/?next=/incidencias/nueva/` y muestra el formulario con «Nombre de usuario» y «Contraseña».
- Con la contraseña incorrecta aparece «Por favor, introduzca un nombre de usuario y clave correctos. Observe que ambos campos pueden ser sensibles a mayúsculas.». Con la correcta, se llega directamente al formulario de nueva incidencia (gracias a `next`) y la cabecera muestra «Nueva incidencia | ana» y el botón «Salir».
- «Salir» vuelve al listado y la cabecera muestra «Entrar». La dirección `/cuentas/logout/` escrita a mano devuelve `405 Method Not Allowed`.

8.4 Protección CSRF

Un ataque de **falsificación de petición en sitios cruzados** (CSRF, *Cross-Site Request Forgery*) aprovecha que el navegador envía automáticamente las cookies de un sitio en cada petición a ese sitio. Si un usuario tiene la sesión iniciada en «Incidencias» y visita una página maliciosa que contiene un formulario oculto que hace POST a `/incidencias/nueva/`, el navegador enviaría la cookie de sesión y la incidencia se crearía en su nombre sin que lo sepa.

Django lo impide con un **token CSRF**:

1. Al generar una página, Django envía al navegador una cookie `csrftoken` y, en cada formulario que lleva `{% csrf_token %}`, un campo oculto `csrfmiddlewaretoken` con un valor relacionado con ella.
2. Al recibir un POST, PUT, PATCH o DELETE, `CsrfViewMiddleware` comprueba que el valor del formulario (o de la cabecera `X-CSRFToken`) corresponde al de la cookie. Comprueba también la cabecera `Origin` de la petición.
3. Una web maliciosa no puede leer la cookie de otro sitio ni, por tanto, conocer el token: su formulario falsificado llega sin un token válido y se rechaza con **403**.

En la API, DRF aplica la protección CSRF **solo** a las peticiones autenticadas con la **sesión** (`SessionAuthentication`), porque es en ellas donde el navegador envía la credencial automáticamente. Las peticiones autenticadas con **token** (apartado 8.4) no la necesitan: el token no se envía solo, lo tiene que añadir expresamente el código del cliente.

Ejercicio 8.3. Peticiones sin token CSRF

Objetivo: comprobar que la aplicación rechaza los envíos sin token CSRF.

1. Abrid el formulario de nueva incidencia (con la sesión iniciada) y mirad su código fuente.
2. Con las herramientas de desarrollo del navegador (`F12`, pestaña *Inspector* o *Elementos*), borrad el campo oculto `csrfmiddlewaretoken` del formulario, rellenadlo y enviadlo.
3. Desde la terminal, intentad enviar el formulario de inicio de sesión con `curl.exe`:

```
> curl.exe -i -d "username=ana&password=Clave.2026"
 ↪  http://127.0.0.1:8000/cuentas/login/
```

Verificación. El código fuente muestra `<input type="hidden" name="csrfmiddlewaretoken" value="...">` en el formulario de alta y en el botón «Salir». En el paso 2, la respuesta es la página «Prohibido (403)» con el texto «La verificación CSRF ha fallado. Solicitud abortada.», y la incidencia no se crea. En el paso 3, `curl.exe` recibe `HTTP/1.1 403 Forbidden` con el mismo mensaje.

8.5 Autenticación por token en la API

La autenticación básica del apartado 7 envía el usuario y la contraseña en **cada** petición, y la autenticación por sesión depende de cookies y de CSRF, pensadas para páginas HTML. Para un cliente separado (la aplicación Vue de la Unidad 3, una aplicación móvil) lo habitual es un **token**: una cadena aleatoria que el servidor entrega al usuario una vez, a cambio de su usuario y contraseña, y que el cliente envía después en la cabecera `Authorization` de cada petición:

```
Authorization: Token 642fc4f346910a7b5c492a1a894414034e92e6c6
```

Si el token se filtra, se puede anular sin cambiar la contraseña. DRF incluye la aplicación `rest_framework.authtoken`, que guarda un token por usuario en una tabla y ofrece la vista `obtain_auth_token` para obtenerlo.

```
# config/settings.py (fragmentos)
INSTALLED_APPS = [
    # ...
    "rest_framework",
    "rest_framework.authtoken",
    "incidencias",
]

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": [
        "rest_framework.authentication.TokenAuthentication",
        "rest_framework.authentication.SessionAuthentication",
    ],
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.IsAuthenticatedOrReadOnly",
    ],
}
```

```
# config/urls.py (importación y ruta añadidas)
from rest_framework.authtoken.views import obtain_auth_token

    path("api/token/", obtain_auth_token, name="api-token"),
```

La ruta `api/token/` se escribe **antes** de `path("api/", include(router.urls))` para que quede claro que no la genera el router. `TokenAuthentication` va en primer lugar porque DRF usa la **primera** clase de la lista para construir la respuesta 401 y su cabecera `WWW-Authenticate: Token`; si la primera fuera `SessionAuthentication`, que no tiene esa cabecera, las peticiones sin credenciales recibirían 403 en lugar de 401. Se mantiene `SessionAuthentication` para seguir usando la API navegable.

`rest_framework.authtoken` tiene sus propios modelos: tras añadirla, hay que ejecutar `migrate`.

Ejercicio 8.4. Tokens de acceso

Objetivo: obtener un token y usarlo para autenticarse en la API.

1. Añadid `rest_framework.authtoken`, cambiad `DEFAULT_AUTHENTICATION_CLASSES`, añadid la ruta `api/token/` y ejecutad `python manage.py migrate`.
2. Pedid un token para `ana` y guardadlo en una variable de PowerShell:

   ```
   > curl.exe -X POST -d "username=ana&password=Clave.2026"
    ↪  http://127.0.0.1:8000/api/token/
   > $token = (curl.exe -s -X POST -d "username=ana&password=Clave.2026"
    ↪  http://127.0.0.1:8000/api/token/ | ConvertFrom-Json).token
   > $token
   ```

3. Probad estas peticiones:

```
> curl.exe -i -X POST -d "username=ana&password=mal"
 ↪  http://127.0.0.1:8000/api/token/
> curl.exe -i http://127.0.0.1:8000/api/token/
> curl.exe -i -X POST -d "titulo=Pantalla rota"
 ↪  http://127.0.0.1:8000/api/incidencias/
> curl.exe -i -H "Authorization: Token 123"
 ↪  http://127.0.0.1:8000/api/incidencias/
> curl.exe -i -u ana:Clave.2026 -d "titulo=Pantalla rota"
 ↪  http://127.0.0.1:8000/api/incidencias/
> curl.exe -i -H "Authorization: Token $token" -d "titulo=Pantalla rota en el
 ↪  aula 3&descripcion=La pantalla del puesto 3 no muestra
 ↪  imagen&categoria=1&prioridad=alta" http://127.0.0.1:8000/api/incidencias/
```

4. Entrad en el panel de administración y buscad la sección «Token de autenticación».

Verificación.

- `migrate` aplica `authtoken.0001_initial`, `authtoken.0002_auto_20160226_1747`, `authtoken.0003_tokenproxy` y `authtoken.0004_alter_tokenproxy_options`.
- El paso 2 devuelve `{"token":"..."}` con 40 caracteres hexadecimales; si se repite, devuelve **el mismo** token.
- Credenciales incorrectas: `400 Bad Request` y `{"non_field_errors":["No puede iniciar sesión con las credenciales proporcionadas."]}`. `GET` a `/api/token/`: `405 Method Not Allowed`.
- POST sin credenciales: `401 Unauthorized` con `WWW-Authenticate: Token`. Token inventado: `401` y `{"detail":"Token inválido."}`. Con `-u` (autenticación básica, ya desactivada): `401`.
- Con el token correcto: `201 Created` y `"autor":"ana"`.
- En el panel aparece la sección «Token de autenticación» con «Tokens», donde se ve el token de `ana` y se puede borrar para anularlo.

8.6 Permisos en la API

Con `IsAuthenticatedOrReadOnly`, **cualquier** usuario autenticado puede modificar o borrar **cualquier** incidencia: `luis` podría borrar las de `ana`. Las clases de **permisos** de DRF deciden, para cada petición, si se permite:

Permiso de DRF	Permite
`AllowAny`	Todo, a todos
`IsAuthenticated`	Todo, solo a usuarios autenticados
`IsAuthenticatedOrReadOnly`	Leer a todos; escribir, solo autenticados
`IsAdminUser`	Todo, solo a usuarios con `is_staff`

Permiso de DRF	Permite
`DjangoModelPermissions`	Según los permisos `add_`, `change_` y `delete_` del usuario

Un permiso propio es una clase que hereda de `BasePermission` y redefine uno o dos métodos:

- `has_permission(request, view)`: se comprueba en **todas** las peticiones, antes de nada.
- `has_object_permission(request, view, obj)`: se comprueba solo en las peticiones sobre **un objeto concreto** (`retrieve`, `update`, `partial_update`, `destroy`), después de obtenerlo.

```
# incidencias/permissions.py
from rest_framework import permissions


class EsAutorOSoloLectura(permissions.BasePermission):
    """Cualquiera puede leer; solo el autor puede modificar o borrar."""

    def has_object_permission(self, request, view, obj):
        if request.method in permissions.SAFE_METHODS:
            return True
        return obj.autor == request.user
```

`SAFE_METHODS` es la tupla `("GET", "HEAD", "OPTIONS")`: los métodos que no modifican nada. El permiso se aplica al ViewSet con el atributo `permission_classes`, que **sustituye** a los permisos por defecto; por eso se incluyen los dos, y se tienen que cumplir ambos:

```
# incidencias/api.py (IncidenciaViewSet completo)
from rest_framework import permissions, status, viewsets   # se añade permissions

from .permissions import EsAutorOSoloLectura   # importación añadida


class IncidenciaViewSet(viewsets.ModelViewSet):
    serializer_class = IncidenciaSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly,
↪   EsAutorOSoloLectura]

    def get_queryset(self):
        consulta = Incidencia.objects.select_related("categoria", "autor")
        estado = self.request.query_params.get("estado")
        prioridad = self.request.query_params.get("prioridad")
        if estado:
            consulta = consulta.filter(estado=estado)
        if prioridad:
            consulta = consulta.filter(prioridad=prioridad)
        return consulta

    def perform_create(self, serializer):
        serializer.save(autor=self.request.user)
```

Ejercicio 8.5. Permiso «solo el autor»

Objetivo: escribir un permiso propio y comprobar que solo el autor puede modificar o borrar sus incidencias.

1. Cread `permissions.py` y aplicad el permiso a `IncidenciaViewSet`.
2. Obtened un token para `ana` (`$token_ana`) y otro para `luis` (`$token_luis`), como en el ejercicio anterior.
3. Con el token de `ana`, cread una incidencia y anotad su `id` (`N`).
4. Intentad modificarla y borrarla con el token de `luis`; después, modificadla con el de `ana`:

```
> curl.exe -i -X PATCH -H "Authorization: Token $token_luis" -d
 ↪  "estado=resuelta" http://127.0.0.1:8000/api/incidencias/N/
> curl.exe -i -X DELETE -H "Authorization: Token $token_luis"
 ↪  http://127.0.0.1:8000/api/incidencias/N/
> curl.exe -i -X PATCH -H "Authorization: Token $token_ana" -d
 ↪  "estado=en_curso" http://127.0.0.1:8000/api/incidencias/N/
> curl.exe -i -X PATCH -H "Authorization: Token $token_ana" -d "autor=luis"
 ↪  http://127.0.0.1:8000/api/incidencias/N/
> curl.exe -i http://127.0.0.1:8000/api/incidencias/N/
```

Verificación. Las dos peticiones de `luis` devuelven `403 Forbidden` y `{"detail":"Usted no tiene permiso para realizar esta acción."}`, y la incidencia no cambia. La primera de `ana` devuelve `200 OK` con `"estado":"en_curso"`. La segunda también devuelve `200 OK`, pero `"autor"` sigue siendo `"ana"`: el campo es de solo lectura y DRF ignora el valor enviado. La lectura sin credenciales devuelve `200 OK`.

8.7 Validación de entrada, inyección SQL y XSS

8.7.1 Validar siempre en el servidor

Todo lo que llega del cliente puede estar manipulado: los campos ocultos, las listas desplegables, los atributos `required` o `maxlength` del HTML y las comprobaciones de JavaScript se saltan con `curl.exe` en segundos. La validación que protege la aplicación es **la del servidor**: los formularios (`ModelForm`), los serializadores y el método `clean()` del modelo. La validación en el cliente solo mejora la experiencia de uso.

En «Incidencias»:

- Los campos con `choices` solo admiten sus valores (`"cerrada" no es una elección válida`).
- La categoría debe existir; el título no puede superar 120 caracteres.
- El autor no se recibe del cliente: lo asigna el servidor con `request.user` (en `form_valid` y en `perform_create`).
- Las reglas de negocio se validan en el modelo y se reutilizan en el serializador.

8.7.2 Inyección SQL

La **inyección SQL** se produce cuando un dato del usuario se **concatena** dentro de una sentencia SQL, de modo que el usuario puede cambiar el significado de la sentencia. Django permite escribir SQL a mano con `raw()`, y es ahí donde puede aparecer el problema:

```
# Ejemplo INSEGURO: no copiar en una aplicación real
texto = "zzz' OR 1=1 --"
sql = f"SELECT * FROM incidencias_incidencia WHERE titulo LIKE '%{texto}%'"
Incidencia.objects.raw(sql)
```

La sentencia resultante es `... WHERE titulo LIKE '%zzz' OR 1=1 --%'`: la comilla del texto cierra la cadena, `OR 1=1` hace que la condición sea siempre cierta y `--` convierte el resto en un comentario. La consulta devuelve **todas** las incidencias. Con otras cadenas se podrían leer otras tablas, como la de usuarios.

La solución es pasar los datos como **parámetros**, que el controlador de la base de datos envía por separado de la sentencia y nunca interpreta como SQL:

```
Incidencia.objects.raw(
    "SELECT * FROM incidencias_incidencia WHERE titulo LIKE %s", [f"%{texto}%"]
)
```

El ORM **siempre** usa parámetros: `Incidencia.objects.filter(titulo__icontains=texto)` es seguro con cualquier texto. La regla práctica es usar el ORM y, si alguna vez hace falta SQL a mano, no formatear nunca la sentencia con f-strings ni con `+`.

8.7.3 XSS y el escapado automático

El **XSS** (*Cross-Site Scripting*) consiste en conseguir que la aplicación muestre a otros usuarios código HTML o JavaScript introducido por un atacante: por ejemplo, un título de incidencia que contiene `<script>...</script>`. Ese código se ejecutaría en el navegador de quien viera el listado, con su sesión.

Las plantillas de Django **escapan automáticamente** todas las variables: los caracteres `<`, `>`, `&`, `'` y `"` se sustituyen por sus entidades HTML (`<`, `>`…), de modo que el navegador los muestra como texto en lugar de interpretarlos. Se puede desactivar con el filtro `safe` o la etiqueta `{% autoescape off %}`, pero **nunca** con datos que vengan del usuario.

En la API, el JSON no se interpreta como HTML; el riesgo aparece cuando el cliente lo inserta en la página. En la Unidad 3 se verá que Vue también escapa los datos por defecto.

Ejercicio 8.6. Inyección SQL y XSS

Objetivo: comprobar el efecto de una inyección SQL con `raw()` y la protección del escapado automático.

1. En la consola de Django, ejecutad la consulta insegura de este apartado y contad sus resultados con `len(list(...))`. Repetid con la versión parametrizada y con `filter(titulo__icontains=texto)`.
2. Con la sesión iniciada como `ana`, cread desde el formulario de la aplicación una incidencia de prioridad baja con el título `<script>alert('XSS')</script>`.
3. Abrid el listado y su código fuente.
4. En `lista.html`, cambiad temporalmente `{{ incidencia.titulo }}` por `{{ incidencia.titulo|safe }}` y recargad. Deshaced el cambio y borrad la incidencia desde el panel de administración.

Verificación. La consulta insegura devuelve `6` (todas las incidencias), y la parametrizada y la del ORM, `0`. En el paso 3, el listado muestra el título como texto, `<script>alert('XSS')</script>`, y el código fuente contiene `<script>alert('XSS')</script>`. En el paso 4, al cargar el listado el navegador ejecuta el script y muestra una ventana con el texto «XSS»: es exactamente lo que el escapado automático impide.

8.8 CORS

8.8.1 La política del mismo origen

El **origen** de una página es la combinación de esquema, dominio y puerto: `http://localhost:5173` y `http://127.0.0.1:8000` son orígenes **distintos**. Por seguridad, los navegadores aplican la **política del mismo origen**: el código JavaScript de una página solo puede leer las respuestas de peticiones a su propio origen. Sin ella, cualquier web que visitara un usuario podría leer, con su sesión, los datos de las demás aplicaciones en las que esté conectado.

En la Unidad 3, el cliente Vue funcionará en `http://localhost:5173` y pedirá datos a la API en `http://127.0.0.1:8000`. El navegador bloqueará esas respuestas salvo que el servidor indique expresamente que confía en ese origen. Ese mecanismo es **CORS** (*Cross-Origin Resource Sharing*): el servidor responde con la cabecera `Access-Control-Allow-Origin` indicando qué orígenes pueden leer sus respuestas. Para las peticiones que modifican datos o llevan cabeceras como `Authorization`, el navegador envía antes una petición **de comprobación previa** (*preflight*) con el método `OPTIONS`, y solo hace la petición real si la respuesta lo permite.

CORS **no** protege al servidor: `curl.exe` y cualquier programa que no sea un navegador ignoran estas cabeceras. Lo que protege es al **usuario del navegador**.

8.8.2 django-cors-headers

El paquete `django-cors-headers` añade a Django un middleware que gestiona CORS:

```
> python -m pip install "django-cors-headers~=4.9"
> pip freeze > requirements.txt
```

```
# config/settings.py (fragmentos)
INSTALLED_APPS = [
    # ...
    "rest_framework",
```

```
    "rest_framework.authtoken",
    "corsheaders",
    "incidencias",
]

MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "corsheaders.middleware.CorsMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
    "incidencias.middleware.TiempoRespuestaMiddleware",
    "incidencias.middleware.MantenimientoMiddleware",
]

CORS_ALLOWED_ORIGINS = [
    "http://localhost:5173",
    "http://127.0.0.1:5173",
]
```

`CorsMiddleware` debe ir **lo más arriba posible**, y en todo caso antes de `CommonMiddleware`, para que añada sus cabeceras también a las respuestas que otras capas generan sin llegar a la vista (como las redirecciones o las respuestas a `OPTIONS`). Es otro ejemplo de por qué importa el orden del middleware. `CORS_ALLOWED_ORIGINS` enumera los orígenes de confianza; existe también `CORS_ALLOW_ALL_ORIGINS = True`, que **no** debe usarse fuera de pruebas puntuales.

Ejercicio 8.7. Cabeceras CORS

Objetivo: configurar CORS para el futuro cliente Vue y comprobar las cabeceras con `curl.exe`.

1. Instalad `django-cors-headers` y configuradlo como en el ejemplo.
2. Simulad las peticiones que hará el navegador desde el cliente Vue, añadiendo la cabecera `Origin`:

```
> curl.exe -i -H "Origin: http://localhost:5173"
 ↪  http://127.0.0.1:8000/api/categorias/
> curl.exe -i -H "Origin: http://localhost:8080"
 ↪  http://127.0.0.1:8000/api/categorias/
> curl.exe -i -X OPTIONS -H "Origin: http://localhost:5173" -H
 ↪  "Access-Control-Request-Method: POST" -H "Access-Control-Request-Headers:
 ↪  authorization, content-type" http://127.0.0.1:8000/api/incidencias/
```

Verificación. La primera respuesta incluye `access-control-allow-origin: http://localhost:5173`. La segunda devuelve `200 OK` con los datos, pero **sin** ninguna cabecera `access-control-...`: un navegador en `http://localhost:8080` no dejaría que el JavaScript leyera la respuesta. La tercera (la comprobación previa) devuelve `200 OK` con `access-control-allow-origin:`

`http://localhost:5173`, `access-control-allow-headers: accept, authorization, content-type, user-agent, x-csrftoken, x-requested-with` y `access-control-allow-methods: DELETE, GET, OPTIONS, PATCH, POST, PUT`.

8.9 Comprobación de despliegue y secretos

La configuración de desarrollo (`DEBUG = True`, la clave secreta en el código, HTTP sin cifrar) es insegura en producción. Django incluye una comprobación que revisa los ajustes de seguridad más importantes:

```
> python manage.py check --deploy
```

Aviso	Ajuste	Qué indica
`security.W004`	`SECURE_HSTS_SECONDS`	No se obliga al navegador a usar siempre HTTPS (HSTS)
`security.W008`	`SECURE_SSL_REDIRECT`	Las peticiones HTTP no se redirigen a HTTPS
`security.W009`	`SECRET_KEY`	La clave es corta o empieza por `django-insecure-`
`security.W012`	`SESSION_COOKIE_SECURE`	La cookie de sesión se envía también por HTTP
`security.W016`	`CSRF_COOKIE_SECURE`	La cookie CSRF se envía también por HTTP
`security.W018`	`DEBUG`	El modo de depuración está activo
`security.W020`	`ALLOWED_HOSTS`	No se han indicado los dominios del sitio

Resolver estos avisos forma parte del despliegue (Unidad 4): se hará con ajustes distintos para producción, leídos de **variables de entorno**, de modo que los secretos no estén en el código ni en el repositorio. Como adelanto, la clave secreta se leería así:

```
# config/settings.py (adelanto de la Unidad 4)
import os

SECRET_KEY = os.environ.get("DJANGO_SECRET_KEY",
 ↪  "django-insecure-solo-para-desarrollo")
DEBUG = os.environ.get("DJANGO_DEBUG", "1") == "1"
```

Ejercicio 8.8. Revisión de seguridad

Objetivo: identificar los ajustes inseguros del proyecto y repasar las medidas aplicadas en la unidad.

1. Ejecutad `python manage.py check --deploy` y contad los avisos.
2. Cambiad temporalmente `DEBUG = False` y `ALLOWED_HOSTS = ["127.0.0.1", "localhost"]`, repetid la comprobación y abrid http://127.0.0.1:8000/no-existe/ con el servidor en marcha. Dejad después los valores originales.
3. Completad esta tabla con la línea de código o el ajuste de «Incidencias» que aplica cada medida:

Medida	Dónde se aplica
Contraseñas con hash	
Vista de alta solo para usuarios autenticados	
Protección CSRF en los formularios	
API: escritura solo autenticada	
API: solo el autor modifica o borra	
El autor no lo elige el cliente	
Escapado de los datos en las páginas	
Orígenes de confianza para el cliente Vue	

Verificación. El paso 1 termina con `System check identified 7 issues (0 silenced).`, con los avisos W004, W008, W009, W012, W016, W018 y W020. En el paso 2 desaparecen W018 y W020 (5 avisos), y la dirección inexistente muestra una página «Not Found» genérica, sin la lista de rutas de la página de depuración. En la tabla del paso 3: `create_user()` y `set_password()`; `LoginRequiredMixin` en `IncidenciaCreateView`; `{% csrf_token %}` y `CsrfViewMiddleware`; `IsAuthenticatedOrReadOnly`; `EsAutorOSoloLectura`; `ReadOnlyField` en el serializador y `perform_create()` o `form_valid()`; el escapado automático de las plantillas; `CORS_ALLOWED_ORIGINS`.

> ***Para reflexionar.*** *Muchas de las protecciones de este apartado funcionan sin que el desarrollador haga nada (hash de contraseñas, CSRF, escapado, ORM parametrizado). ¿Qué riesgos tiene confiar en que «el framework ya lo hace»? ¿En qué puntos de la unidad ha habido que tomar decisiones de seguridad de forma consciente?*

Capítulo 9

Cierre de la unidad

9.1 Resumen

- **Python** tiene tipado dinámico y fuerte, bloques delimitados por la sangría, colecciones integradas (listas, tuplas, diccionarios y conjuntos), comprensiones, argumentos por nombre y valores por defecto, clases con `__init__` y `self`, excepciones con `try`/`except` y anotaciones de tipo opcionales que no se comprueban al ejecutar.
- Un **proyecto Django** reúne la configuración (`config/settings.py`, `config/urls.py`) y una o varias **aplicaciones**, que deben registrarse en `INSTALLED_APPS`. `manage.py` ejecuta las órdenes de administración y se pueden añadir órdenes propias.
- Las **rutas** (`path()`, convertidores, `include()`, `app_name`, `name`) asocian direcciones con **vistas**, de función o basadas en clase. Las direcciones se calculan a partir del nombre con `reverse()` y `{% url %}`.
- Las **plantillas** separan la presentación: variables, etiquetas, filtros, herencia con `{% extends %}` y `{% block %}`, ficheros estáticos y escapado automático.
- El **middleware** procesa todas las peticiones y respuestas en capas ordenadas. Los componentes de serie gestionan seguridad, sesiones, CSRF y autenticación; los propios se escriben como clases con `__init__(get_response)` y `__call__(request)`, y pueden cortar el recorrido.
- Los **modelos** describen los datos; el **ORM** los traduce a tablas. Las **migraciones** (`makemigrations`, `migrate`, `showmigrations`, `sqlmigrate`) llevan el historial de la estructura de la base de datos.
- Los **QuerySets** construyen consultas perezosas y encadenables: `filter`, `exclude`, `get`, búsquedas con `__`, relaciones, `order_by`, `Q`, `F`, `annotate`, `aggregate` y `select_related` para evitar las N+1 consultas.
- El **panel de administración** gestiona los datos a partir de los modelos registrados. Las **vistas genéricas** (`ListView`, `DetailView`, `CreateView`) y los **formularios** (`ModelForm`) resuelven las operaciones habituales, con validación en el formulario y en el método `clean()` del modelo.
- **Django REST Framework** construye una **API REST** con serializadores, ViewSets y routers, con filtros, paginación, códigos de estado HTTP correctos y una API navegable.
- La **seguridad** combina las protecciones del framework (hash de contraseñas, CSRF, escapado, ORM parametrizado) con decisiones del desarrollador: proteger vistas, elegir permisos, usar tokens, no confiar en los datos del cliente, configurar CORS y revisar `check --deploy`.

9.2 Glosario

Término	Definición
Tipado dinámico y fuerte	El tipo lo tiene el valor, no la variable, y no se mezclan tipos sin una conversión explícita
Comprensión	Expresión que construye una lista, un diccionario o un conjunto a partir de otra colección
Argumento por nombre	Argumento que se pasa indicando el nombre del parámetro (`max_length=120`)
Módulo / paquete	Fichero `.py` / carpeta de módulos con `__init__.py`
Type hint	Anotación opcional del tipo de una variable, un parámetro o un valor devuelto
Ajuste (*setting*)	Variable de configuración de `settings.py`
Orden de administración	Tarea que se ejecuta con `python manage.py nombre`
Ruta	Asociación entre un patrón de dirección y una vista
Convertidor	Parte de una ruta que reconoce un tipo de valor (`int`, `str`, `slug`…) y lo pasa a la vista
Espacio de nombres	Prefijo (`app_name`) que agrupa los nombres de las rutas de una aplicación
Vista basada en clase	Vista escrita como clase con un método por método HTTP
Vista genérica	Vista basada en clase de Django que resuelve una tarea habitual (`ListView`, `CreateView`…)
Plantilla / contexto	Fichero con marcas que se rellenan / diccionario de datos que se le pasa
Middleware	Componente que procesa todas las peticiones y respuestas de la aplicación
ORM	Capa que traduce entre objetos y tablas de una base de datos relacional
Modelo	Clase que describe una tabla y su comportamiento
Migración	Fichero que describe un cambio en la estructura de la base de datos
QuerySet	Consulta perezosa y encadenable sobre un modelo
N+1 consultas	Problema de rendimiento por hacer una consulta adicional por cada objeto de una lista
`ModelForm`	Formulario generado a partir de un modelo, que valida y guarda
API REST	Interfaz que expone recursos con direcciones, métodos HTTP, JSON y códigos de estado
Serializador	Componente que convierte objetos en JSON y valida los datos recibidos

Término	Definición
ViewSet / router	Clase con todas las operaciones de un recurso / generador de sus rutas
Autenticación / autorización	Comprobar quién es el usuario / decidir qué puede hacer
Hash	Transformación de un solo sentido con la que se guardan las contraseñas
CSRF	Ataque que envía peticiones en nombre de un usuario con sesión iniciada desde otro sitio
Token	Cadena aleatoria que identifica al usuario en las peticiones a la API
Inyección SQL	Ataque que altera una sentencia SQL a través de los datos del usuario
XSS	Ataque que inserta código HTML o JavaScript en las páginas que ven otros usuarios
CORS	Mecanismo por el que un servidor autoriza a otros orígenes a leer sus respuestas desde el navegador

9.3 Preguntas de autoevaluación

1. Escribid en Python, PHP y Java un bucle que muestre los elementos de una lista con su posición. ¿Qué diferencias hay?
2. ¿Qué diferencia hay entre `estados = otra_lista` y `estados = otra_lista.copy()`? ¿Por qué es peligroso usar una lista como valor por defecto de un parámetro?
3. ¿Qué ocurre si se crea una aplicación con `startapp` y no se añade a `INSTALLED_APPS`?
4. ¿Por qué se usa `{% url 'incidencias:detalle' incidencia.pk %}` en lugar de escribir `/incidencias/{{ incidencia.pk }}/`?
5. ¿Qué responde Django a `/incidencias/abc/` si la ruta es `incidencias/<int:pk>/`? ¿Se ejecuta la vista?
6. Explicad con un ejemplo cómo influye el orden de `MIDDLEWARE` cuando un componente corta el recorrido.
7. ¿Por qué un middleware que lee `request.user` antes de llamar a `get_response` debe ir después de `AuthenticationMiddleware`?
8. ¿Qué diferencia hay entre `makemigrations` y `migrate`? ¿Qué se sube al repositorio y qué no?
9. ¿Qué diferencia hay entre `null=True` y `blank=True`? ¿Cuál se usa en un campo de texto opcional?
10. ¿Qué hace `on_delete=models.PROTECT`? ¿Cómo se ha evitado que produzca un error 500 en la API?
11. Escribid la consulta que obtiene las categorías con su número de incidencias abiertas, ordenadas de más a menos.
12. ¿Qué es el problema de las N+1 consultas y cómo se resuelve?
13. ¿Por qué la regla de la descripción mínima está en el método `clean()` del modelo? ¿Qué hubo que hacer para que la API también la aplicara?
14. ¿Qué códigos de estado devuelve la API al crear, al borrar, ante datos incorrectos, sin

credenciales y sin permiso?

15. ¿Qué diferencia hay entre `has_permission` y `has_object_permission`?
16. ¿Por qué las peticiones autenticadas con token no necesitan protección CSRF y las autenticadas con sesión sí?
17. ¿Qué protege CORS, y a quién? ¿Por qué `curl.exe` recibe los datos aunque el origen no esté permitido?
18. ¿Qué avisos de `check --deploy` desaparecen al poner `DEBUG = False` y rellenar `ALLOWED_HOSTS`? ¿Qué falta para los demás?

9.4 Referencias

- Documentación oficial de Python 3.13 (en español): https://docs.python.org/es/3.13/
- Tutorial oficial de Python: https://docs.python.org/es/3.13/tutorial/
- PEP 8, guía de estilo para el código Python: https://peps.python.org/pep-0008/
- Documentación oficial de Django 5.2: https://docs.djangoproject.com/es/5.2/
- Tutorial de Django, partes 1 a 7: https://docs.djangoproject.com/es/5.2/intro/tutorial01/
- Configuración (*settings*): https://docs.djangoproject.com/es/5.2/ref/settings/
- Órdenes de `manage.py`: https://docs.djangoproject.com/es/5.2/ref/django-admin/
- Órdenes de administración propias: https://docs.djangoproject.com/es/5.2/howto/custom-management-commands/
- Distribuidor de URL (rutas): https://docs.djangoproject.com/es/5.2/topics/http/urls/
- Vistas y atajos: https://docs.djangoproject.com/es/5.2/topics/http/views/ y https://docs.djangoproject.com/es/5.2/topics/http/shortcuts/
- Vistas basadas en clase: https://docs.djangoproject.com/es/5.2/topics/class-based-views/
- Lenguaje de plantillas: https://docs.djangoproject.com/es/5.2/ref/templates/language/ y etiquetas y filtros: https://docs.djangoproject.com/es/5.2/ref/templates/builtins/
- Ficheros estáticos: https://docs.djangoproject.com/es/5.2/howto/static-files/
- Middleware: https://docs.djangoproject.com/es/5.2/topics/http/middleware/ y referencia: https://docs.djangoproject.com/es/5.2/ref/middleware/
- Registro (*logging*): https://docs.djangoproject.com/es/5.2/topics/logging/
- Modelos: https://docs.djangoproject.com/es/5.2/topics/db/models/ y tipos de campo: https://docs.djangoproject.com/es/5.2/ref/models/fields/
- Migraciones: https://docs.djangoproject.com/es/5.2/topics/migrations/
- Consultas: https://docs.djangoproject.com/es/5.2/topics/db/queries/, agregaciones: https://docs.djangoproject.com/es/5.2/topics/db/aggregation/ y referencia de QuerySet: https://docs.djangoproject.com/es/5.2/ref/models/querysets/
- Panel de administración: https://docs.djangoproject.com/es/5.2/ref/contrib/admin/
- Formularios: https://docs.djangoproject.com/es/5.2/topics/forms/ y formularios de modelos: https://docs.djangoproject.com/es/5.2/topics/forms/modelforms/
- Autenticación: https://docs.djangoproject.com/es/5.2/topics/auth/default/ y contraseñas: https://docs.djangoproject.com/es/5.2/topics/auth/passwords/
- Protección CSRF: https://docs.djangoproject.com/es/5.2/ref/csrf/
- Seguridad en Django: https://docs.djangoproject.com/es/5.2/topics/security/
- Lista de comprobación de despliegue: https://docs.djangoproject.com/es/5.2/howto/deployment/checklist/
- Django REST Framework: https://www.django-rest-framework.org/
- Tutorial de DRF: https://www.django-rest-framework.org/tutorial/quickstart/

- Serializadores: https://www.django-rest-framework.org/api-guide/serializers/
- ViewSets y routers: https://www.django-rest-framework.org/api-guide/viewsets/ y https://www.django-rest-framework.org/api-guide/routers/
- Autenticación y permisos en DRF: https://www.django-rest-framework.org/api-guide/authentication/ y https://www.django-rest-framework.org/api-guide/permissions/
- Paginación y filtros en DRF: https://www.django-rest-framework.org/api-guide/pagination/ y https://www.django-rest-framework.org/api-guide/filtering/
- Códigos de estado en DRF: https://www.django-rest-framework.org/api-guide/status-codes/
- django-cors-headers: https://github.com/adamchainz/django-cors-headers
- CORS en MDN: https://developer.mozilla.org/es/docs/Web/HTTP/Guides/CORS
- OWASP Top 10: https://owasp.org/www-project-top-ten/